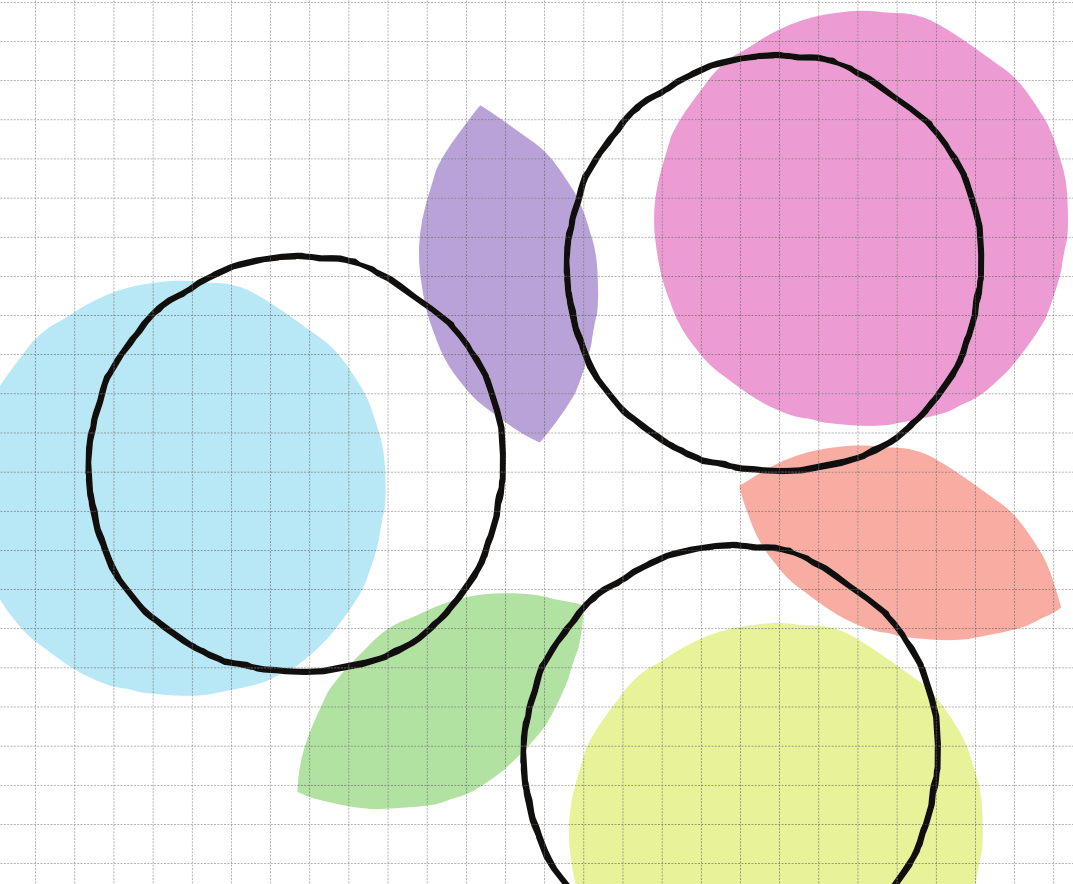
# Program Sketching with Types & Examples

Niek Mulleners



# Program Sketching with Types & Examples

**Niek Mulleners**

*ISBN:* 978-94-93539-73-0
*DOI:* 10.33540/3607
*Cover by:* Mirte Ebel and Niek Mulleners

# Program Sketching with Types & Examples

## Programma's Schetsen met Types & Voorbeelden

(met een samenvatting in het Nederlands)

## Proefschrift

ter verkrijging van de graad van doctor aan de
Universiteit Utrecht
op gezag van de
rector magnificus, prof. dr. ir. W. Hazeleger,
ingevolge het besluit van het College voor Promoties
in het openbaar te verdedigen op

dinsdag 22 september 2026 des middags te 4.15 uur

door


## Niek Mulleners

geboren op 31 december 1996
te Maastricht

**Promotor:**

Prof. dr. J.T. Jeuring

**Copromotoren:**

Dr. B.J. Heeren †

Dr. W.S. Swierstra

**Beoordelingscommissie:**

Prof. dr. G.K. Keller

Dr. N. Polikarpova

Prof. dr. ir. T. Schrijvers

Prof. dr. J. Voigtländer

Prof. dr. N. Wu

# Contents

# Part I

# Opening

Contents

# 1

# Introduction

Compilers must assist programmers when writing code. To do so, compilers need to understand the programmer's intent, which in turn requires programmers to make their intent explicit. In *How to Design Programs*, Felleisen et al. (2018) describe a systematic approach to program design, wherein the programmer is encouraged to express their intent by

1. thinking about the kind of data the program consumes and produces;
2. coming up with examples that illustrate the program's purpose; and
3. writing an outline of the program.

In statically typed functional programming languages, these ways of expressing intent correspond to type signatures, input-output examples, and *program sketches* (that is, unfinished programs that still contain holes, denoted by ◯). For example, a programmer might write the following:

```
pivot : Ord a ⇒ a → [a] → ([a], [a])          -- Type signature
pivot x xs = (◯, ◯)                           -- Sketch
assert pivot 3 [6,2,1,4,5] ≡≡ ([2,1], [6,4,5]) -- Input-output example
```

This piece of code cannot yet be executed, but it contains the programmer's intent. Can the compiler interpret this information to guide the programmer towards an executable program? This is precisely the goal of *program synthesis*: to (interactively) generate programs from incomplete specifications. Many synthesizers take a type, a set of input-output examples, and/or a sketch as inputs. Unfortunately, these three components rarely capture the full intent of the programmer. This means that program synthesizers may return a large number of possible solutions, and the burden of choosing the right one falls on the programmer.

Interactive synthesis can help alleviate this burden: by involving the programmer more directly in the synthesis process, the choices they have to make become more manageable. Top-down synthesizers gradually construct a program by stepwise refinement of a sketch, similar to how programmers often approach program development. This naturally supports

interactive synthesis, where the programmer is involved in deciding which refinements to apply.

Most top-down synthesizers are type-driven: they quickly enumerate programs based on their type, and discard any programs that do not match the input-output examples. As a result, intermediate programs have the right type, but they do not necessarily have the correct behavior. This is problematic for interactive synthesis, since the programmer may be asked to choose between refinements that will ultimately be discarded.

To address this issue, I will study the following research question:

**RQ.** *How can input-output examples be used to guide the search in top-down type-directed program synthesis, such that all intermediate steps are valid refinements?*

Some existing synthesizers use input-output examples to guide refinements. In particular, Myth (Osera and Zdancewic 2015) and $\lambda^2$ (Feser et al. 2015) work by preserving example information throughout the synthesis. Each refinement step propagates example information to the subgoals. This information is then used to discard refinements that contradict the input-output behavior.

While successful in their own right, Myth and $\lambda^2$ do not fully answer our research question, for several reasons:

1. Preservation of example information is only defined for some select refinements.
2. Not all incorrect refinements are discarded, particularly because the interaction between types and input-output examples is disregarded.
3. Examples are only used to prune, rather than guide the synthesis.

These shortcomings lead to several subquestions.

**SQ1.** *Can example information be preserved during sketching with arbitrary components?*

During component-based type-driven synthesis, the types of subgoals can be inferred as long as the types of components are known. Can the expected input-output behavior of subgoals be inferred if the implementations of components are known?

**SQ2.** *Can parametricity be used to guarantee that only feasible sketches are explored?*

Type-driven enumeration guarantees that only sketches of the correct type are explored. Similarly, equational reasoning can guarantee that sketches do not contradict the input-output behavior. To ensure that a sketch may lead to a solution with *both* the correct type and the right input-output behavior, one has to take the interaction between types and examples into account. In particular, *parametricity* (Reynolds 1983) describes the way types constrain the semantics of a program, and thus how types interact with input-output examples. Can we use parametricity to analyze the interaction between types and examples, thereby guaranteeing that synthesis is on the right track?

**SQ3.** *Can examples be used to decide which refinements to introduce?*

During type-driven synthesis, types describe which next-steps are possible, but not which ones are more likely to lead to a desirable solution. Can input-output examples guide this search, deciding which next steps are worth exploring, and in which order?

## 1.1 Contributions and Outline

Chapters 2, 3, and 4 detail my contributions to the field of type-and-example-directed program synthesis. These chapters are based on peer-reviewed articles and can be read in isolation. Each chapter is accompanied by an artifact containing the tools described in that chapter. Chapter 5 explores how feasibility reasoning might be used to find the right level of abstraction in various contexts. Chapter 6 concludes and presents avenues for future work.

The rest of this section gives an overview of Chapters 2, 3, and 4 and the papers they are based on.

**Chapter 2: Component-Based Synthesis Using Example Propagation.** This chapter explores the usage of evaluation-based example propagation for component-based type-and-example-directed synthesis. The programmer can provide a set of functions (imports) that should be used for synthesizing a solution. Example propagation ensures that only next steps conforming to the input-output examples are considered. This chapter is based on the publication

> **Niek Mulleners**, Johan Jeuring, and Bastiaan Heeren (2023). Program Synthesis Using Example Propagation. In *Proceedings of the 25th International Symposium on Practical Aspects of Declarative Languages*.

The main contributions are:

- We present Scrybe,[1] a tool for synthesizing programs using types, input-output examples and a set of components.
- We define a benchmark for component-based synthesis tasks.
- We show that evaluation-based example propagation can achieve similar speedups to handcrafted deduction rules in program synthesis.

**Chapter 3: Feasibility of Polymorphic Programs.** This chapter shows how to automate reasoning about the feasibility of programs implementing a polymorphic type and a set of input-output examples. The core idea is that many functions can be interpreted as container morphisms. Type and example constraints can be translated to constraints on container morphisms, allowing them to be solved by an SMT solver. This chapter is based on the publication

> **Niek Mulleners**, Johan Jeuring, and Bastiaan Heeren (2024). Example-Based Reasoning about the Realizability of Polymorphic Programs. In *Proceedings of the 29th ACM SIGPLAN International Conference on Functional Programming*.

which received a distinguished paper award.

The main contributions are:

[1] https://github.com/NiekM/scrybe

- We define a framework for automated reasoning about the feasibility of polymorphic programs with input-output examples.[2]
- We show how to use feasibility reasoning to automatically detect whether a function is a fold.
- We define a benchmark for automated type-and-example-based fold detection.

**Chapter 4: Type-and-Example-Driven Refinements.** This chapter shows how type-and-example-based feasibility reasoning can be made efficient enough to be used as a pruning tool during program synthesis. In addition, it describes how feasibility reasoning opens the doors for further analyses for pruning and shortcutting the synthesis search. This chapter is based on the publication

> **Niek Mulleners**, Johan Jeuring, and Wouter Swierstra (2026). Hole Refinements for Polymorphic Type-and-Example Driven Synthesis. In *Proceedings of the 2026 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*.

The main contributions are:

- We present Taxi,[3] a tool for efficient reasoning about the feasibility of polymorphic programs with input-output examples.
- We define an extended benchmark for type-and-example-based automated fold detection, showing that Taxi outperforms the SMT-based approach presented in Chapter 3.
- We present Driver,[3] a tactics-based synthesizer that
  - uses Taxi to guarantee that every intermediate state corresponds to a program conforming the specification;
  - exploits feasibility reasoning to allow additional synthesis components while avoiding additional branching; and
  - uses polymorphic example coverage checking to shortcut the synthesis.

**My Contributions.** I am the main author of these papers, and the sole author of the accompanying artifacts. I designed almost all solutions described in these papers and did the majority of the writing.

[2] https://github.com/NiekM/parametrickery.haskell
[3] https://github.com/NiekM/taxi-driver

Part II

# Main Content

Contents

# 2

# Component-Based Synthesis Using Example Propagation

We present Scrybe, an example-based synthesis tool for a statically-typed functional programming language, which combines top-down deductive reasoning in the style of $\lambda^2$ with Smyth-style live bidirectional evaluation. During synthesis, example constraints are propagated through sketches to prune and guide the search. This enables Scrybe to make more effective use of functions provided in the context. To evaluate our tool, it is run on the combined, largely disjoint, benchmarks of $\lambda^2$ and Myth. Scrybe is able to synthesize most of the combined benchmark tasks.

## 2.1 Introduction

Type-and-example-driven program synthesis is the process of automatically generating a program that adheres to a type and a set of input-output examples. The general idea is that the space of type-correct programs is enumerated, evaluating each program against the input-output examples until a program is found that does not result in a counterexample. Recent work in this field has aimed to make the enumeration of programs more efficient, using various pruning techniques and other optimizations. Hoogle+ (Guo et al. 2019) and Hectare (Koppel et al. 2022) explore efficient data structures to represent the search space. Smith and Albarghouthi (2019) describe how synthesis procedures can be adapted only to consider programs in normal form. MagicHaskeller (Katayama 2008) and RESL (Peleg et al. 2020) filter out programs that evaluate to the same result. Instead of only using input-output examples for the verification of generated programs, Myth (Osera and Zdancewic 2015, Frankle et al. 2016), Smyth (Lubin et al. 2020), $\lambda^2$ (Feser et al. 2015), and Igor II (Hofmann and Kitzelmann 2010) use input-output examples during pruning, by eagerly checking incomplete programs for counterexamples using constraint propagation.

**Constraint Propagation.** Top-down synthesis incrementally builds up a sketch, a program that may contain holes (denoted by ◯). Holes may be annotated with constraints, e.g. type constraints. During synthesis, holes are filled with new sketches (possibly containing more

holes) until no holes are left. For example, for type-directed synthesis, let us start from a single hole ⓪ annotated with a type constraint:

$$⓪ \texttt{ : [Nat] → [Nat]}$$

We may fill ⓪ using the function `map : ∀a b. (a → b) → [a] → [b]`, which applies a function to the elements of a list. This introduces a new hole ①, with a new type constraint:

$$⓪ \longmapsto \texttt{map } ① \qquad \textbf{where } ① \texttt{ : Nat → Nat}$$

We say that the constraint on ⓪ is propagated through `map` to the hole ①. Note that type information is preserved: the type constraint on ⓪ is satisfied exactly if the type constraint on ① is satisfied. We say that the hole filling $⓪ \longmapsto \texttt{map } ①$ refines the sketch with regard to its type constraint.

A similar approach is possible for example constraints, which partially specify the behavior of a function using input-output examples. For instance, we can further specify hole ⓪, to try and synthesize a program that doubles each value in a list:[4]

$$⓪ \vDash \texttt{[0,1,2]} \longmapsto \texttt{[0,2,4]}$$

Now, when introducing `map`, we expect its argument ① to be constrained by three input-output examples, representing the doubling of a natural number:

$$⓪ \longmapsto \texttt{map } ① \qquad \textbf{where } ① \vDash \left\{ \begin{array}{l} \texttt{0} \longmapsto \texttt{0} \\ \texttt{1} \longmapsto \texttt{2} \\ \texttt{2} \longmapsto \texttt{4} \end{array} \right\}$$

Similar to type constraints, we want example constraints to be correctly propagated through each hole filling, such that example information is preserved. Unlike with type constraints, which are propagated through hole fillings using type checking/inference, it is not obvious how to propagate example constraints through arbitrary functions. Typically, synthesizers define propagation of example constraints for a hand-picked set of functions and language constructs. Feser et al. (2015) define example propagation for a set of combinators, including `map` and `foldr`, for their synthesizer $\lambda^2$. Limited to this set of combinators, $\lambda^2$ excels at composition but lacks in generality. Myth (Osera and Zdancewic 2015, Frankle et al. 2016), and by extension Smyth (Lubin et al. 2020), takes a more general approach, in exchange for compositionality, defining example propagation for basic language constructs, including constructors and pattern matches.

**Presenting Scrybe.** In this chapter, we explore how the techniques of $\lambda^2$ and Smyth can be combined to create a general-purpose, compositional example-driven synthesizer, which we call Scrybe. Figure 1 shows four different interactions with Scrybe, where the function `dupli` is synthesized with different sets of functions. Scrybe is able to propagate examples through all of the provided functions using live bidirectional evaluation as introduced by Lubin et al. (2020) for their synthesizer Smyth, originally intended to support sketching (Solar-Lezama 2008, 2009). By choosing the right set of functions (for example, the set of combinators used in $\lambda^2$), Scrybe is able to cover different synthesis domains. Additionally,

[4] In this example, as well as in the rest of this chapter, we leave type constraints implicit.

```
import (...)
dupli : ∀ a. [a] → [a]
dupli xs = ◯
```

```
assert dupli []    ≡ []
assert dupli [0]   ≡ [0,0]
assert dupli [0,1] ≡ [0,0,1,1]
```

```
import (foldr)          ◯ ⟼ foldr (λx r. x:x:r) [] xs
import (concat, map)    ◯ ⟼ concat (map (λx. [x,x]) xs)
import (foldl, (++))    ◯ ⟼ foldl (λr x. r ++ [x,x]) [] xs
import (interleave)     ◯ ⟼ interleave xs xs
```

Figure 1. (Top) A program sketch in Scrybe for synthesizing the function `dupli`, which duplicates each value in a list. (Bottom) Different synthesis results returned by Scrybe, for different sets of imported functions.

allowing the programmer to choose this set of functions opens up a new way for the programmer to express their intent to the synthesizer, without having to provide an exact specification.

**Main Contributions.** The contributions of this chapter are as follows:

- We give an overview of example propagation and how it can be used to perform program synthesis (Section 2.2).
- We show how live bidirectional evaluation as introduced by Lubin et al. (2020) allows arbitrary sets of functions to be used as refinements during program synthesis (Section 2.3).
- We present Scrybe, an extension of Smyth (Lubin et al. 2020), and evaluate it against existing benchmarks from different synthesis domains (Section 2.4).

## 2.2 Example Propagation

Example constraints give a specification of a function in terms of input-output examples. To specify the behavior of `mult`, which multiplies two numbers, we may write the following constraint:

$$\left\{ \begin{array}{l} 0\ 1 \longmapsto 0 \\ 1\ 1 \longmapsto 1 \\ 2\ 3 \longmapsto 6 \end{array} \right\}$$

The constraint consists of three input-output examples. Each arrow ($\longmapsto$) maps the inputs on its left to the output on its right. A function can be checked against an example constraint by evaluating it on the inputs and matching the results against the corresponding outputs. During synthesis, we want to check that generated expressions adhere to this constraint. For example, to synthesize `mult`, we may generate a range of expressions of type `Nat → Nat → Nat` and then check each against the example constraint. The expression `λx y. double (plus x y)` will be discarded, as it maps the inputs to `2`, `4`, and `10`, respectively. It would be more efficient, however, to recognize that any expression of the form

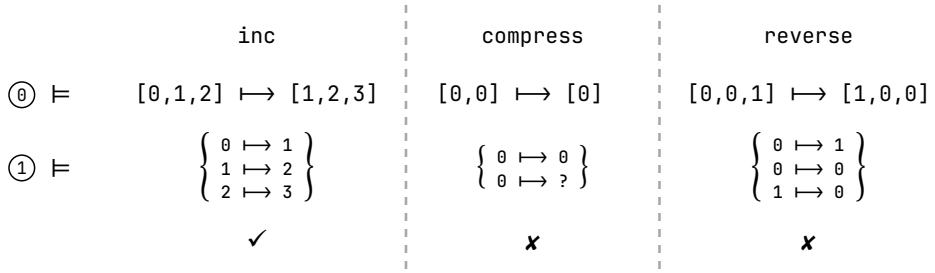



Figure 2. Inputs and outputs for example propagation through the hole filling ⓪ ⟼ `map` ①, for example constraints taken from the functions `inc`, `compress`, and `reverse`. The latter two cannot be implemented using `map`, which is reflected in the contradictory constraints: for `compress` there is a length mismatch, and for `reverse` the same input is mapped to different outputs.

`λx y. double e`, for some expression `e`, can be discarded, since there is no natural number whose double is `1`.

To discard incorrect expressions as early as possible, we incrementally construct a sketch, where each hole is annotated with an example constraint. Each time a hole is filled, its example constraint is propagated to the new holes and checked for contradictions. Let us start from a single hole ⓪. We refine the sketch by $\eta$-expansion, binding the inputs to the variables `x` and `y`.

$$\text{⓪} \longmapsto \texttt{λx y.}\ \text{①} \qquad \textbf{where}\ \text{①} \models \left\{\begin{array}{ll} \texttt{x} & \texttt{y} \\ \texttt{0} & \texttt{1} \longmapsto \texttt{0} \\ \texttt{1} & \texttt{1} \longmapsto \texttt{1} \\ \texttt{2} & \texttt{3} \longmapsto \texttt{6} \end{array}\right\}$$

A new hole ① is introduced, annotated with a constraint that captures the values of `x` and `y`. Example propagation through `double` should be able to recognize that the value `1` is not in the codomain of `double`, so that the hole filling ① ⟼ `double` ② can be discarded.

### 2.2.1 Program Synthesis Using Example Propagation

Program synthesizers based on example propagation iteratively build a program by filling holes. At each iteration, the synthesizer may choose to fill a hole using either a *refinement* or a *guess*. A *refinement* is an expression for which example propagation is defined. For example, $\eta$-expansion is a refinement, as shown in the previous example. To propagate an example constraint through a lambda abstraction, we simply bind the inputs to the newly introduced variables. A *guess* is an expression for which example propagation is *not* defined. The new holes introduced by a guess will not have example constraints. In a sense, guessing comes down to brute-force enumerative search. Refinements are preferred over guesses, since they preserve constraint information. It is, however, not feasible to define example propagation for every possible expression.

### 2.2.2 Handcrafted Example Propagation

One way to implement example propagation is to use handcrafted propagation rules on a per function basis. Consider, for example, `map`, which maps a function over a list. Refinement using `map` replaces a constraint on a list with constraints on its elements, while checking that the input and output lists have equal length and that equal values in the input list are mapped to equal values in the output list. Figure 2 illustrates how specific input-output examples (taken from constraints on common list functions) are propagated through `map`. The function `inc`, which increments each number in a list by one, can be implemented using `map`. As such, example propagation succeeds, resulting in a constraint that represents incrementing a number by one. The function `compress`, which removes consecutive duplicates from a list, cannot be implemented using `map`, since the input and output lists can have different lengths. As such, example propagation fails. The function `reverse`, which reverses a list, has input and output lists of the same length. It can, however, not be implemented using `map`, as `map` cannot take the positions of elements in a list into account. The resulting constraint is inconsistent, mapping `0` to two different values.

For their tool $\lambda^2$, Feser et al. (2015) provide such handcrafted propagation in terms of deduction rules for a set of combinators, including `map`, `foldr`, and `filter`. This allows $\lambda^2$ to efficiently synthesize complex functions in terms of these combinators. For example, $\lambda^2$ is able to synthesize a function computing the Cartesian product in terms of `foldr`, believed to be the first functional pearl (Danvy and Spivey 2007). Feser et al. show that synthesis using example propagation is feasible, but $\lambda^2$ is not general-purpose. Many synthesis problems require other recursion schemes or are defined over different types. Example propagation can be added for other functions in a similar fashion by adding new deduction rules, but this is very laborious.

### 2.2.3 Example Propagation for Algebraic Datatypes

As shown by Osera and Zdancewic (2015) in their synthesizer Myth, example constraints can be propagated through constructors and pattern matches, as long as they operate on algebraic datatypes. To illustrate this, we will use Peano-style natural numbers (we use the literals `0`, `1`, `2`, etc. as syntactic sugar):

```
data Nat = Zero | Succ Nat
```

**Constructors.** To propagate a constraint through a constructor, we have to check that all possible outputs agree with this constructor. For example, the constraint `Zero` can be refined by the constructor `Zero`. No constraints need to be propagated, since `Zero` has no arguments. In the next example, the constraint on ⓪ has multiple possible outputs, depending on the value of `x`.

$$⓪ \vDash \left\{ \begin{array}{l} \texttt{x} \\ \texttt{...} \longmapsto \texttt{Succ Zero} \\ \texttt{...} \longmapsto \texttt{Succ (Succ Zero)} \end{array} \right\}$$

Since every possible output is larger than zero, the constraint can be propagated through `Succ` by removing one `Succ` constructor from each output, i.e. decreasing each output by one.

$$\textcircled{0} \mapsto \texttt{Succ}\ \textcircled{1} \qquad \textbf{where}\ \textcircled{1} \vDash \left\{ \begin{array}{l} \texttt{x} \\ \texttt{...} \mapsto \texttt{Zero} \\ \texttt{...} \mapsto \texttt{Succ Zero} \end{array} \right\}$$

The resulting constraint on ① cannot be refined by a constructor, since the outputs do not all agree.

**Pattern Matching.** The elimination of constructors (i.e. pattern matching) is a bit more involved. For now, we will only consider non-nested pattern matches, where the scrutinee has no holes. Consider the following example, wherein the sketch `double = λn. ⓪` is refined by propagating the constraint on ⓪ through a pattern match on the local variable `n`.

$$\textcircled{0} \vDash \left\{ \begin{array}{l} \texttt{n} \\ \texttt{0} \mapsto \texttt{0} \\ \texttt{1} \mapsto \texttt{2} \\ \texttt{2} \mapsto \texttt{4} \end{array} \right\}$$

Pattern matching on `n` creates two branches, one for each constructor of `Nat`, with holes on the right-hand side. The constraint on ⓪ is propagated to each branch by splitting up the constraint based on the value of `n`. For brevity, we leave `n` out of the new constraints. The newly introduced variable `m` is exactly one less than `n`, i.e. one `Succ` constructor is stripped away.

$$\textcircled{0} \mapsto \textbf{case}\ \texttt{n}\ \textbf{of}\ \{\ \texttt{Zero} \to \textcircled{1};\ \texttt{Succ m} \to \textcircled{2}\ \}$$

$$\textbf{where}\ \textcircled{1} \vDash \texttt{0};\ \textcircled{2} \vDash \left\{ \begin{array}{l} \texttt{m} \\ \texttt{0} \mapsto \texttt{2} \\ \texttt{1} \mapsto \texttt{4} \end{array} \right\}$$

**Tying the Knot.** By combining example propagation for algebraic datatypes with structural recursion, Myth is able to perform general-purpose, propagation-based synthesis. To illustrate this, we show how Myth synthesizes the function `double`, starting from the previous sketch. Hole ① is easily refined with `Zero`. Hole ② can be refined with `Succ` twice, since every output is at least 2:

$$\textcircled{2} \mapsto \texttt{Succ}\ (\texttt{Succ}\ \textcircled{3}) \qquad \textbf{where}\ \textcircled{3} \vDash \left\{ \begin{array}{l} \texttt{m} \\ \texttt{0} \mapsto \texttt{0} \\ \texttt{1} \mapsto \texttt{2} \end{array} \right\}$$

At this point, to tie the knot, Myth should introduce the recursive call `double m`. Note, however, that `double` is not yet implemented, so we cannot directly test the correctness of this guess. We can, however, use the original constraint (on ⓪) as a partial implementation of `double`. The example constraint on ③ is a subset of this original constraint, with `m` substituted for `n`. This implies that `double m` is a valid refinement. This property of example constraints, i.e. that the specification for recursive calls is a subset of the original constraint, is known as *trace-completeness* (Osera and Zdancewic 2015), and is a prerequisite for synthesizing recursive functions in Myth.

### 2.2.4 Evaluation-Based Example Propagation

In their synthesizer Smyth, Lubin et al. (2020) extend Myth with sketching, i.e. program synthesis starting from a sketch, a program containing holes. Lubin et al. define evaluation-based example propagation, in order to propagate global example constraints through the sketch, after which Myth-style synthesis takes over using the local example constraints.

Take, for example, the constraint `[0,1,2]` $\longmapsto$ `[0,2,4]`, which represents doubling each number in a list. The programmer may provide the sketch `map` ◯ as a starting point for the synthesis procedure. Unlike $\lambda^2$, Smyth does not provide a handcrafted rule for `map`. Instead, Smyth determines how examples are propagated through functions based on their implementation.

The crucial idea is that the sketch is first evaluated, essentially inlining all function calls[5] until only simple language constructs remain, each of which supports example propagation. Omar et al. (2019) describe how to evaluate an expression containing holes using live evaluation. The sketch is applied to the provided input, after which `map` is inlined and evaluated as far as possible:

$$\texttt{map}\ \bigcirc\ \texttt{[0, 1, 2]} \quad \rightsquigarrow \quad \texttt{[}\bigcirc\ \texttt{0, }\bigcirc\ \texttt{1, }\bigcirc\ \texttt{2]}$$

At this point, the constraint can be propagated through the resulting expression. Lubin et al. (2020) extend Myth-style example propagation to work for the primitives returned by live evaluation. The constraints propagated to the different occurrences of ◯ in the evaluated expression are collected into a single constraint:

$$\bigcirc \vDash \left\{ \begin{array}{l} \texttt{0} \longmapsto \texttt{0} \\ \texttt{1} \longmapsto \texttt{2} \\ \texttt{2} \longmapsto \texttt{4} \end{array} \right\}$$

This kind of example propagation based on evaluation is called *live bidirectional evaluation*. For a full description, see (Lubin et al. 2020). Smyth uses live bidirectional evaluation to extend Myth with sketching. Note, however, that Smyth does not use live bidirectional evaluation to introduce refinements during synthesis.

## 2.3 Program Synthesis Using Example Propagation

Inspired by Smith and Albarghouthi (2019), we give a high-level overview of program synthesis using example propagation as a set of guarded rules that can be applied non-deterministically, shown in Figure 3. We keep track of a set of candidate programs $\mathscr{P}$, which is initialized by the rule init and then expanded by the rule expand until the rule final applies, returning a solution.

The rule init initializes $\mathscr{P}$ with a single hole $\textcircled{0}$, given that the initial constraint $\varphi_0$ is not contradictory. Each invocation of the rule expand non-deterministically picks a program $p$ from $\mathscr{P}$ and fills one of its holes $\textcircled{i}$ using a refinement $f$ from the domain $\mathscr{D}_i$, given that the constraint $\varphi_i$ can be propagated through $f$. The new program is considered a valid candidate and added to $\mathscr{P}$. As an invariant, $\mathscr{P}$ only contains programs that do not conflict with the

[5] Note that function calls within the branches of a stuck pattern match are not inlined.

$$\frac{\varphi_0 \neq \bot}{\mathscr{P} \leftarrow \{\ \textcircled{0}\ \}}\ \textsc{init}$$

$$\frac{p \in \mathscr{P} \quad \textcircled{i} \in \text{holes}(p) \quad f \in \mathscr{D}_i \quad \textcircled{i} \vDash \varphi_i \xrightarrow{f} f(\textcircled{1} \vDash \varphi_1, \ldots, \textcircled{n} \vDash \varphi_n)}{\mathscr{P} \leftarrow \mathscr{P} \cup \{\ [\textcircled{i} \mapsto f(\textcircled{1}, \ldots, \textcircled{n})]p\ \}}\ \textsc{expand}$$

$$\frac{p \in \mathscr{P} \quad \text{holes}(p) = \emptyset}{p \text{ is a solution}}\ \textsc{final}$$

Figure 3. Program synthesis using example propagation as a set of guarded rules that can be applied non-deterministically.

$$\begin{aligned}
\textcircled{0} &\longmapsto \texttt{map (}\lambda\texttt{x. }\textcircled{1}\texttt{) }\textcircled{2}\\
\textcircled{1} &\longmapsto \texttt{Succ }\textcircled{3}\\
\textcircled{2} &\longmapsto \texttt{xs}\\
\textcircled{3} &\longmapsto \texttt{x}
\end{aligned}$$

Figure 4. A set of hole fillings synthesizing an expression that increments each value in a list, starting from the sketch $\lambda$`xs.` $\textcircled{0}$.

original constraint $\varphi_0$. A solution to the synthesis problem is therefore simply any program $p \in \mathscr{P}$ that contains no holes.

To implement a synthesizer according to these rules, we have to make the non-deterministic choices explicit: we have to decide in which order programs are expanded ($p \in \mathscr{P}$); which holes are selected for expansion ($\textcircled{i} \in \text{holes}(p)$); and how refinements are chosen ($f \in \mathscr{D}_i$). How each of these choices is made is described in Section 2.3.1, Section 2.3.2, and Section 2.3.3, respectively.

### 2.3.1 Weighted Search

To decide which candidate expressions are selected for expansion, we define an order on expressions by assigning a weight to each of them. We implement $\mathscr{P}$ as a priority queue and select expressions for expansion in increasing order of their weight. The weight of an expression can be seen as a heuristic to guide the synthesis search. Intuitively, we want expressions that are closer to a possible solution to have a lower weight.

Firstly, we assign a weight to each *application* in an expression. This leads to a preference for smaller expressions, which converge more quickly and are less prone to overfitting. Secondly, we assign a weight to each *lambda abstraction*, since each lambda abstraction introduces a variable, increasing the number of possible hole fillings. Additionally, this disincentivizes an overuse of higher-order functions, as they are always $\eta$-expanded. Lastly, we assign a weight to complex *hole constraints*, i.e. hole constraints that took very long to compute or that have a large number of disjunctions. We assume that smaller, simpler hole constraints are more easily resolved during synthesis and are thus closer to a solution.

### 2.3.2 Hole Order

Choosing in which order holes are filled during synthesis is a bit more involved. Consider, for example, the hole fillings in Figure 4, synthesizing the expression `λxs. map (λx. Succ x) xs` starting from `λxs. ⓪`. There are three different synthesis paths that lead to this result, depending on which holes are filled first. More specifically, ② can be filled independently of ① and ③, so it could be filled before, between, or after them. To avoid generating the same expression three times, we should fix the order in which holes are filled, so that there is a unique path to every possible expression.

Because our techniques rely heavily on evaluation, we let evaluation guide the hole order. After filling hole ⓪, we live evaluate.

$$\texttt{map (λx. ①) ②} \quad \rightsquigarrow \quad \texttt{case ② of ...}$$

At this point, evaluation cannot continue, because we do not know which pattern ② will be matched on. We say that ② blocks the evaluation. By filling ②, the pattern match may resolve and generate new example constraints for ①. Conversely, filling ① before ② does not immediately introduce any new constraints. Hence, we always fill blocking holes first. Blocking holes are easily found by live evaluating the expression against the example constraints. If there are no blocking holes, we simply fill holes in order of their appearance in the sketch.

### 2.3.3 Generating Hole Fillings

Hole fillings depend on the local context and the type of a hole, and may consist of constructors, pattern matches, variables, and function calls. To avoid synthesizing multiple equivalent expressions, we only generate expressions in $\beta$-normal, $\eta$-long form. An expression is in $\beta$-normal, $\eta$-long form exactly if no $\eta$-expansions or $\beta$-reductions are possible. During synthesis, we guarantee $\beta$-normal, $\eta$-long form by greedily $\eta$-expanding newly introduced holes and always fully applying functions, variables, and constructors. Consider, for example, the function `map`. To use `map` as a refinement, it is applied to two holes, the first of which is $\eta$-expanded:

$$\texttt{map (λx. ⓪) ①}$$

We add some syntactic restrictions to the generated expressions: pattern matches are non-nested and exhaustive; and we do not allow constructors in the recursive argument of recursion schemes such as `foldr`. These are similar to the restrictions on pattern matching and structural recursion in Myth (Osera and Zdancewic 2015, Frankle et al. 2016) and Smyth (Lubin et al. 2020). Additionally, we disallow some expressions that are not in normal form, somewhat similar to equivalence reduction as described by Smith and Albarghouthi (2019). To do so, our tool provides a handcrafted set of expressions that are not in normal form, which are prohibited during synthesis. Ideally, these sets of disallowed expressions would be taken from an existing data set (such as HLint[6]), or approximated using evaluation-based techniques such as QuickSpec (Claessen et al. 2010).

[6] https://github.com/ndmitchell/hlint

### 2.3.4 Pruning the Program Space

For a program $p \in \mathscr{P}$ and a hole $\textcircled{i} \in \text{holes}(p)$, we generate a set of possible hole fillings based on the hole domain $\mathscr{D}_i$. For each of these hole fillings, we try to apply the EXPAND rule. To do so, we must ensure that the constraint $\varphi_i$ on $\textcircled{i}$ can be propagated through the hole filling. We use evaluation-based example propagation in the style of SMYTH to compute hole constraints for the newly introduced holes and check these for consistency. If example propagation fails, we do not extend $\mathscr{P}$, essentially pruning the program space.

**Diverging Constraints.** Unfortunately, example propagation is not feasible for all possible expressions. Consider, for instance, the function `sum`. If we try to propagate a constraint through `sum` ◯, we first use live evaluation, resulting in the following partially evaluated result, with ◯ in a scrutinized position:

```
case ◯ of
  []   → 0
  x:xs → plus x (sum xs)
```

Unlike MYTH, SMYTH (and by extension SCRYBE) allows examples to be propagated through pattern matches whose scrutinee may contain holes, considering each branch separately under the assumption that the scrutinee evaluates to the corresponding pattern. This introduces disjunctions in the example constraint. Propagating `0` through the previous expression results in a constraint that cannot be finitely captured in our constraint language:

$$\bigcirc \vDash \texttt{[]} \;\vee\; \bigcirc \vDash \texttt{[0]} \;\vee\; \bigcirc \vDash \texttt{[0,0]} \;\vee\; \ldots$$

Without extending the constraint language it is impossible to compute such a constraint. Instead, we try to recognize that example propagation diverges by setting a maximum to the amount of recursive calls allowed during example propagation. If the maximum recursion depth is reached, we cancel example propagation.

Since example propagation through `sum` ◯ always diverges, we could decide to disallow it as a hole filling. This is, however, too restrictive, as example propagation becomes feasible again when the length of the argument to `sum` is no longer unrestricted. Take, for example, the following constraint, representing counting the number of `True`s in a list, and a possible series of hole fillings:

$$\textcircled{0} \vDash \left\{ \begin{array}{ll} \texttt{xs} & \\ \texttt{[False]} & \longmapsto \texttt{0} \\ \texttt{[False, True]} & \longmapsto \texttt{1} \\ \texttt{[True , True]} & \longmapsto \texttt{2} \end{array} \right\} \qquad \begin{array}{l} \textcircled{0} \longmapsto \texttt{sum}\ \textcircled{1} \\ \textcircled{1} \longmapsto \texttt{map}\ (\lambda\texttt{x.}\ \textcircled{2})\ \textcircled{3} \\ \textcircled{3} \longmapsto \texttt{xs} \end{array}$$

Trying to propagate through $\textcircled{0} \longmapsto \texttt{sum}\ \textcircled{1}$ diverges, since $\textcircled{1}$ could be a list of any length. At this point, we could decide to disregard this hole filling, but this would incorrectly prune away a valid solution. Instead, we allow synthesis to continue guessing hole fillings, until we get back on the right track: after guessing $\textcircled{1} \longmapsto \texttt{map}\ (\lambda\texttt{x.}\ \textcircled{2})\ \textcircled{3}$ and $\textcircled{3} \longmapsto \texttt{xs}$, the length of the argument to `sum` becomes restricted and example propagation no longer diverges:

$$\texttt{sum (map (λx. ②) xs)} \quad \texttt{where ②} \vDash \left\{ \begin{array}{l} \texttt{x} \\ \texttt{False} \longmapsto \texttt{0} \\ \texttt{True} \longmapsto \texttt{1} \end{array} \right\}$$

At this point, synthesis easily finishes by pattern matching on x. Note that, unlike $\lambda^2$, Myth, and Smyth, Scrybe is able to interleave refinements and guesses.

**Exponential Constraints.** Even if example propagation does not diverge, it can still take too long to compute or generate a disproportionately large constraint, slowing down the synthesis procedure. Lubin et al. (2020) compute the feasibility of an example constraint by first transforming it to disjunctive normal form (DNF), which may lead to exponential growth of the constraint size. For example, consider the function or, defined as follows:

```
or = λa b. case a of
  False → b
  True  → True
```

Propagating the example constraint True through the expression or ⓪ ① puts the hole ⓪ in a scrutinized position, resulting in the following constraint:

$$(\texttt{⓪} \vDash \texttt{False} \land \texttt{①} \vDash \texttt{True}) \lor \texttt{⓪} \vDash \texttt{True}$$

This constraint has size three (the number of hole occurrences). We can extend this example by mapping it over a list of length $n$ as follows:

```
map (λx. or ⓪ ①) [0,1,2,...] ⊨ [True, True, True, ...]
```

Propagation generates a conjunction of $n$ constraints that are all exactly the same apart from their local context, which differs in the value of $x$. This constraint, unsurprisingly, has size $3n$. Computing the disjunctive normal form of this constraint, however, results in a constraint of size of $2^n \times \frac{3}{2}n$, which is exponential.

In some cases, generating such a large constraint may cause example propagation to reach the maximum recursion depth. In other cases, example propagation succeeds, but returns such a large constraint that subsequent refinements will take too long to compute. In both cases, we treat it the same as diverging example propagation.

## 2.4 Evaluation

We have implemented our synthesis algorithm in the tool Scrybe. The code is available on Zenodo (Mulleners 2026a) for reproduction and on GitHub[7] for reuse. To evaluate Scrybe, we combine the benchmarks of Myth (Osera and Zdancewic 2015) and $\lambda^2$ (Feser et al. 2015). We ran the Scrybe benchmarks using the Criterion[8] library on an HP Elitebook 850 G6 with an Intel® Core™ i7-8565U CPU (1.80GHz) and 32 GB of RAM.

This evaluation is not intended to compare our technique directly with previous techniques in terms of efficiency, but rather to gain insight into the effectiveness of evaluation-based

[7] https://github.com/NiekM/scrybe
[8] https://github.com/haskell/criterion

Table 1. Benchmark for functions acting on lists. Each row describes a single benchmark task and the time it takes for each function to synthesize with example propagation (*EP*) and without (*NoEP*) respectively. Some tasks cannot be synthesized within 10 seconds, leading to a timeout (⊥).

| | | Runtime (ms) | | | | |
|---|---|---|---|---|---|---|
| name | -- Description | *EP* | *NoEP* | Myth | Smyth | $\lambda^2$ |
| add | -- Increment each value in a list by n | 4.70 | 5.65 | – | – | ✓ |
| append | -- Append two lists | 13.35 | 41.36 | ✓ | ✓ | ✓ |
| cartesian | -- The cartesian product | ⊥ | ⊥ | – | – | ✓ |
| compress | -- Remove consecutive duplicates from a list | ⊥ | ⊥ | ✓ | ✘ | – |
| concat | -- Flatten a list of lists | 7.54 | 8.12 | ✓ | ✓ | ✓ |
| copy_first | -- Replace each element in a list with the first | 30.71 | 20.55 | – | – | ✓ |
| copy_last | -- Replace each element in a list with the last | 14.89 | 28.00 | – | – | ✓ |
| delete_max | -- Remove the largest numbers from a list | 16.05 | 31.95 | – | – | ✓ |
| delete_maxs* | -- Remove the largest numbers from a list of lists | 1875.43 | ⊥ | – | – | ✓ |
| drop‡ | -- All but the first n elements of a list | 554.96 | ⊥ | ✓ | ✓ | – |
| dupli | -- Duplicate each element in a list | 6.43 | 7.60 | ✓ | ✓ | ✓ |
| even_parity | -- Whether a list has an odd number of Trues | 42.72 | 201.73 | ✓ | ✘ | – |
| evens | -- Remove any odd numbers from a list | 1.84 | 2.62 | – | – | ✓ |
| filter | -- The elements in a list that satisfy p | 80.25 | 162.70 | ✓ | ✓ | – |
| fold | -- A catamorphism over a list | 9.48 | 6.43 | ✓ | ✓ | – |
| head† | -- The first element of a list | 1.40 | 2.20 | ✓ | ✓ | – |
| inc | -- Increment each value in a list by one | 5.38 | 24.36 | ✓ | ✓ | – |
| incs | -- Increment each value in a list of lists by one | 12.79 | 19.73 | – | – | ✓ |
| index‡ | -- Index a list starting at zero | 79.01 | 2956.53 | ✓ | ✓ | – |
| init† | -- All but the last element of a list | 115.59 | 453.28 | – | – | ✓ |
| last† | -- The last element of a list | 9.63 | 15.14 | ✓ | ✓ | ✓ |
| length | -- The number of elements in a list | 1.33 | 2.16 | ✓ | ✓ | ✓ |
| map | -- Map a function over a list | 2.82 | 4.75 | ✓ | ✓ | – |
| maximum | -- The largest number in a list | 120.38 | 231.44 | – | – | ✓ |
| member | -- Whether a number occurs in a list | 212.45 | 1145.07 | – | – | ✓ |
| nub | -- Remove duplicates from a list | 450.56 | 9245.26 | – | – | ✓ |
| reverse* | -- Reverse a list | 131.04 | 574.82 | ✓ | ✓ | – |
| set_insert | -- Insert an element in a set | ⊥ | ⊥ | ✓ | ✓ | – |
| shiftl | -- Shift all elements in a list to the left | 723.97 | 525.29 | – | – | ✓ |
| shiftr | -- Shift all elements in a list to the right | 369.27 | 620.67 | – | – | ✓ |
| snoc | -- Add an element to the end of a list | 6.77 | 55.76 | ✓ | ✓ | ✓ |
| sum | -- The sum of all numbers in a list | 4.36 | 17.05 | ✓ | ✓ | ✓ |
| sums | -- The sum of each nested list in a list of lists | 69.71 | 677.12 | – | – | ✓ |
| swap* | -- Swap the elements in a list pairwise | ⊥ | ⊥ | ✓ | ✓ | – |
| tail† | -- All but the first element of a list | 1.65 | 5.43 | ✓ | ✓ | – |
| take‡ | -- The first n elements of a list | 462.50 | ⊥ | ✓ | ✓ | – |
| to_set | -- Sort a list, removing duplicates | 37.06 | 41.18 | ✓ | ✓ | – |

example propagation as a pruning technique, as well as to show the wide range of synthesis problems that Scrybe can handle.

For ease of readability, the benchmark suite is split up into a set of functions operating on lists (Table 1) and a set of functions operating on binary trees (Table 2). We have excluded functions operating on just booleans or natural numbers, as these are all trivial and synthesize in a few milliseconds. For consistency, and to avoid naming conflicts, the

Table 2. Benchmark for functions acting on binary trees. Each row describes a single benchmark task and the time it takes for each function to synthesize with example propagation (*EP*) and without (*NoEP*) respectively. Some tasks cannot be synthesized within 10 seconds, leading to a timeout (⊥).

| | | Runtime (ms) | | | | |
|---|---|---|---|---|---|---|
| name | -- Description | *EP* | *NoEP* | Myth | Smyth | $\lambda^2$ |
| cons_nodes | -- Prepend an element to each list in a tree of lists | 3.38 | 5.08 | – | – | ✓ |
| find | -- Whether a number occurs in a tree | 907.81 | ⊥ | – | – | ✓ |
| flatten | -- Flatten a tree of lists into a list | 68.34 | 72.29 | – | – | ✓ |
| height | -- The height of a tree | 7.01 | 34.89 | – | – | ✓ |
| inc_nodes | -- Increment each element in a tree by one | 1.40 | 2.21 | – | – | ✓ |
| inorder | -- Inorder traversal of a tree | 15.77 | 18.28 | ✓ | ✓ | ✓ |
| insert | -- Insert an element in a binary tree | ⊥ | ⊥ | ✓ | ✘ | – |
| leaves | -- The number of leaves in a tree | 14.77 | 45.67 | ✓ | ✓ | ✓ |
| level‡ | -- The number of nodes at depth n | ⊥ | ⊥ | ✓ | ✘ | – |
| map_tree | -- Map a function over a tree | 3.38 | 11.77 | ✓ | ✓ | – |
| max_node | -- The largest number in a tree | 25.02 | 114.68 | – | – | ✓ |
| postorder | -- Postorder traversal of a tree | 19.50 | 44.79 | ✓ | ✘ | – |
| preorder | -- Preorder traversal of a tree | 9.09 | 21.10 | ✓ | ✓ | – |
| search | -- Whether a number occurs in a tree of lists | 1218.10 | 5253.07 | – | – | ✓ |
| select | -- All nodes in a tree that satisfy p | 652.12 | 1268.76 | – | – | ✓ |
| size | -- The number of nodes in a tree | 31.48 | 252.63 | ✓ | ✓ | ✓ |
| snoc_nodes | -- Append an element to each list in a tree of lists | 57.13 | 2156.82 | – | – | ✓ |
| sum_lists | -- The sum of each list in a tree of lists | 19.32 | 285.03 | – | – | ✓ |
| sum_nodes | -- The sum of all nodes in a tree | 18.21 | 89.11 | – | – | ✓ |
| sum_trees | -- The sum of each tree in a list of trees | 1000.26 | ⊥ | – | – | ✓ |

names of some of the benchmarks are changed to better reflect the corresponding functions in the Haskell prelude. To avoid confusion, each benchmark function comes with a short description.

Each row describes a single synthesis problem in terms of a function that needs to be synthesized. The first two columns give the name and a short description of this function. The third and fourth columns show, in milliseconds, the average time Scrybe takes to correctly synthesize the function with example propagation (*EP*) and without example propagation (*NoEP*), respectively. Some functions fail to synthesize within 10 seconds, leading to a timeout (⊥). The last three columns show, for Myth, Smyth, and $\lambda^2$, respectively, whether the function synthesizes (✓), fails to synthesize (✘), or is not included in their benchmark (–).

Each benchmark uses the same context as the original benchmark (if the benchmark occurs in both Myth and $\lambda^2$, the context from the Myth benchmark is chosen). Additionally, benchmarks from Myth use a catamorphism in place of structural recursion, except for `set_insert` and `insert`, which use a paramorphism instead.

The benchmarks `head`, `tail`, `init`, and `last` are all partial functions (marked †). We do not support partial functions; therefore, these functions are replaced by their total counterparts by wrapping their return type in `Maybe`. For example, `last` is defined as follows, where the outlined hole filling is the result returned by Scrybe (input-output constraints are omitted for brevity):

```
import (foldr)
last : ∀ a. [a] → Maybe a
last xs = ◯
```

Scrybe

```
◯ ⟼ foldr (λx r. case r of
          Nothing → Just x
          Just y → r) Nothing xs
```

The benchmarks `drop`, `index`, `take`, and `level` (marked ‡) all recurse over two datatypes at the same time. As such, they cannot be implemented using `foldr` as it is used in Section 2.3.3. Instead, we provide a specialized version of `foldr` (called `foldr+`) whose type is specialized to take an extra argument. For example, for `take`:

```
import (foldr+)
take : ∀ a. Nat → [a] → [a]
take n xs = ◯
```

Scrybe

```
◯ ⟼ foldr+ (λx r m. case m of
          Zero → []
          Succ o → x : r o
        ) (λm. []) xs n
```

A few functions (marked `*`) could not straightforwardly be adapted from the benchmarks of Myth and $\lambda^2$:

- The function `delete_maxs` replaces `delete_mins` from the $\lambda^2$ benchmark. The original function `delete_mins` requires a total function in scope that returns the minimum number in a list. This is not possible for natural numbers, as there is no obvious number to return for the empty list.
- The function `swap` (from the Myth benchmark) should be implemented using nested pattern matching. Neither a catamorphism (a fold) or a paramorphism can adequately mimic this behavior, leading `swap` to timeout.
- The function `reverse` occurs several times in the Myth benchmark, where it is synthesized using different techniques. Since these techniques do not have a clear counterpart in Scrybe, we have combined them into a single function.

### 2.4.1 Results

Scrybe is able to synthesize most of the combined benchmarks of Myth and $\lambda^2$, with a median runtime of 19 milliseconds. Furthermore, synthesis with example propagation is on average 5.1 times as fast as without example propagation, disregarding the benchmarks where synthesis without example propagation failed. $\lambda^2$ noticed a similar improvement (6 times as fast) for example propagation based on automated deduction, which indicates that example propagation using live-bidirectional evaluation is similar in effectiveness, while being more general.

Some functions benefit especially from example propagation, in particular those that can be decomposed into smaller problems. Take, for example, the function `snoc_nodes`, which can be decomposed into `map_tree` and `snoc`. By preserving example constraints between these subproblems, Scrybe greatly reduces the search space.

```
import (map_tree, foldr, ...)
snoc_nodes : ∀ a. Tree [a] → Tree [a]
snoc_nodes x t = ◯
```

Scrybe

```
◯ ⟼ map_tree (λxs. foldr (λy r. y:r) [x] xs) t
```

On the other hand, for some functions, such as `shiftl`, synthesis is noticeably faster without example propagation, showing that the overhead of example propagation sometimes outweighs the benefits. This indicates that it might be helpful to use some heuristics to decide when example propagation is beneficial. A few functions that fail to synthesize, such as `compress`, do synthesize when a simple sketch is provided:

```
import (foldr, (==), ...)
compress : [Nat] → [Nat]
compress xs = foldr (λx r. ⓪) ①
```

Scrybe

```
⓪ ⟼ x : case r of
          []   → r
          y:ys → if x == y then ys else r
① ⟼ []
```

Since our evaluation is not aimed at sketching, we still denote `compress` as failing ($\bot$) in the benchmark.

## 2.5 Conclusion

We presented an approach to program synthesis using example propagation that specializes in compositionality, by allowing arbitrary functions to be used as refinement steps. One of the key ideas is to hold on to constraint information as long as possible, rather than resorting to brute-force, enumerative search. Our experiments show that we are able to synthesize a wide range of synthesis problems from different synthesis domains.

There are many avenues for future research. One direction we wish to explore is to replace the currently ad hoc constraint solver with a more general-purpose SMT solver. Our hope is that this paves the way for the addition of primitive data types such as integers and floating point numbers.

# 3

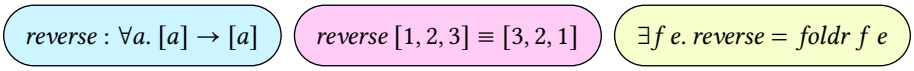
# Feasibility of Polymorphic Programs

Parametricity states that polymorphic functions behave the same regardless of how they are instantiated. When developing polymorphic programs, Wadler's free theorems can serve as *free specifications*, which can turn otherwise partial specifications into total ones, and can make otherwise feasible specifications *infeasible*. This is of particular interest to the field of program synthesis, where the infeasibility of a specification can be used to prune the search space. In this chapter, we focus on the interaction between parametricity, input-output examples, and sketches. Unfortunately, free theorems introduce universally quantified functions that make automated reasoning difficult. Container morphisms provide an alternative representation for polymorphic functions that captures parametricity in a more manageable way. By using a translation to the container setting, we show how reasoning about the infeasibility of polymorphic programs with input-output examples can be automated.

## 3.1 Introduction

The design of a program typically starts by specifying knowledge about the problem (Felleisen et al. 2018). Often, this knowledge is in the form of *types*, *input-output examples*, and *sketches* (incomplete programs containing holes). An example of each of these specifications for the function *reverse* is shown in Figure 5. Types, examples, and sketches can help a programmer formulate a mental model of the problem at hand. Many modern languages also have native support for specifying types, examples, and sketches through type annotations, assertions, and holes. These specifications allow the programmer to express their intent to a programming environment and, in return, the environment could check whether the programmer's mental model is correct and catch possible mistakes early. To do so, the environment has to reason about the *feasibility* of a program as specified by the programmer, i.e. whether a program adhering to this specification exists.[9]

[9] In this chapter, as opposed to the publication it is based on, we use the term *feasibility* rather than *realizability*. Both are used to describe specifications for which a solution exists. In recent work, however, the term realizability tends to refer to the existence of a solution in the context of a specific grammar (Kim

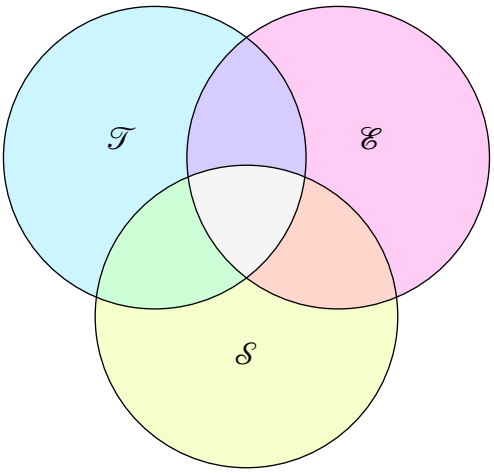



(a) Type (b) Input-output example (c) Sketch

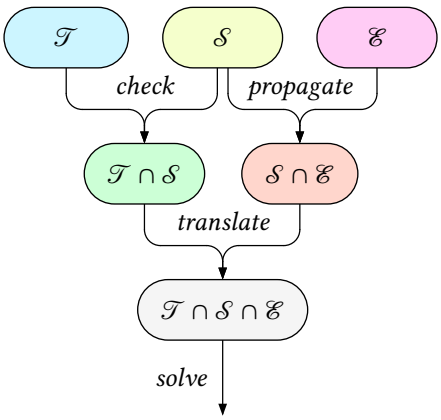
Figure 5. A specification for the function *reverse*.

A type is feasible exactly if it is inhabited (Urzyczyn 1997). If a type is uninhabited, such as the type in Figure 7a, no program of that type exists. A set of input-output examples is feasible exactly if the examples do not contradict each other. If a set of input-output examples is contradictory, such as the examples in Figure 7b, no program implementing those examples exists. Feasibility of a type and feasibility of a set of examples do not imply feasibility of the combination! We are interested in exploring whether the intersections of program spaces as specified by types, examples, and sketches are empty. These intersections are visualized in Figure 6a.

**Types and Examples.** The interaction between types and examples can be fairly subtle. Take, for example, the specification in Figure 7c. Both the type and example are individually feasible, but *parametricity* tells us that the only function with this type is the identity function (Reynolds 1983, Wadler 1989), contradicting the input-output example. We are not aware of prior work that investigates the feasibility of polymorphic programs with input-output examples. One reason for this could be that types and examples (as opposed to sketches) are typically assumed to be ground truth. This assumption does not hold when



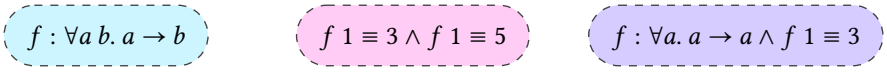

(a) The intersections of programs of type $\tau$, implementing the set of examples $\varepsilon$, and refining the sketch $\varsigma$.




(b) The pipeline for computing the feasibility of a type $\tau$, a set of examples $\varepsilon$, and a sketch $\varsigma$.

Figure 6. $\mathscr{T}$ is the set of programs of type $\tau$. $\mathscr{E}$ is the set of programs implementing the examples $\varepsilon$. $\mathscr{S}$ is the set of programs refining the sketch $\varsigma$.

et al. 2023). As the reasoning in this chapter does not consider the grammar of the language, we have opted to use the slightly more general term feasibility.




(a) An uninhabited type (b) Contradicting examples (c) Parametricity at work

Figure 7. Specifications that are infeasible.

those types and examples are generated automatically. In particular, we will consider types and examples as propagated through a sketch.

**Types and Sketches.** Types and program sketching go hand in hand. Typed holes have recently been embraced by statically typed functional programming languages such as Haskell (Gissurarson 2018, GHC Team 2023), Agda (Norell 2009), and Hazel (Omar et al. 2017). Holes allow partial programs to type check, while providing clear subgoals (hole types) for finishing those programs.

**Examples and Sketches.** Whereas examples are typically used as unit tests for complete programs, recent work has explored how and when incomplete programs (sketches) can be checked against input-output examples, resulting in new input-output examples on the holes (Osera and Zdancewic 2015, Feser et al. 2015, Lubin et al. 2020). We refer to this process as *example propagation*.

**Types, Examples, and Sketches.** When inferring types and input-output examples for the holes of a sketch, reasoning about the feasibility of those types and examples becomes a worthy avenue, since the feasibility of a sketch relies on the feasibility of its holes. For example, assume a programmer specifies *reverse* as in Figure 5, but instead incorrectly writes the sketch $reverse = map\ f$. To finish the program, there has to exist a function $f$ with the right semantics. In other words, the feasibility of *reverse* with this sketch relies on the feasibility of $f$. The sketch can be evaluated against the input-output example by inlining the definitions of *map* and equality on lists:

$$map\ f\ [1,2,3] \equiv [3,2,1] \Longrightarrow [f\ 1, f\ 2, f\ 3] \equiv [3,2,1] \Longrightarrow f\ 1 \equiv 3 \wedge f\ 2 \equiv 2 \wedge f\ 3 \equiv 1$$

From the type of *reverse*, the type of $f$ is inferred to be $a \to a$, for some fixed $a$. As we saw before, the only function of this type is the identity function, which contradicts the input-output examples. The infeasibility of $f$ allows us to correctly conclude that the sketch $reverse = map\ f$ is infeasible.

**Parametricity.** It is possible to prove the infeasibility of *reverse* in terms of *map* directly using Reynolds' abstraction theorem (Reynolds 1983) or Wadler's free theorems (Wadler 1989). However, automating such proofs is nontrivial and, in general, undecidable. We want a decision procedure that decidably computes whether a specification is feasible or not. Intuitively, a parametrically polymorphic function cannot *inspect* or *produce* polymorphic elements, only "pass them around". This intuition is perfectly captured by *container functors* and *container morphisms* (Abbott et al. 2005), which provide normal forms for polymorphic datatypes and polymorphic functions respectively.

In this chapter, we propose a general procedure for computing the feasibility of polymorphic programs: first, *type checking* and *example propagation* are used to infer types and input-

output examples for the holes of a sketch; then, the resulting constraints are *translated* to the container setting; lastly, the translated constraints are *solved* using an SMT solver. This procedure is visualized in Figure 6b. More specifically, our contributions are to show how to

- compute the feasibility of polymorphic types with monomorphic input-output examples (Section 3.3);
- use example propagation to extend this reasoning to sketching with higher-order combinators such as *map* (Section 3.3.4);
- reason about input-output traces to support sketching with recursion schemes such as *foldr* (Section 3.4);
- generalize the notion of trace-completeness to take parametricity into account (Section 3.4.2).

## 3.2 Background: Containers

Any list can be uniquely represented by its length $n$ and a function $f$ assigning a value to every index in $\{0, 1, ..., n-1\}$. For example, the list $[A, B, C]$ can be represented by $n = 3$, $f\ 0 = A$, $f\ 1 = B$, and $f\ 2 = C$. We say that a list of length $n$ is a *container* with *shape* $n$ and *positions* $\{0, 1, ..., n-1\}$. Many datatypes can similarly be represented in terms of their shape and positions.

Containers (Abbott et al. 2003) present a generic way of modeling datatypes that *contain* elements, making the distinction between the structure and the content of a datatype explicit. A container $S \triangleright P$ (sometimes written $(s : S) \triangleright P_s$) is defined by a type of shapes $S$, representing the structure of the datatype, and for every shape $s$ a type of positions $P_s$, describing where the elements are located.

Definition 1. *The list container is defined as* $\mathbb{N} \triangleright \mathrm{Fin}$, *where* $\mathrm{Fin}\ n = \{0, 1, ..., n-1\}$, *i.e. the type of natural numbers smaller than n. A list* $[x_0, ..., x_{n-1}]$ *is represented using the list container as a pair* $(n, \lambda i.\ x_i)$.

Functors that can be defined as containers are called *container functors*. A container functor $F$ is isomorphic to the *extension* of some container $S \triangleright P$, defined as $[\![S \triangleright P]\!]\ a = \Sigma_{(s:S)}.\ (P_s \to a)$. In other words, if $F$ is a container functor corresponding to the container $S \triangleright P$, values of type $F\ a$ can be represented by a pair $(s, p)$, where $s$ is a shape of type $S$ and $p$ is a function assigning an element of type $a$ to every position of type $P_s$.

### 3.2.1 Constructing Containers

The simplest container is the identity container, having only a single shape and a single position.

Definition 2. *The identity container is defined as* $\mathbb{1} \triangleright K\ \mathbb{1}$, *where* $\mathbb{1}$ *is the unit type with a single element* $\star$ *and* $K$ *is the constant function, i.e.* $K\ x\ y = x$. *A value* $x : a$ *is represented using the identity container as a pair* $(\star, K\ x)$.

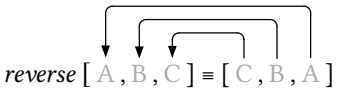
Containers can be combined to create more complex containers. A pair of lists, for example, has two natural numbers $n_1$ and $n_2$ as its shape (the lengths of the lists) and exactly $n_1 + n_2$ positions where elements are stored.

Definition 3. *The product of two containers $S \triangleright P$ and $T \triangleright Q$ is defined as a container $((s,t) : S \times T) \triangleright (P_s + Q_t)$. Given that the container representations of $x$ and $y$ are $(s,p)$ and $(t,q)$ respectively, the pair $(x,y)$ is represented as $((s,t), p \oplus q)$, where $(p \oplus q)\,(\mathbf{inl}\ z) = p\ z$ and $(p \oplus q)\,(\mathbf{inr}\ z) = q\ z$.*

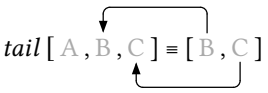
Using combinators like these, containers can be used to construct all *strictly positive types*, which can intuitively be understood as all types that have a tree-like structure. Additionally, the translation between values of inductively defined strictly positive types and their container representation can be automated (Abbott et al. 2005). As a convention, we use $\hat{\bullet}$ to denote the translated shape component and $\mathring{\bullet}$ to denote the translated position component. For example, the functor $F$ corresponds to the container $\hat{F} \triangleright \mathring{F}$ and a value $x$ of type $F\,a$ is translated to a pair $(\hat{x}, \mathring{x})$.

### 3.2.2 Container Morphisms

Given a list $[x_0, x_1, ..., x_{n-1}]$, the reverse of that list is $[x_{n-1}, x_{n-2}, ..., x_0]$. Using the container representation of lists, the reverse of the list $(n, \lambda i.\ x_i)$ is the list $(n, \lambda i.\ x_{n-i-1})$. Note how we can describe list reversal by specifying, for each index $i$ in the output list, where the element at that index comes from in the input list: the index $n - i - 1$. We can use arrows to visualize where each element comes from:




Similarly, the tail of the list $(n, \lambda i.\ x_i)$ is the list $(n-1, \lambda i.\ x_{i+1})$:




This idea of defining a function in terms of how the shape changes and where each element in the output comes from is formalized using *container morphisms*. A container morphism between the containers $S \triangleright P$ and $T \triangleright Q$ is a pair of functions $(u, g)$, where $u : S \to T$ maps input shapes to output shapes and $g : \Pi_{s:S}\,(Q_{(u\ s)} \to P_s)$ maps output positions to input positions (Abbott et al. 2003). Notice again the explicit separation between structure and content. The *extension* of a container morphism $(u, g)$ (written $\langle\!\langle u, g \rangle\!\rangle$) is a function mapping $(s, p)$ to $(u\ s, p \circ g_s)$. We can define *reverse* and *tail* as the extension of a container morphism as follows:

$$\text{reverse} = \langle\!\langle (\lambda n.\ n), (\lambda n\ i.\ n - i - 1) \rangle\!\rangle \qquad \text{tail} = \langle\!\langle (\lambda n.\ n - 1), (\lambda n\ i.\ i + 1) \rangle\!\rangle$$

A container morphism represents a natural transformation between container functors. Moreover, every natural transformation between container functors can be defined as the extension of a container morphism (Abbott et al. 2005). Since polymorphic functions between functors are natural transformations, we can reason about the feasibility of a polymorphic function between container functors in terms of the feasibility of the corresponding container morphism.

### 3.2.3 Small Containers

*Small containers* (also known as *finitary containers*) are containers whose shapes have a decidable equality and whose positions are finite sets (Prince et al. 2008). The list container is an example of a small container. Even though a list may contain any number of elements, for any given shape it has a finite number of positions. All the containers in this chapter are small, enabling us to enumerate over their positions, which is crucial in decidably computing the feasibility of a program.

## 3.3 Feasibility of Input-Output Examples with Polymorphic Types

To reason about the feasibility of an input-output example $f\ x \equiv y$, where $f$ is a natural transformation, we assume that there exists a container morphism $(\hat{f}, \mathring{f})$ satisfying this example. To do so, we have to translate the input $x$ and the output $y$ to the container setting. If we can show that such a container morphism $(\hat{f}, \mathring{f})$ is not feasible, then neither is the natural transformation $f$.

### 3.3.1 Translating a Single Example

An input-output example for $f : \forall a.\, F\ a \to G\ a$ is defined by a triple $(\tau, x, y)$, where $\tau$ is a monomorphic type, $x$ is an input of type $F\ \tau$, and $y$ is an output of type $G\ \tau$, such that $f\ x \equiv y$. Given that $F$ and $G$ are container functors isomorphic to $[\![\hat{F} \triangleright \mathring{F}]\!]$ and $[\![\hat{G} \triangleright \mathring{G}]\!]$ respectively, we can translate $x$ and $y$ to the container setting. The input $x$ is translated to a pair $(\hat{x}, \mathring{x})$, where $\hat{x} : \hat{F}$ and $\mathring{x} : \mathring{F}_{\hat{x}} \to \tau$ and the output $y$ is translated to a pair $(\hat{y}, \mathring{y})$, where $\hat{y} : \hat{G}$ and $\mathring{y} : \mathring{G}_{\hat{y}} \to \tau$. Since $f$ is a natural transformation between $F$ and $G$, it corresponds to a container morphism $(\hat{f}, \mathring{f})$ between $\hat{F} \triangleright \mathring{F}$ and $\hat{G} \triangleright \mathring{G}$. There exists a function $f$ such that $f\ x \equiv y$ exactly if there exists a container morphism $(\hat{f}, \mathring{f})$ such that $\langle\!\langle \hat{f}, \mathring{f} \rangle\!\rangle\ (\hat{x}, \mathring{x}) \equiv (\hat{y}, \mathring{y})$:

$$\exists f.\ \underbrace{\forall a.\, f : F\ a \to G\ a}\ \wedge\ \underbrace{f\ x \equiv y}$$
$$\exists (\hat{f}, \mathring{f}).\ \underbrace{\langle\!\langle \hat{f}, \mathring{f} \rangle\!\rangle\ (\hat{x}, \mathring{x}) \equiv (\hat{y}, \mathring{y})}$$

To solve the equation $\langle\!\langle \hat{f}, \mathring{f} \rangle\!\rangle\ (\hat{x}, \mathring{x}) \equiv (\hat{y}, \mathring{y})$, we first rewrite it by applying the container morphism extension, resulting in $(\hat{f}\ \hat{x}, \mathring{x} \circ \mathring{f}) \equiv (\hat{y}, \mathring{y})$.[10] Using equivalence on containers, as well as function extensionality, we separate the *shape* constraints on $\hat{f}$ from the *position* constraints on $\mathring{f}$:

$$\underbrace{\hat{f}\ \hat{x} \equiv \hat{y}}_{\textit{shape}}\ \wedge\ \underbrace{\forall q.\, \mathring{x}\ (\mathring{f}\ q) \equiv \mathring{y}\ q}_{\textit{position}}$$

Next, we make the intermediate argument returned by $\hat{f}$ explicit by introducing an existential quantification:

[10] For the sake of readability, we leave the dependent argument to $\mathring{f}$ implicit, as it can be inferred from the type of its argument.

$$\underbrace{\hat{f}\ \hat{x} \equiv \hat{y}}_{\text{(a)}} \ \wedge\ \forall q.\, \exists p.\, \underbrace{\mathring{f}\ q \equiv p}_{\text{(b)}} \ \wedge\ \underbrace{\mathring{x}\ p \equiv \mathring{y}\ q}_{\text{(c)}} \tag{1}$$

The resulting equation consists of three components:

(a) An input-output example for $\hat{f}$, describing the feasibility of the shape morphism.
(b) A set of input-output examples for $\mathring{f}$, describing the feasibility of the position morphism.
(c) A consistency check with respect to parametricity, ensuring that each element in the output also occurs in the input. Note how $\forall q.\, \exists p.\, \mathring{x}\ p \equiv \mathring{y}\ q$ can be rewritten as $\mathit{codomain}(\mathring{y}) \subseteq \mathit{codomain}(\mathring{x})$.

Similar to how the consistency of input-output examples with respect to a sketch can be computed using example propagation, resulting in input-output examples for the subcomponents of the program (the holes), our translation to the container setting checks the consistency of input-output examples with respect to a polymorphic type, resulting in input-output examples for the abstract subcomponents of the program (the shape and position morphisms).

To figure out if $f$ is feasible, we have to solve the satisfiability of Equation 1, but this is complicated by the universal and existential quantifications. If we assume, however, that $\hat{F} \triangleright \mathring{F}$ and $\hat{G} \triangleright \mathring{G}$ are *small containers*, we can get rid of any quantifications by enumerating over the positions:

$$\hat{f}\ \hat{x} \equiv \hat{y} \ \wedge\ \bigwedge_q \bigvee_p \mathring{f}\ q \equiv p \ \wedge\ \mathring{x}\ p \equiv \mathring{y}\ q \tag{2}$$

When we know $\mathring{x}$ and $\mathring{y}$, the satisfiability of this formula is trivially decided by an SMT solver.

### 3.3.2 Example: Reverse as Map

To see how example consistency can be used to reason about the feasibility of a program, we will turn our attention once more to the example of *reverse* in terms of *map*:

$$\forall a.\, \mathit{reverse} : [a] \to [a] \ \wedge\ \exists f.\, \mathit{reverse} = \mathit{map}\ f \ \wedge\ \mathit{reverse}\ [A, B, C] \equiv [C, B, A]$$

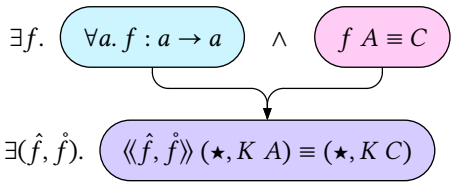
The first step in checking the feasibility of *reverse* is to propagate the type and input-output example through the sketch. The type of $f$ is $a \to a$, for a fixed $a$. To propagate the example, we evaluate the sketch applied to the input $[A, B, C]$ as far as possible, by inlining the definition of *map*:

$$\mathit{map}\ f\ [A, B, C] \quad \rightsquigarrow \quad [f\ A, f\ B, f\ C]$$

Then, we simplify the resulting equation using equality on lists:

$$[f\ A, f\ B, f\ C] \equiv [C, B, A] \quad \Longrightarrow \quad f\ A \equiv C \wedge f\ B \equiv B \wedge f\ C \equiv A$$

We will focus on the first conjunct (highlighted): intuitively, $f\ A \equiv C$ is inconsistent, because the $C$ in the output does not occur in the input of $f$. This implies that $f$ is infeasible, which we can prove formally by translating the example $f\ A \equiv C$ to the container setting. We interpret $f$ as a natural transformation $(\hat{f}, \mathring{f})$ between the identity functor and translate the values $A$ and $C$ accordingly:




We write out the resulting constraint on $(\hat{f}, \mathring{f})$ according to Equation 1 and simplify:

$$\underbrace{\hat{f}\ \star \equiv \star}_{\text{(a)}} \ \wedge \ \underbrace{\mathring{f}\ \star \equiv \star}_{\text{(b)}} \ \wedge \ \underbrace{A \equiv C}_{\text{(c)}}$$

The constraints on $\hat{f}$ and $\mathring{f}$ are trivially satisfied, but the consistency check (c) fails: the codomain of $K\ C$ is not a subset of the codomain of $K\ A$, i.e. $\{C\} \not\subseteq \{A\}$. We conclude that $f$, and, by extension, *reverse* as a *map*, is infeasible.

### 3.3.3 Translating Multiple Examples

The only way to show that a single input-output example is infeasible is by showing that it fails the consistency check (c). However, when considering multiple input-output examples, their feasibility additionally relies on the feasibility of their respective shape morphisms (a) and position morphisms (b).

A polymorphic function $f : \forall a.\ [a] \to [a]$ cannot *inspect* the elements in the input list and can therefore not differentiate between different input lists that have the same length (i.e. the same *shape*). When a set of input-output examples for $f$ requires inspecting the input elements, this will result in either a shape conflict on $\hat{f}$ or a position conflict on $\mathring{f}$. For example, assume that $f$ is supposed to sort the input list and remove any duplicates. This requires inspecting the elements in the list, so we expect to find a contradiction:

**Shape Conflict** The examples $f\ [A, A] \equiv [A] \wedge f\ [A, B] \equiv [A, B]$ imply the contradictory shape constraint $\hat{f}\ 2 \equiv 1 \wedge \hat{f}\ 2 \equiv 2$, leading to a *shape conflict*.

**Position Conflict** The examples $f\ [A, B] \equiv [A, B] \wedge f\ [B, A] \equiv [A, B]$ imply the contradictory position constraint $\mathring{f}\ 0 \equiv 0 \wedge \mathring{f}\ 0 \equiv 1$, leading to a *position conflict*.

### 3.3.4 Sketching with Map

Now that we have seen how to reason about the feasibility of sets of input-output examples, it is straightforward to extend this reasoning to any program $p$ specified with the sketch $\exists f.\ p = map\ f$, by recognizing that an input-output example for $p$ represents a set of input-output examples for $f$. To be precise, an input-output example for $p$ with an input list of length $n$ corresponds to exactly $n$ input-output examples for $f$. The complete pipeline for programs defined as a *map* is shown in Figure 8. Note how each step in the pipeline changes

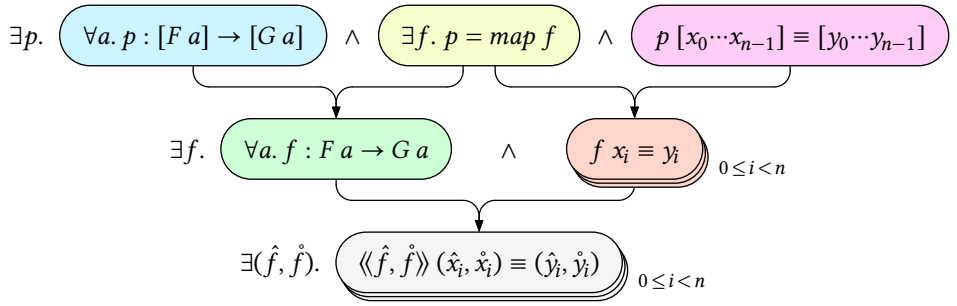



Figure 8. The pipeline for computing the feasibility of a polymorphic program defined as a *map* with a single input-output example.

the focus: first the whole program $p$; then the hole $f$; and finally the container morphism $(\hat{f}, \mathring{f})$.

**Soundness.** Feasibility reasoning is sound if we cannot draw false conclusions from its results. This is the case exactly if the initial equation in the pipeline is equivalent to the final equation. That way, a solution to the final equation implies that a correct program exists; and a contradiction implies that no such program exists. We argue that reasoning about the feasibility of a program as a *map* is sound, as all the steps shown in Figure 8 are valid rewritings.

**Completeness.** Feasibility reasoning is complete exactly if it is a decision procedure, i.e. it always returns either a solution or a contradiction. This is true if the final equation in the pipeline is in a decidable logic. Reasoning about the feasibility of a program as a *map* is complete as long as $F$ and $G$ are small container functors. That way, the final equation in Figure 8 is free of quantifiers, and each equation is an instantiation of Equation 2.

## 3.4 Reasoning About Recursion Schemes

One of the most ubiquitous functions in functional programming is *foldr*, a recursion scheme that embodies structural recursion over inductively defined datastructures. Gibbons et al. (2001) ask the question "When is a function a fold?" and show how to prove whether a function is indeed a fold. Intuitively speaking, and restricting ourselves to folds over lists, we say that a function $h$ is a fold if, for any element $x$ and any list $xs$, the result of $h\ (x : xs)$ is uniquely determined by $x$ and $h\ xs$. For example, the function *reverse* is a fold, since

$$reverse\ \ (x : xs) = reverse\ xs +\!\!+ [x]$$

The function $tail : \forall a.\ [a] \to [a]$ (taken from (Gibbons et al. 2001)), however, given by the equations

$$\begin{aligned} tail\ \ [] &= [] \\ tail\ \ (x : xs) &= xs \end{aligned}$$

is not a fold, because $tail\ xs$ throws away the first element of $xs$, which is needed in the tail of $x : xs$. In other words, *tail* is not a fold because there exist no $f$ and $e$ such that $tail = foldr\ f\ e$.

### 3.4.1 Feasibility of Tail

Hofmann (2010) shows how to detect whether a set of input-output examples specifies a fold. Similar results can be achieved using example propagation (Feser et al. 2015, Lubin et al. 2020). In this section, we will use a combination of example propagation and parametric reasoning to show that *tail* is *not* a fold using a minimal number of input-output examples.

To prove automatically that *tail* is not a fold, we will prove that $tail : \forall a.\ [a] \to [a]$ with the sketch $\exists f.\ tail = foldr\ f\ []$ is infeasible:[11]

$$\forall a.\, tail : [a] \to [a] \quad \wedge \quad \exists f.\, tail = foldr\ f\ [] \quad \wedge \quad \cdots$$

Whether and how we can prove the infeasibility of this specification depends on the exact input-output examples. For example, take the following input-output examples for *tail*:

$$tail\ [A, B, C] \equiv [B, C] \quad \wedge \quad tail\ [D, E] \equiv [E]$$

Since *foldr* uses structural recursion, the call $tail\ [A, B, C]$ relies on recursive calls to $[B, C]$, $[C]$, and $[]$. Because we cannot inspect the elements, we cannot distinguish the call to $[B, C]$ from the call to $[D, E]$. Since the call to $[D, E]$ returns $[E]$, throwing away the first element, the element $B$ is also thrown away in the call to $[B, C]$. This implies that the result of tail $[A, B, C]$ cannot contain the value $B$, contradicting our example.

Formally, the proof starts with propagating the type and input-output examples through the sketch. Given the types of *tail* and *foldr*, the type of $f$ is inferred to be $a \times [a] \to [a]$, for some fixed $a$. To propagate the example $tail\ [A, B, C] \equiv [B, C]$, we evaluate the sketch applied to the input $[A, B, C]$, by inlining the definition of *foldr*:

$$foldr\ f\ []\ [A, B, C] \quad \rightsquigarrow \quad f\ (A, f\ (B, f\ (C, [])))$$

Unfortunately, the resulting equation $f\ (A, f\ (B, f\ (C, []))) \equiv [B, C]$ is *not* an input-output example for $f$, but rather an input-output *trace*, specifying the trace of calls made to $f$ resulting in the output $[B, C]$. We will make the intermediate results returned by $f$ explicit by introducing two existentially quantified variables, revealing that an input-output trace represents a set of related input-output examples:

$$\exists x\ y.\ f\ (A, x) \equiv [B, C] \quad \wedge \quad f\ (B, y) \equiv x \quad \wedge \quad f\ (C, []) \equiv y$$

To translate these examples, we interpret $f$ as a natural transformation $(\hat{f}, \mathring{f})$ between the non-empty list container (i.e. the product of the identity container and the list container) and the list container. For each existentially quantified variable $x$, we introduce two existentially quantified variables $\hat{x}$ and $\mathring{x}$:

[11] Note that this defines $tail\ [] = []$.

$$\begin{aligned}\exists(\hat{x},\mathring{x})\,(\hat{y},\mathring{y}).\ \langle\!\langle \hat{f},\mathring{f}\rangle\!\rangle\,((\star,\hat{x}),K\ A\oplus\mathring{x}) &\equiv (2,\{0\mapsto B,1\mapsto C\})\\ \wedge\ \langle\!\langle \hat{f},\mathring{f}\rangle\!\rangle\,((\star,\hat{y}),K\ B\oplus\mathring{y}) &\equiv (\hat{x},\mathring{x})\\ \wedge\ \langle\!\langle \hat{f},\mathring{f}\rangle\!\rangle\,((\star,0),K\ C) &\equiv (\hat{y},\mathring{y})\end{aligned}$$

We first focus on the position component of $f\ (A,x)\equiv[B,C]$. After simplifying:

$$(K\ A\oplus\mathring{x})\,(\mathring{f}\ 0)\equiv B\ \wedge\ (K\ A\oplus\mathring{x})\,(\mathring{f}\ 1)\equiv C$$

The constant function $K\ A$ cannot return $B$ or $C$, implying that both $B$ and $C$ are in the codomain of $\mathring{x}$:

$$\{B,C\}\subseteq codomain(\mathring{x}) \tag{3}$$

In other words, both $B$ and $C$ are returned by the recursive call $tail\,[B,C]$. Given the polymorphic type of *tail*, this should conflict the constraint $tail\ \ [D,E]\equiv[E]$. To prove this, we make the input-output trace incurred by $tail\ \ [D,E]\equiv[E]$ explicit:

$$\exists z.\ f\ (D,z)\equiv[E]\ \ \wedge\ \ f\ (E,[])\equiv[]$$

Next, we will match up the shape components of the different calls to $f$: the input shapes of $f\ (C,[])$ and $f\ (E,[])$ are equal, so we can unify their output shapes $\hat{y}$ and $\hat{z}$; in doing so, the input shapes of $f\ (B,y)$ and $f\ (D,z)$ become equal, showing that $\hat{x}$ is equal to 1:

$$\left.\begin{array}{lll}f\ (C,[])\equiv y & \Longrightarrow \hat{f}\ (\star,0)\equiv\hat{y} & \left.\begin{array}{l}\\ \\ \end{array}\right\}\ \hat{y}\equiv\hat{z}\\ f\ (E,[])\equiv z & \Longrightarrow \hat{f}\ (\star,0)\equiv\hat{z} & \\ f\ (B,y)\equiv x & \Longrightarrow \hat{f}\ (\star,\hat{y})\equiv\hat{x} & \\ f\ (D,z)\equiv[E] & \Longrightarrow \hat{f}\ (\star,\hat{z})\equiv 1 & \end{array}\right\}\ \hat{x}\equiv 1$$

This implies that the domain of $\mathring{x} : Fin\,\hat{x}\to\tau$ is equal to $Fin\,1$. In other words, we used the shape component of $tail\,[D,E]\equiv[E]$ to show that the recursive call to $tail\,[B,C]$ returns a list with a single element. Naturally, this list cannot contain both $B$ and $C$, contradicting Equation 3 and thereby proving that *tail* is not a fold. Finding this contradiction by hand requires some effort, but this is trivial for an SMT solver.

**Generalizing Beyond Tail.** Adding a constant, monomorphic input to a set of examples does not affect feasibility. Hence, our proof readily extends to functions that generalize over *tail* using an additional argument:

- the function $drop : \mathbb{N}\to[a]\to[a]$, which drops the first $n$ elements from a list, specializes to *tail* by setting $n$ to 1;
- the function $removeAt : \mathbb{N}\to[a]\to[a]$, which removes an element from a list at index $i$, specializes to *tail* by setting $i$ to 0.

Neither *drop* nor *removeAt* can be implemented using the sketch $\lambda n.\ foldr\,(f\ n)\,[]$. There is, however, a way to implement *drop* and *removeAt* as a fold, using the sketch $\lambda n\ xs.\ foldr\ f\ e\ xs\ n$, which instantiates *foldr* such that the output type is $\mathbb{N}\to[a]$. See Section 3.5.1 for a short discussion of these different interpretations.

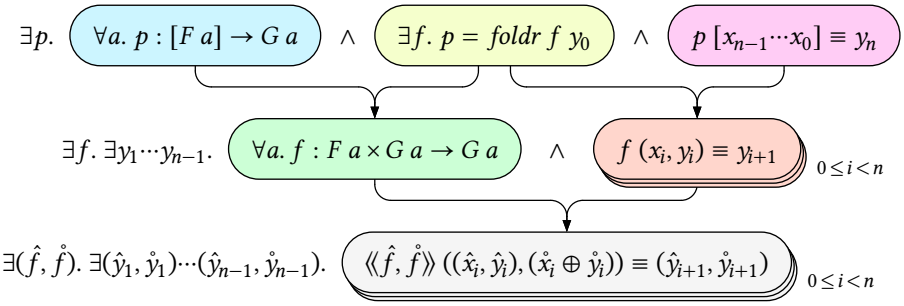



Figure 9. The pipeline for computing the feasibility of a polymorphic program defined as a fold with a single input-output example.

### 3.4.2 Sketching with Foldr

To reason about program traces generated by *foldr*, we look at how *foldr* iteratively builds up a result. During the evaluation of $foldr\ (+)\ 0\ [1, 2, 3]$, which after inlining corresponds to $1 + (2 + (3 + 0))$, two intermediate results are computed ($3 + 0 = 3$ and $2 + 3 = 5$) before the final result ($1 + 5 = 6$) is computed.[12] Note how each intermediate result is computed by combining the previous result with an element from the input list, going through the list from right to left (hence the name *foldr*, which stands for *right fold*). To reflect this, we write an input-output example for *foldr* as follows, numbering the elements in the list from right to left:

$$foldr\ f\ y_0\ [x_{n-1}, ..., x_0] \equiv y_n$$

The argument $y_0$ is the initial result and the output $y_n$ is the final result. There are $n - 1$ intermediate results $(y_1, ..., y_{n-1})$, which we can make explicit:

$$\begin{aligned} \exists y_1 \cdots y_{n-1}.\ & y_1 \equiv f\ (x_0, y_0) \\ \wedge\ & y_2 \equiv f\ (x_1, y_1) \\ & \vdots \\ \wedge\ & y_n \equiv f\ (x_{n-1}, y_{n-1}) \end{aligned}$$

This allows us to concisely define the input-output example on our fold as a set of input-output examples on $f$:

$$\exists y_1 \cdots y_{n-1}.\ \bigwedge_{0 \le i < n} f\ (x_i, y_i) \equiv y_{i+1}$$

Along with the inferred type of $f$, these input-output examples are translated to the container setting. The resulting pipeline for programs defined as a fold is shown in Figure 9.

**Soundness.** Using the same argument as in Section 3.3.4, reasoning about the feasibility of a program as a fold is sound, as all the steps shown in Figure 9 are valid rewritings.

[12] The exact order of evaluation depends on the evaluation strategy, but we can still reason about the intermediate results.

**Completeness.** When considering only a single input-output example, reasoning about the feasibility of a program as a fold is *not* complete. To see why this is the case, let us write out the final equation of Figure 9 to separate the *shape* and *position* constraints:

$$\bigwedge_{0 \leq i < n} \underbrace{\hat{f}\ (\hat{x}_i, \hat{y}_i) \equiv \hat{y}_{i+1}}_{shape} \ \wedge \ \underbrace{\forall q.\ (\mathring{x}_i \oplus \mathring{y}_i)\ (\mathring{f}\ q) \equiv \mathring{y}_{i+1}\ q}_{position} \tag{4}$$

Note that the values of the intermediate results $y_1, ..., y_{n-1}$ (and therefore their shapes $\hat{y}_1, ..., \hat{y}_{n-1}$) are not known. This means that we do not know the exact types of $q$ and cannot simply eliminate the universal quantifications by enumeration as we did in Equation 2. In other words, the feasibility of an input-output trace cannot be expressed in a quantifier-free logic.

If, however, all recursive calls are represented in the set of input-output examples (a property known as *trace-completeness* (Osera and Zdancewic 2015)), the values of the intermediate results would be known and all universal quantifiers could be eliminated. Moreover, since the types of universally quantified variables are only determined by the *shapes* of intermediate results, we can relax trace-completeness to only require that the shape component of each recursive call is represented in the example set. Such an example set is called *trace-complete modulo parametricity* (or *shape-complete*). For example, the following set of input-output examples is shape-complete:

$$tail\ [A, B, C] \equiv [B, C] \ \wedge \ tail\ [1, 2] \equiv [2] \ \wedge \ tail\ [True] \equiv []$$

To conclude, reasoning about the feasibility of a program as a fold is complete if $F$ and $G$ are small container functors and the set of input-output examples is shape-complete.

### 3.4.3 A Note on Scoping

In the previous sections, we have shown how to rephrase the question of whether a function is a fold as a feasibility problem. We have been very careful to state that a fold can be defined as *foldr f e*, and not just defined "in terms of *foldr*". As it turns out, any function on lists can be defined in terms of *foldr*! Take, for example, the following implementation for *tail*:

```
tail : ∀ a. [a] → [a]
tail xs = foldr (λx r → tail' xs) [] xs
    where tail' []     = []
          tail' (y:ys) = ys
```

More generally, any function `h` on lists for which an implementation `h'` exists can be implemented in terms of *foldr* in at least two different ways:

```
h xs = foldr (λx r. r)      (h' xs) xs
h xs = foldr (λx r. h' xs) (h' []) xs
```

The problem is that the algebra of a fold (represented in the function *foldr* by the first two arguments) should be independent of the input list. Hence, to figure out if a function is a

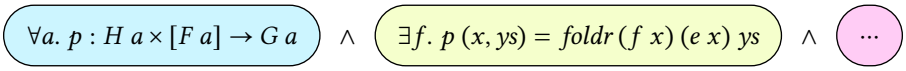
fold, we should make sure that $xs$ is not in scope of the arguments $f$ and $e$ when computing the feasibility of $foldr\ f\ e\ xs$.

## 3.5 Evaluation

To test how well our technique is able to check for feasibility, we will evaluate it on a benchmark of polymorphic Haskell functions, checking for each whether it can be implemented as a fold. To create this benchmark, we selected all functions from the Haskell prelude[13] that are natural transformations of type $\forall a.\, H\, a \times [F\, a] \to G\, a$, i.e. any total polymorphic function taking at least a list as input. We made small changes to some functions to make them fit these criteria:

- All functions with a `Foldable` constraint are specialized to `[]` (`concat`, `null`, `length`).
- Partial functions that return a list return the empty list for invalid inputs (`tail`, `init`).
- Other partial functions have their return type wrapped in `Maybe` (`head`, `last`, `index`).
- Since there are two ways to make the function `(++)` fit this scheme (i.e. we can try to fold over either of its input lists), we made both ways explicit in `append` and `prepend`.
- Functions with multiple type parameters are specialized to the same type parameter (`zip`, `unzip`).

The functions are listed in Table 3. For each function $p$ in the benchmark, we check if the sketch $\exists f.\, p\ (x, ys) = foldr\ (f\ x)\ (e\ x)\ ys$ is feasible:




Note how we use an extended version of the pipeline in Figure 9 that takes an additional argument of type $H\ a$. This argument is passed to the function $f$ and allows us to check the feasibility of functions taking additional arguments (beyond an input list). While there is no technical limitation that stops us from reasoning about multiple holes, for simplicity we assume that the base case $e\ x$ is given. As such, $e$ is not existentially quantified.

### 3.5.1 Additional Arguments

The functions `index`, `drop`, `take`, and `splitAt` take an integer as an argument in addition to the input list. There are multiple ways to interpret these functions as natural transformations between container functors. For example, for the function `drop`, we can either

- choose $H\ a = Int$, $F\ a = a$, and $G\ a = [a]$, keeping the integer argument constant throughout the fold;
- or choose $H\ a = ()$, $F\ a = a$, and $G\ a = Int \to [a]$, allowing a different integer to be passed to recursive calls.

In the first interpretation, `drop` is *not* a fold, but in the second interpretation it is:

```
drop : ∀ a. [a] → Int → [a]
drop = foldr f (const [])
  where f x r i = if i > 0 then r (i - 1) else x : r i
```

[13] https://hackage.haskell.org/package/base-4.19.1.0/docs/Prelude.html

Table 3. Benchmark computing the feasibility of Haskell prelude functions as folds. For each function, its type is given and whether it is a fold. The running time with shape-complete (SC) and shape-incomplete (SI) example sets is shown in milliseconds.

| `name` | `: Type` | Fold? | Runtime (ms) | |
|---|---|---|---|---|
| | | | SC | SI |
| `null` | `: ∀a. [a] → Bool` | ✓ | 106 | 98 |
| `length` | `: ∀a. [a] → Int` | ✓ | 110 | 99 |
| `head` | `: ∀a. [a] → Maybe a` | ✓ | 123 | 107 |
| `last` | `: ∀a. [a] → Maybe a` | ✓ | 121 | 107 |
| `tail` | `: ∀a. [a] → [a]` | ✘ | 120 | 130 |
| `init` | `: ∀a. [a] → [a]` | ✘ | 117 | 123 |
| `reverse` | `: ∀a. [a] → [a]` | ✓ | 157 | 144 |
| `index` | `: ∀a. Int → [a] → Maybe a` | ✘ | 131 | 132 |
| `drop` | `: ∀a. Int → [a] → [a]` | ✘ | 140 | 149 |
| `take` | `: ∀a. Int → [a] → [a]` | ✓ | 197 | 217 |
| `splitAt` | `: ∀a. Int → [a] → ([a], [a])` | ✓ | 566 | 545 |
| `append` | `: ∀a. [a] → [a] → [a]` | ✓ | 238 | 275 |
| `prepend` | `: ∀a. [a] → [a] → [a]` | ✓ | 237 | 240 |
| `zip` | `: ∀a. [a] → [a] → [(a, a)]` | ✓ | $223^{\dagger}$ | $234^{\dagger}$ |
| `unzip` | `: ∀a. [(a, a)] → ([a], [a])` | ✓ | $228^{\dagger}$ | $\bot$ |
| `concat` | `: ∀a. [[a]] → [a]` | ✓ | $247^{\dagger}$ | $318^{\dagger}$ |

We only consider the first interpretation in our benchmark, since the functor $Int \to [-]$ is *not* a small container, as there are an infinite number of positions for every shape. The same approach is taken for all functions that take an extra argument. This includes `index`, `take`, and `splitAt`, but also `append`, `prepend`, and `zip`. In general, any additional arguments can either be kept constant or not. Unless the argument type has a finite number of values, the resulting container functors are *not* small.

### 3.5.2 Input-Output Examples

Each function is tested on a shape-complete example set curated by the author (each consisting of 4 to 10 input-output examples), as well as a shape-incomplete example set constructed by removing every other example from the shape-complete example set.

### 3.5.3 Running Times

For each function, we automatically prove its (in)feasibility as a fold with shape-complete (SC) and shape-incomplete (SI) example sets. Each proof is performed 10 times and the average results are shown in Table 3.

Interestingly, apart from `unzip`, there is no significant difference in runtime between proofs with shape-complete and shape-incomplete example sets. Some proofs perform slightly faster with shape-incomplete example sets. This could be explained by a naive choice we made in the implementation, which seems to prevent the solver from effectively making use of the guarantees that shape-completeness provides (see Section 3.6.3 for a more in-

depth discussion). As a result, those shape-incomplete example sets seem to perform better simply because they have fewer input-output examples, and thus fewer constraints.

With the exception of `unzip` with shape-incomplete examples, all automated proofs finish in less than a second. These running times are fast enough for automated program analysis and feedback, but not yet for extensive pruning during synthesis. However, our tool could be used for top-level pruning, where feasibility is only checked at the start of a synthesis problem to figure out the right recursion scheme.

### 3.5.4 Naive Container Translations

The return type of the functions `zip`, `unzip`, and `splitAt` uses a product. When using a naive translation to container functors using Definition 3, computing the feasibility of these functions results in a timeout (marked †). Our tool uses a more efficient translation described in more depth in Section 3.6.3.

## 3.6 Implementation

We have implemented a tool for reasoning about the feasibility of polymorphic programs. All code is available on Zenodo (Mulleners 2026b) for reproduction and on GitHub[14] for reuse. The tool includes the pipelines from Figure 8 and Figure 9 and provides the basics for defining more feasibility problems. It is implemented in Haskell, as it allows us to reason about actual Haskell functions and values, rather than describe an intermediate language to reason about. For translating to SMT-LIB, we use the SBV library,[15] a well-documented library with many features that provides a statically typed API into SMT-LIB. We use Z3 (Moura and Bjørner 2008) as the underlying solver.

### 3.6.1 Encoding Containers

To encode a functor `f`, we have to associate it to its shape and position types. To do so, we define two type families `Shape` and `Position`.

```
type family Shape    (f :: Type → Type) :: Type
type family Position (f :: Type → Type) :: Type
```

For example, to associate the list functor `[]` to a shape and position (see Definition 1) we define:

```
newtype Fin = Fin Nat deriving (Eq, Ord, Num)
type instance Shape    [] = Nat
type instance Position [] = Fin
```

Note that Haskell does not support dependent types, so we overapproximate `Fin` as a newtype over natural numbers. Additionally, SMT-LIB has no native support for $\mathbb{N}$ and *Fin*, so we employ another type family `Sym` (for *sym*bolic representation) to associate our Haskell datatypes to types supported by SMT-LIB, along with a type class `Encode` for encoding values. Natively supported types are exactly those for which the constraint `SymVal` holds, which is exported by the SBV library.

[14] https://github.com/NiekM/parametrickery.haskell
[15] https://leventerkok.github.io/sbv/

```
type family Sym (a :: Type) :: Type
class SymVal (Sym a) ⇒ Encode a where
  encode :: a → SMT a
```

Both `Nat` and `Fin` are represented in the SMT solver as integers:

```
type instance Sym Nat = Integer
type instance Sym Fin = Integer
instance Encode Nat where encode = fromIntegral
instance Encode Fin where encode = fromIntegral
```

Of course, these are overapproximations, which we will resolve by describing how the symbolic representations should be constrained in the SMT solver. A value of type `Nat` is encoded as an integer $n$ constrained by $n \geq 0$ and an associated value of type `Fin` as an integer $m$ constrained by $m \geq 0 \wedge m < n$, for some natural number $n$. We refer to `Nat` (and other shape types) as refinement types and to `Fin` (and other position types) as dependent types.

The type class `Ref` describes how the symbolic representation of a refinement type is constrained in terms of a function `refine` returning a symbolic boolean `SBool`. Note that when calling `refine`, we have to disambiguate between multiple refinement types that have the same symbolic representation. To do so, we use visible dependent quantification (denoted `forall a →` ) as introduced by the `RequiredTypeArguments` extension,[16] to avoid having to use proxy arguments or ambiguous types.

```
class Encode a ⇒ Ref a where
  refine :: forall a → Sym a → SBool
```

By convention, operators that act on symbolic values start with a dot (`.`).

```
instance Ref Nat where
  refine _ n = n .≥ 0
```

For dependent types, we additionally define a type family `Arg` to associate them with the argument type they depend on. The type class `Dep` describes, for a dependent type, how its symbolic representation is constrained in terms of its argument.

```
type family Arg (a :: Type) :: Type
class (Encode a, Ref (Arg a)) ⇒ Dep a where
  depend :: forall a → Sym (Arg a) → Sym a → SBool

type instance Arg Fin = Nat
instance Dep Fin where
  depend _ n m = m .≥ 0 .&& m .< n
```

Using `Ref` and `Dep`, we can define a type dependency between a refinement type `t` and a dependent type `u` by stating that the argument type of `u` is equal to `t`.

```
type Dependent :: Type → Type → Constraint
type Dependent t u = (Ref t, Dep u, Arg u ~ t)
```

[16] https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/required_type_arguments.html

The type class `Container` describes an isomorphism between a container functor `f` and its extension, given a dependency between its shape and position types.

```
class Dependent (Shape f) (Position f) ⇒ Container f where
  toExtension   :: f a → Extension f a
  fromExtension :: Extension f a → f a
```

As described in Section 3.2, the extension of a container is a shape along with a function mapping positions to elements. In our implementation, we choose to represent this function as a dictionary, to emphasize that we are dealing with *small* containers.

```
data Extension f a = Extension
  { shape    :: Shape f
  , position :: Map (Position f) a
  }
```

Finally, we define `Container []` by giving a translation to and from the container extension.

```
instance Container [] where
  toExtension xs = Extension
    { shape    = genericLength xs
    , position = Map.fromList (zip [0..] xs)
    }
  fromExtension (Extension _ p) = Map.elems p
```

### 3.6.2 Encoding the Pipelines

For each of the pipelines in Figure 8 and Figure 9, we introduce a function that takes a list of input-output examples and produces a set of constraints to be passed to the SMT solver. All inputs and outputs are translated to their container representation. For each existentially quantified variable in the final equation of the pipeline, an uninterpreted function is introduced. These uninterpreted functions are then constrained as in Equation 1 and Equation 4, making sure to insert calls to `refine` and `depend` so that the functions are only constrained on their true domain. For example, a symbolic function of type $Int \to Int$ representing a shape morphism $\hat{f} : \mathbb{N} \to \mathbb{N}$ should not be constrained on negative inputs, and all its results should be nonnegative.

### 3.6.3 Efficient Encodings of Position Sets

In Section 3.4.2 we describe how a shape-complete example set leads to a quantifier-free formula because the quantified positions can be enumerated over. In our implementation, however, we have not made proper use of this observation. Rather, we simply describe how the shapes constrain the position types and leave it to the SMT solver to figure out that these constraints imply a finite set of positions. This approach turns out to be too naive, especially for position types that contain unions.[17]

For example, the function `splitAt` returns a pair of lists (i.e. it is a natural transformation to the functor `Product [] []`). Using the standard constructors for containers (as taken from Abbott et al. (2005)), this corresponds to the container

[17] Both product and sum containers use unions in their position types.

$$((n, m) : \mathbb{N} \times \mathbb{N}) \triangleright \mathit{Fin}\ n + \mathit{Fin}\ m$$

The position type contains a union, and using this container to compute the feasibility of `splitAt` in terms of `foldr` results in a timeout. However, an equivalent, more efficient container exists, namely the container

$$((n, m) : \mathbb{N} \times \mathbb{N}) \triangleright \mathit{Fin}\ (n + m)$$

By using this optimized container representation, our tool can efficiently compute the feasibility of `splitAt` in terms of `foldr`. Similarly, a list of pairs (the return type of `zip` and `unzip`) can efficiently be represented by the container $(n : \mathbb{N}) \triangleright \mathit{Fin}\ 2n$.

In our tool, these efficient containers are defined ad hoc, but they pave the way for a more general solution: every small container can be represented as $(s : S) \triangleright \mathit{Fin}\ (f\ s)$, where $f$ is some function that returns the number of positions given a shape $s$ of type $S$. The problem is that $f$ may be difficult to turn into a constraint that is efficiently solved by an SMT solver. When the example set is shape-complete, however, all shapes are known, making it possible to compute $f\ s$ before translating to SMT-LIB. This way, both shapes and positions could be represented in SMT-LIB as integers. We leave the exploration of this approach to future work.

## 3.7 Related Work

### 3.7.1 Typed-Directed Program Synthesis

Many program synthesizers use types to constrain the search space (Katayama 2008, Osera and Zdancewic 2015, Feser et al. 2015, Polikarpova et al. 2016, Frankle et al. 2016, Inala et al. 2017, Osera 2019, Lubin et al. 2020, Koppel et al. 2022, Gissurarson et al. 2023, Lee and Cho 2023). Scrybe (Chapter 2) also takes this approach. By inferring synthesis rules from typing rules, synthesized programs are type correct by construction (Osera and Zdancewic 2015, Inala et al. 2017). Polymorphic types additionally constrain the search space, by restricting the production forms to those that make no assumptions about the types, allowing synthesizers to implicitly benefit from the abstractions provided by parametricity (Reynolds 1983, Wadler 1989). This is emphasized when polymorphic types are combined with other specifications: often, parametricity can turn an otherwise partial specification into a total specification (Polikarpova et al. 2016, Frankle et al. 2016, Osera 2019). However, none of these synthesizers explicitly take the interaction between polymorphic types and other specifications into account.

### 3.7.2 Reasoning About Polymorphic Functions

Parametrically polymorphic functions behave the same regardless of how they are instantiated. This property is known as *parametricity*. Reynolds (1983) captures parametricity in his abstraction theorem, by giving a relational interpretation to types. Wadler (1989) shows how this interpretation can be used to derive free theorems for functions based solely on their types. Abbott et al. (2003, 2005) define the notion of *containers* and show that any polymorphic function between strictly positive types corresponds to a morphism between containers. Prince et al. (2008) use morphisms between containers to prove properties about

polymorphic functions. Seidel and Voigtländer (2010) extend this reasoning to ad hoc polymorphic functions. Bernardy et al. (2010) show how properties of polymorphic functions can be faithfully tested on a single monomorphic instance. However, these techniques only reason about the correctness of complete programs, rather than the feasibility of incomplete programs.

### 3.7.3 Example Propagation

Whereas input-output examples are commonly used to test the correctness of complete programs, they can often also be used to reason about the feasibility of incomplete programs, a process known as *example propagation*. This is of particular interest to top-down enumerative synthesizers, for pruning the search space. Example propagation is typically implemented using specific deduction rules (Hofmann and Kitzelmann 2010, Osera and Zdancewic 2015, Feser et al. 2015, Frankle et al. 2016, Osera 2019), live evaluation (Omar et al. 2019, Lubin et al. 2020), or function inversion (Teegen et al. 2021, Lee and Cho 2023). In this chapter, example propagation for *map* and *foldr* is defined in an ad hoc manner, not unlike the deduction rules used by Feser et al. (2015).

### 3.7.4 Unrealizability

The feasibility of a program restricted to a particular grammar is often referred to as *realizability*. In recent years, a lot of advancements have been made in reasoning about the unrealizability of programs in the context of syntax-guided synthesis (SyGuS) (Hu and D'Antoni 2018, Hu et al. 2019, Hu et al. 2020) and semantics-guided synthesis (SemGuS) (Kim et al. 2021, Kim et al. 2023). By proving that a specification restricted to a specific grammar is infeasible, program synthesizers can incrementally extend the grammar until a minimal solution (in terms of program size) is found. A similar approach could be possible using infeasibility reasoning as described in this chapter, where a synthesizer checks the feasibility of a specification against a hierarchy of increasingly expressive recursion schemes.

A different approach to synthesis using unrealizability is taken by Farzan et al. (2022). Their tool Synduce extends counterexample-guided inductive synthesis (CEGIS) with a dual inductive procedure for generating unrealizability witnesses.

## 3.8 Conclusion

We have shown how to automatically compute the feasibility of polymorphic functions with input-output examples by interpreting those functions as container morphisms. This approach goes hand in hand with example propagation, a technique to compute the feasibility of sketches with input-output examples. We have presented a general schema for computing the feasibility of polymorphic functions with input-output examples and sketches, in the form of a pipeline that combines type checking, example propagation, translation to containers, and SMT solving. Two concrete instantiations for this pipeline are presented for reasoning about programs defined using *map* and *foldr*. To support recursion schemes such as *foldr*, we have extended our technique to reason not just about input-output examples, but input-output traces. Additionally, we have introduced the notion of shape-completeness, a generalization of trace-completeness (Osera and Zdancewic 2015)

that takes parametricity into account, which is required to show that reasoning about folds is decidable.

### 3.8.1 Future Work

The examples in this chapter focus primarily on functions acting on lists. It would, however, be interesting to explore more complex datatypes, such as binary trees. Our current technique is restricted to polymorphic functions between strictly positive, unary functors, as this allows an easy translation of inputs and outputs to the container setting as described by Abbott et al. (2005). We can relax these restrictions by extending the theory of containers. For example, switching from unary to $n$-ary containers would enable reasoning about multiple type variables. Supporting ad hoc polymorphism and functors that are not strictly positive would require more complicated extensions to containers, such as described by Seidel and Voigtländer (2010) and Bernardy et al. (2010). Alternatively, future work could explore the possibility of expressing the feasibility of polymorphic programs directly in terms of relational parametricity (Reynolds 1983), rather than relying on the abstraction of container functors.

Our current approach to sketching is rather ad hoc, describing only how to reason about examples propagated through *map* and *foldr*. While it is possible to add support for more such combinators, it would be interesting to take a more general approach to sketching, such as live bidirectional evaluation (Lubin et al. 2020) or automated function inversion (Teegen et al. 2021).

Another avenue for future work is to explore the different applications of feasibility reasoning, including automated program analysis in IDEs, automated feedback in programming tutors, and pruning in program synthesis.

# 4

# Type-and-Example-Driven Refinements

Many synthesizers implicitly benefit from using polymorphic types, since parametric polymorphism reduces the search space. Additional synthesis constraints may interfere with *parametricity*. In particular, a polymorphic type may cause otherwise feasible input-output examples to contradict each other. We present Taxi (a type-and-example-based inferencer), a tool for efficiently reasoning about the feasibility of polymorphic programs specified by input-output examples, and Driver, a tactic language for top-down program synthesis that uses feasibility reasoning to prune the search space. Taxi guarantees that every search state corresponds to a correct (albeit possibly partial) implementation. In addition, it allows for shortcutting the synthesis when a subspecification covers all cases. We show that these techniques have the potential to speed up top-down enumerative type-and-example-driven synthesizers.

## 4.1 Introduction

In statically typed functional languages, programmers often rely on the compiler as an assistant, both to *check* their progress and to *guide* them along (Lubin and Chasins 2021). Program sketching (Solar-Lezama 2009), or programming with holes, allows the programmer to be explicit about which parts of the program are unfinished. The compiler can then check that the program is type correct as well as infer the types of the holes, which can be used to guide the programmer further, for example by giving suggestions of valid next steps (Gissurarson 2018).

Figure 10 shows some examples of possible suggestions to refine a hole ① of type `[a] → [a]`. Following the suggestions ensures that the program remains type-correct. However, there are no guarantees that the resulting program is semantically correct. In fact, many of the suggestions are not very sensible at all: due to the polymorphic type, `map` ② can only ever be refined to `map id`; `const` ② to `const Nil`; and `filter` ② to either `filter (const True)` or `filter (const False)`.[18] Most likely, none of these refinements are what the programmer intended. If only the programmer could express their intent to the

[18] We assume here the programmer is only interested in a total implementation.

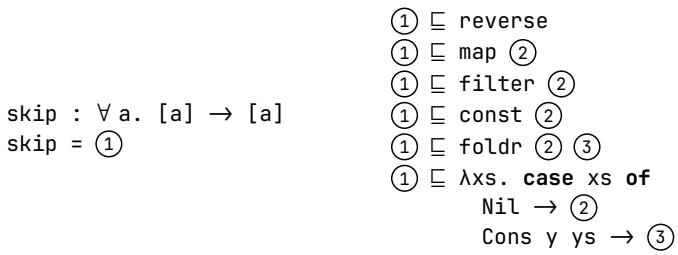



Figure 10. Possible suggestions of hole refinements based on the type of a hole.

compiler, we would be able to give more appropriate suggestions. Input-output examples are arguably the easiest and most straightforward way to do so. For example, the programmer might intend to write a function removing the first element from a list. With just a few input-output examples and some elbow grease, all suggestions but pattern matching can be discarded.

Once only valid hole refinements are allowed, the programmer may repeatedly ask for more refinements, to the point where the program practically writes itself. In this process of stepwise refinement, the program space is narrowed down, while ensuring that the programs within never contradict the type or the input-output examples.

In this chapter, we explore how to automatically check whether a hole refinement is valid with respect to a type and input-output examples, and how this technique can benefit program synthesis. To do so, we first define Taxi, a tool for reasoning about the feasibility of programs specified by a polymorphic type and a set of input-output examples. Next, we define Driver, a program synthesizer aimed at customizability (through a language of tactics) and correctness (by employing feasibility reasoning to ensure only correct refinements are introduced).

**Contributions.** In this chapter, we extend reasoning about the feasibility of polymorphic programs with monomorphic input-output examples as introduced in Chapter 3.

In particular, we

- define a general form for polymorphic examples (Section 4.3);
- show how partial programs can be extracted from these specifications (Section 4.3.6);
- define a notion of coverage checking to check the exhaustiveness of polymorphic examples (Section 4.3.7);
- provide support for arbitrary small container functors (Section 3.3), as well as limited support for ad hoc polymorphism (Section 4.3.8); and
- implement these techniques in a tool called Taxi and show that it outperforms SMT-based feasibility reasoning.

In addition, to show the potential of Taxi in program synthesis, we implement Driver, a customizable, tactics-based synthesizer that employs the aforementioned techniques to prune the search space while ensuring that every state corresponds to a partial implementation (Section 4.4).

## 4.2 Motivating Examples

In this section, we highlight how the techniques described in this chapter can be used to reason about the validity of hole refinements and to prune the search space during program synthesis.

### 4.2.1 Valid Hole Refinements

Suppose the programmer intends to implement the `skip` function, which removes the first element from a list (if it has any). Based on the type alone, the compiler may give the suggested refinements shown in Figure 10. Uncertain about which of these suggestions is the correct one, the programmer may write some assertions:

```
assert skip [1,2,3] == [2,3]
assert skip [4,5]   == [5]
```

We will show how, using just these input-output examples, most of the suggestions can be discarded.

**`reverse`** For refinements that do not introduce holes, such as `reverse`, we can evaluate the expression against the input-output examples. The call `reverse [1,2,3]` evaluates to `[3,2,1]`, rather than the expected result `[2,3]`. Thus `reverse` is not a valid refinement.

**`map`** For refinements introducing holes, the correctness relies on whether the holes can still be filled to get the correct result. In the case of `map`, the first input-output example states that $\exists f$`. map` $f$ `[1,2,3] ≡ [2,3]`. Using equational reasoning, we show that this is incorrect, since `map` preserves the length of the input list.

$$\texttt{map}\ f\ \texttt{[1,2,3]} \equiv \texttt{[}f\ \texttt{1,}\ f\ \texttt{2,}\ f\ \texttt{3]} \not\equiv \texttt{[2,3]}$$

We cannot get the expected output, regardless of how $f$ is instantiated, so `map` is not a valid refinement.

**`filter`** For `filter`, we get $\exists p$`. filter` $p$ `[1,2,3] ≡ [2,3]`. From this we can derive the following constraint on $p$:

$$p\ \texttt{1} \equiv \texttt{False} \wedge p\ \texttt{2} \equiv \texttt{True} \wedge p\ \texttt{3} \equiv \texttt{True}$$

It may seem that this constraint is satisfiable. However, we can use *parametricity* (Reynolds 1983, Wadler 1989) to show that it is not. Simply put, parametricity states that we cannot *inspect* or *produce* values of a polymorphic type `a`, only pass them around. Since the type of $p$ is `a → Bool`, we cannot distinguish between the inputs `1`, `2`, and `3`. This means that the predicate $p$ cannot return different outputs, hence `filter` is *not* a valid refinement.

**`const`** The function `const` is also not a valid refinement, since it throws away the input, which is relevant for computing the output. We can once again use equational reasoning and parametricity. From the equation $\exists c$`. const` $c$ `[1,2,3] ≡ [2,3]`, we derive that $c$ is constrained to the constant `[2,3]`. Parametricity states, however, that we cannot produce `2` or `3`, since $c$ has type `[a]`.

**`foldr`** Reasoning about recursion schemes, such as `foldr`, is a bit more involved. Given the polymorphic type of `skip`, the input-output example `skip [4,5] ≡ [5]` implies `skip [2,3] ≡ [3]`. This leads to the following equation:

$$\exists\, f\ e.\ \texttt{foldr}\ f\ e\ \texttt{[1,2,3]} \equiv \texttt{[2,3]} \wedge \texttt{foldr}\ f\ e\ \texttt{[2,3]} \equiv \texttt{[3]}$$

We can see how these two input-output examples are related by expanding the call to `foldr` $f$ $e$ `[1,2,3]` once:

$$f\ \texttt{1}\ (\texttt{foldr}\ f\ e\ \texttt{[2,3]}) \equiv \texttt{[2,3]}$$

Using `foldr` $f$ $e$ `[2,3]` $\equiv$ `[3]`, we end up with the input-output example $f$ `1 [3]` $\equiv$ `[2,3]`. Since $f$ has type `[a]` $\to$ `[a]`, this example is contradictory. As such, `foldr` is *not* a valid refinement.

**`case`** The last suggested refinement pattern matches on the input. This refinement is always valid, regardless of the input-output examples.

After checking each possible refinement from Figure 10 for validity, we can conclude that only the last suggestion should be given. If the programmer accepts the suggestion to pattern match on the input, suggestions for the remaining holes lead to the following correct implementation:

```
skip xs = case xs of
  Nil → Nil
  Cons y ys → ys
```

With the help of the suggestions, the program practically wrote itself. In the next section, we take this a step further, showing how the synthesis of a program can be automated by performing repeated refinements.

### 4.2.2 Program Synthesis

By repeatedly applying valid refinements, we end up with a program that is correct by construction. For example, consider the function `nub`, which removes all duplicates from a list, keeping only the first occurrences. We can specify `nub` using the following type signature and input-output examples.

```
nub : ∀ a. Eq a ⇒ [a] → [a]
nub xs = ①
```

$$① \vDash \left\{ \begin{array}{ll} \texttt{xs} & \\ \texttt{[]} & \mapsto \texttt{[]} \\ \texttt{[1]} & \mapsto \texttt{[1]} \\ \texttt{[1,1]} & \mapsto \texttt{[1]} \\ \texttt{[2,1]} & \mapsto \texttt{[2,1]} \\ \texttt{[1,1,1]} & \mapsto \texttt{[1]} \\ \texttt{[1,2,1]} & \mapsto \texttt{[1,2]} \end{array} \right\}$$

The `synth` tactic described in Section 4.4 will find the following hole refinements, leading to the desired solution.

```
① ⊑ foldr (λx r. ②) ③ xs
② ⊑ Cons ④ ⑤
③ ⊑ Nil
④ ⊑ x
⑤ ⊑ filter (λy. ⑥) r
⑥ ⊑ x ≠ y
```

① We try `map` and `filter` *before* trying `foldr`, as described in Section 4.4.5. Since `map` and `filter` fail, the first valid refinement for hole ① uses `foldr`. This results in the following constraints on the holes.

$$② \vDash \left\{ \begin{array}{llll} \texttt{x} & \texttt{r} & & \\ \texttt{1} & \texttt{[]} & \longmapsto & \texttt{[1]} \\ \texttt{1} & \texttt{[1]} & \longmapsto & \texttt{[1]} \\ \texttt{2} & \texttt{[1]} & \longmapsto & \texttt{[2,1]} \\ \texttt{1} & \texttt{[2,1]} & \longmapsto & \texttt{[1,2]} \end{array} \right\} \qquad ③ \vDash \texttt{[]}$$

② All examples have an output list with at least one element. So we can introduce the `Cons` constructor. This has precedence over introducing functions.

$$④ \vDash \left\{ \begin{array}{llll} \texttt{x} & \texttt{r} & & \\ \texttt{1} & \texttt{[]} & \longmapsto & \texttt{1} \\ \texttt{1} & \texttt{[1]} & \longmapsto & \texttt{1} \\ \texttt{2} & \texttt{[1]} & \longmapsto & \texttt{2} \\ \texttt{1} & \texttt{[2,1]} & \longmapsto & \texttt{1} \end{array} \right\} \qquad ⑤ \vDash \left\{ \begin{array}{llll} \texttt{x} & \texttt{r} & & \\ \texttt{1} & \texttt{[]} & \longmapsto & \texttt{[]} \\ \texttt{1} & \texttt{[1]} & \longmapsto & \texttt{[]} \\ \texttt{2} & \texttt{[1]} & \longmapsto & \texttt{[1]} \\ \texttt{1} & \texttt{[2,1]} & \longmapsto & \texttt{[2]} \end{array} \right\}$$

③ This hole is constrained to the constant `Nil`, so no need to try other refinements.

④ Before trying any other tactics, we check if a variable in scope matches the examples. In this case, `x` is the right value.

⑤ The refinement `map` fails, but `filter` succeeds. Since `filter` can be defined in terms of `foldr`, we do not try a refinement with `foldr`.

$$⑥ \vDash \left\{ \begin{array}{llll} \texttt{x} & \texttt{y} & & \\ \texttt{1} & \texttt{2} & \longmapsto & \texttt{True} \\ \texttt{1} & \texttt{1} & \longmapsto & \texttt{False} \end{array} \right\}$$

⑥ At this point, the input-output examples along with polymorphic type cover all possible cases. We can recognize this automatically by means of example coverage checking (Section 4.3.7). Further refinements can no longer influence the semantics of the program, so we can use program extraction to fill hole ⑥ (Section 4.3.6).

After this final step, the synthesis is finished and the final program follows from the refinements:

```
nub xs = foldr (λx r. Cons x (filter (λy. x ≠ y) r)) Nil xs
```

Not only is this the first solution found by our synthesizer, but every refinement step is the first valid refinement considered for its hole. This means that no backtracking or unnecessary branching occurred during the synthesis of this program. Note, however, that

this particular example works especially well with the `synth` tactic defined in Section 4.4. In practice, the user may find it necessary to write their own tactic to achieve the best results.

## 4.3 Polymorphic Examples

It may seem that a single input-output example, such as $f\ [1,2,3] \equiv [2,3]$, only restricts $f$ on a single input. However, when the constrained function has a polymorphic type, such as $f : \forall a.\, [a] \to [a]$, parametricity constrains many more inputs based on this single input-output example. In particular, $f$ is restricted on every input list of length three, regardless of the type and values of the list elements.In a way, our single input-output actually represents a set of input-outputs, described by the following equation:

$$\forall x\ y\ z.\ f\ [x,y,z] \equiv [y,z]$$

We refer to such an equation as a *polymorphic example*, and to the original input-output example as a *monomorphic example*. Informally, to compute a polymorphic example, we traverse the elements in the monomorphic example, assigning a unique variable to each. Since polymorphic functions work for any input, we universally quantify over the input elements. The output elements of a polymorphic function should come from the input, so we constrain them to the inputs that have the same value. When the input contains duplicate elements, outputs can be chosen arbitrarily among them in the corresponding polymorphic example. For example, for the monomorphic example $f\ [1,1] \equiv 1$:

$$\forall x\ y.\ \exists z \in \{x,y\}.\ f\ [x,y] \equiv [z]$$

Sometimes, an input-output example and a polymorphic type contradict each other. For example, the monomorphic example $f\ [1,2] \equiv [2,3]$ contradicts the type $\forall a.\, [a] \to [a]$, since the output 3 does not occur in the input. This contradiction is visible in the corresponding polymorphic example:

$$\forall x\ y.\ \exists z \in \{\}.\ f\ [x,y] \equiv [y,z]$$

In Chapter 3, we have shown how to automate such reasoning about the feasibility of polymorphic functions with monomorphic input-output examples. Polymorphic functions are interpreted as morphisms between finitary container functors. Translating input-output examples to the setting of container functors results in more manageable constraints that can be solved by an SMT solver. Note how finitary containers are exactly traversable functors (Jaskelioff and O'Connor 2015), matching our intuition of polymorphic examples being computed by *traversing* the input-output examples.

In this section, we give an overview of our previous technique and extend upon it in several ways in our tool Taxi, allowing for faster reasoning, more transparent feedback, as well as support for arbitrary small containers and limited ad hoc polymorphism.

### 4.3.1 Container Morphisms

Abbott et al. (2003, 2005) show how some datatypes that *contain* other datatypes can be represented in terms of their *structure* and their *contents*. A *container* $S \triangleright P$ has a shape of type $S$, representing its structure and, for every shape $s : S$, a set of positions of type $P_s$,

specifying the locations within that structure where each element is located. For example, the structure of a list is its length (a natural number), and a list of length $n$ has exactly $n$ elements. As such, the list container is defined as $\mathbb{N} \triangleright \mathrm{Fin}$, where $\mathrm{Fin}\ n = \{0, ..., n-1\}$, the type containing exactly $n$ elements. The *extension* of a container $S \triangleright P$ (written $[\![S \triangleright P]\!]$) is a functor defined using a dependent pair as

$$[\![S \triangleright P]\!]\ X = \Sigma_{(s:S)}(P_s \to X).$$

Values of type $[\![S \triangleright P]\!]\ X$ consist of a shape $s : S$ and a function mapping each position $p : P_s$ to the corresponding element of type $X$. For example, the list $[x_0, ..., x_{n-1}]$ is represented in the list container extension as $(n, \lambda i.\, x_i)$.

When reversing a list, its elements are reordered from highest to lowest index. This is easily done in the container representation: the reverse of $(n, \lambda i.\, x_i)$ is $(n, \lambda i.\, x_{n-i-1})$. Note how this reversal can be expressed in terms of only the indices of elements, rather than their values. Such functions, that only refer to the indices of elements, are exactly *container morphisms*.

A container morphism between the containers $S \triangleright P$ and $T \triangleright Q$ is a pair $(u, g)$, where $u : S \to T$ is a function from input shapes to output shapes and $g : \Pi_{(s:S)}\left(Q_{(u\,s)} \to P_s\right)$ is a function mapping, for each input shape, output positions to input positions. In other words, $g$ describes where each element in the output comes from. Container morphisms can be applied using their *extension*:

$$\langle\!\langle u, g \rangle\!\rangle : \forall a.\, [\![S \triangleright P]\!]\ a \to [\![T \triangleright Q]\!]\ a$$
$$\langle\!\langle u, g \rangle\!\rangle = \lambda(s, p).\, (u\,s, p \circ g_s)$$

We can define list reversal as the extension of a container morphism as

$$\langle\!\langle (\lambda n.\, n), (\lambda n\, i.\, n - i - 1) \rangle\!\rangle.$$

### 4.3.2 Reasoning about Feasibility

Container morphisms are exactly natural transformations between container functors: if a function $f$ is a natural transformation between container functors, there exists a container morphism $(u, g)$ such that $f \cong \langle\!\langle u, g \rangle\!\rangle$ (Abbott et al. 2005). In Chapter 3, we used this insight to reason about the feasibility of input-output examples on polymorphic functions, by translating those examples to the container setting, where they can easily be solved by an SMT solver.

More concretely, we asked the following question: given container functors $F$ and $G$, and a set of input-output pairs, does there exist a function $f : \forall a.\, F\,a \to G\,a$ that implements the input-output pairs? To answer this question, we then defined the following procedure:

1. find a container $S \triangleright P$ such that $F \cong [\![S \triangleright P]\!]$;
2. find a container $T \triangleright Q$ such that $G \cong [\![T \triangleright Q]\!]$;
3. assert that there exists a container morphism $(u, g)$ such that $f \cong \langle\!\langle u, g \rangle\!\rangle$;
4. for every input-output example $f\,i \cong o$ instantiating $f$ at type $\tau$
   a. compute $(s, p) : [\![S \triangleright P]\!]\ \tau$ such that $i \cong (s, p)$;
   b. compute $(t, q) : [\![T \triangleright Q]\!]\ \tau$ such that $o \cong (t, q)$;

c. note that $\langle\!\langle u, g \rangle\!\rangle(s, p) \equiv (t, q)$;
d. check the *shape* component $u\ s \equiv t$;
e. check the *position* component $p \circ g \equiv q$.

Finding a contradiction in step 4d or 4e shows that no such function $f$ exists, and solutions for $u$ and $g$ correspond to a correct definition of $f$.

**Challenges of SMT-based Feasibility Reasoning.** In Chapter 3, to try and find solutions for $u$ and $g$, we used an SMT solver. In steps 1, 2, 4a, and 4b, we used handcrafted translations, translating common functors to their canonical container representations.

The effectiveness of this approach on list functions relies heavily on the fact that the shape and position types of the canonical list container ($\mathbb{N}$ and Fin respectively) can be represented as integers with lower and upper bounds, which SMT solvers are optimized for. In contrast, consider the following binary `Tree` datatype:

```
data Tree a = Node (Tree a) a (Tree a) | Leaf
```

We can represent it as a container whose shape is a bare tree (without values in the nodes) and its positions are indices in that tree. To encode these in an SMT solver, we have to represent the bare tree as an algebraic datatype and check that the indices do not fall outside that tree. While this is possible, it leads to a large bump in complexity compared to lists. We recognize the following downsides of SMT-based feasibility reasoning, especially in the context of program synthesis:

**Flexibility** Container translations have to be constructed manually for every datatype.

**Efficiency** Shape and position types are dependent types: to model them in an SMT solver, we have to make sure only correct values can be represented. While this is easy for simple types like $\mathbb{N}$ and Fin, more complex datatypes are difficult to represent efficiently.

**Transparency** When the specification is infeasible, there is no explanation as to which constraint failed. When it is feasible, the solver returns arbitrary implementations of $u$ and $g$, which prove feasibility, but cannot easily be translated back into a program in the target language. Additionally, there is no way to inspect or reason about the polymorphic examples.

**Reliability** There are no clear guarantees as to when or whether the SMT solver terminates.

**How We Address These Challenges.** In this chapter, we use the same general procedure for reasoning about feasibility, but use a generic container translation and forgo the use of a solver.

For the container translation, we use the observation that $F \cong [\![F\ \top \triangleright \text{Size}]\!]$, where Size $s$ has exactly one element for each unit in $s : F\ \top$. In other words, we use a translation where the shape of a functor $F$ is that functor filled with units, and the type of positions has exactly one element for each such unit. By using this generic translation, we do not need specialized container translations for every type, as it works for any strictly positive type.

To solve the constraints in steps 4d and 4e, rather than relying on an SMT solver, we define a terminating procedure for computing a normal form for polymorphic examples. This allows

$$
\begin{array}{llll}
\textbf{Constructor} & C & \in & \textit{Constructors} \\
\textbf{Type Variable} & a & \in & \textit{Variables} \\
\textbf{Position} & n & \in & \mathbb{N} \\
\textbf{Datatype} & D & ::= & \overline{C\ \pi} \\
\textbf{Positive Type} & \pi & ::= & \top \mid \pi_1 \times \pi_2 \mid D \mid a \\
\textbf{Value} & v & ::= & () \mid (v_1, v_2) \mid C\ v \\
\textbf{Shape} & s & ::= & () \mid (s_1, s_2) \mid C\ s \mid \boxed{n} \\
\textbf{Position Map} & p & ::= & \{\overline{n \mapsto v : a}\} \\
\textbf{Origin Map} & \Omega & ::= & \left\{\overline{n \mapsto \{n_1, \ldots, n_i\}}\right\}
\end{array}
$$

Figure 11. Syntax for type-and-example specifications.

$$\boxed{v : \pi \downarrow_n^m s \triangleright p}$$

$$\frac{}{() : \top \downarrow_n^n () \triangleright -}\ \textsc{unit}$$

$$\frac{v_1 : \pi_1 \downarrow_m^l s_1 \triangleright p_1 \quad v_2 : \pi_2 \downarrow_n^m s_2 \triangleright p_2}{(v_1, v_2) : (\pi_1 \times \pi_2) \downarrow_n^l (s_1, s_2) \triangleright (p_1 \cup p_2)}\ \textsc{tuple}$$

$$\frac{v : \pi \downarrow_n^m s \triangleright p \quad C\ \pi \in \text{constructors}(D)}{C\ v : D \downarrow_n^m C\ s \triangleright p}\ \textsc{constructor}$$

$$\frac{}{v : a \downarrow_{n+1}^n \boxed{n} \triangleright \{n \mapsto v : a\}}\ \textsc{free}$$

Figure 12. Inference rules for translating values to their container representation.

us to make better use of the structure that is present in these constraints, for more efficient, transparent, and reliable reasoning.

### 4.3.3 A Generic Container Translation

In Figure 11, we describe the syntax of simple values, types, and input-output examples. A datatype $D$ is defined as a (possibly recursive) sum of products. A strictly positive type $\pi$ consists of products, datatypes, and free variables. A value $v$ consists of tuples and constructors.

We translate values to their container representation, a shape $s$ and a position map $p$, using the inference rules in Figure 12. We write $v : \pi \downarrow_n^m s \triangleright p$ to denote that the value $v$ checked against the type $\pi$ has shape $s$ and a position map $p$ that maps the positions $[m, n)$ to values. We may leave out position bounds ($m$ and/or $n$) when they are not relevant. The translation of a value to its container representation replaces each value at type $a$ with a uniquely labeled box $\boxed{n}$ and stores the value in the position map at position $n$. Intuitively, this translation makes the separation of *structure* and *content* explicit.

To translate a tuple, both parts are translated separately such that their position bounds are consecutive, ensuring that each position is uniquely and consistently labeled. Their shapes are then tupled and their position maps unioned. The FREE rule is the most interesting: any

value $v$ at type $a$ is stored in the position map at position $n$, and the value $v$ is replaced by $\boxed{n}$.[19] Note that this is the only rule that strictly increases the range of positions.

The translation can easily be reverted by substituting the boxes in a shape $s$ with the values found in the position map $p$. We write $\hat{p}(s)$ to substitute the boxes in $s$ for values according to $p$. If $v : \pi \downarrow s \triangleright p$, then $\hat{p}(s) = v$.

### 4.3.4 Computing Polymorphic Examples

In this section, we follow the steps from Section 4.3.2 to turn monomorphic examples into polymorphic examples and check their feasibility. Consider once again the function $f : \forall a.\, F\, a \to G\, a$. Following steps 1 and 2, we start by picking containers for $F$ and $G$. We assume that $F\, a$ and $G\, a$ are strictly positive, i.e. $F\, a = \pi_1$ and $G\, a = \pi_2$. As such, we can use the generic container translation described in Section 4.3.3.

For every input-output example $f\, i \equiv o$, following steps 4a and 4b, we translate the input and output using the inference rules in Figure 12:

$$i : \pi_1 \downarrow s \triangleright p \qquad\qquad o : \pi_2 \downarrow t \triangleright q$$

We can then reformulate the input-output example in terms of $s$, $t$, $p$, and $q$:

$$f\, \hat{p}(s) \equiv \hat{q}(t)$$

As we did in the introduction of Section 4.3, we want to generalize this input-output example to constrain inputs of any type $\tau$, containing any elements. To do so, we have to show how $p$ and $q$ constrain $f$. Together, the position maps $p$ and $q$ define, for every output position in $t$, which input positions in $s$ are valid origins. Concretely, each output position corresponds to a set of input positions, given by the *origin map* $\Omega$:

$$\Omega = \{\, n \mapsto \{\, m \mid (m \mapsto v : a) \in p \,\} \mid (n \mapsto v : a) \in q \,\}$$

To give a general form for polymorphic examples in terms of the triple $\langle s, t, \Omega \rangle$, we generalize the input-output example by quantifying over $p$ and $q$:

$$f \vDash \langle s, t, \Omega \rangle \quad \Leftrightarrow \quad \forall p.\, \exists q.\, f\, \hat{p}(s) \equiv \hat{q}(t) \wedge p \ltimes_\Omega q \tag{5}$$

where $p \ltimes_\Omega q = \forall (n \mapsto v : a) \in q.\, \exists (m \mapsto v : a) \in p.\, \exists (n \mapsto o) \in \Omega.\, m \in o$.

**An Example Translation.** Consider the input-output example $f\, [1, 1] \equiv [1]$ for the function $f : \forall a.\, [a] \to [a]$. After translating, we end up with a polymorphic example $f \vDash \langle s, t, \Omega \rangle$, where

$$s = [\boxed{0}, \boxed{1}], \quad t = [\boxed{0}], \quad \text{and} \quad \Omega = \{\, 0 \mapsto \{0, 1\} \,\}.$$

Equation 5 quantifies over the position maps $p$ and $q$. Since we know the number of positions in $s$ and $t$, we can make $p$ and $q$ explicit. They are of the form $\{\, 0 \mapsto x : a, 1 \mapsto y : a \,\}$ and $\{\, 0 \mapsto z : a \,\}$ respectively, so we can quantify over $x$, $y$, and $z$ instead. In doing so, the relation $p \ltimes_\Omega q$ simplifies to $z \in \{x, y\}$:

[19] Note that there is no ambiguity as to which rule applies; $a$ is a free type variable and distinct from other types.

$$\forall x\ y.\ \exists z \in \{x, y\}.\ f\ [x, y] \equiv [z]$$

Typically, when the shapes of polymorphic examples are known, we will write them in this form, by quantifying over the elements rather than the position maps.

### 4.3.5 Feasibility Conflicts

We have seen how to translate input-output examples into polymorphic examples. For a set of examples, we get

$$\overline{f\ i_k \equiv o_k} \quad \Rightarrow \quad \overline{f \vDash \langle s_k, t_k, \Omega_k \rangle}$$

We are interested in knowing whether a program $f$ satisfying the input-output examples exists, i.e. whether the polymorphic examples are feasible. As shown in Section 3.3, if polymorphic examples are infeasible, this is due to one of three conflicts. We can easily check for these conflicts by inspecting the polymorphic examples.

**Magic Output** An example has an output element that does not occur in the input. When this is the case, $\Omega$ is empty at the corresponding output position.

**Shape Conflict** Two examples have the same input shape, but not the same output shape.

**Position Conflict** Two or more examples have the same input and output shape, but they disagree on the origins of the output elements. To check for position conflicts, we merge examples with the same input shape by taking the pointwise intersection of their origin maps:

$$\left.\begin{array}{c} f \vDash \langle s, t, \Omega_1 \rangle \\ \ldots \\ f \vDash \langle s, t, \Omega_n \rangle \end{array}\right\} \quad f \vDash \langle s, t, \bigcap_i \Omega_i \rangle$$

### 4.3.6 Program Extraction

After checking and merging the constraints, we end up with a set of polymorphic examples with unique input shapes and origin maps that are non-empty at every position. We can show that a partial program implementing these constraints exists. For every position $n$ in every output shape $t$, we pick an input position from the origin map, denoted $\omega(t, n)$. We can then define a program $f$ by pattern matching on the input shapes. For every triple $\langle s, t, \Omega \rangle$, we create a pattern by filling the positions in the input shape with fresh variables. All other cases are caught by a wildcard pattern, whose right-hand side is simply a hole.

$$\begin{array}{l} f = \lambda\texttt{case} \\ \quad \hat{p}(s) \to \hat{q}(t) \quad \texttt{where}\ p = \{\overline{m \mapsto x_m}\};\ q = \{\overline{n \mapsto x_{\omega(t,n)}}\} \\ \quad \_ \to \bigcirc \end{array}$$

This implementation is guaranteed to satisfy all the input-output examples, but may be partial if not all patterns are covered.

### 4.3.7 Example Coverage Checking

We can perform a basic form of pattern match coverage checking to see which cases are missing from a set of polymorphic examples. Note that we only have to check which input *shapes* are missing, by enumerating all possible shapes of the input container. We are

particularly interested in cases where all shapes are covered, as this means a program found by program extraction is guaranteed to be total. For example:

```
swap : ∀ a b. Either a b → Either b a
swap (Left 3) ≡≡ Right 3
swap (Right ()) ≡≡ Left ()
```

$\xrightarrow{\text{Extract}}$

```
λcase
  Left  x → Right x
  Right x → Left  x
```

### 4.3.8 Ad Hoc Polymorphism

To reason about ad hoc polymorphic functions, we adopt the approach by Seidel and Voigtländer (2010) for extending containers with ad hoc polymorphism. When the ad hoc polymorphism describes a relation (such as an equality or an ordering), it can be used to indirectly inspect elements by checking if the relation holds between them. Consider, for example, the following specification for the `max` function:

```
max : ∀ a. Ord a ⇒ a → a → a
max 1 2 ≡≡ 2
max 1 1 ≡≡ 1
max 3 2 ≡≡ 3
```

We would like to compute conditional polymorphic examples, as follows:

$$\forall x\, y.\, \exists z \in \{x, y\}.\quad \begin{cases} \texttt{max}\ x\ y \equiv y & \text{if } x < y \\ \texttt{max}\ x\ y \equiv z & \text{if } x \equiv y \\ \texttt{max}\ x\ y \equiv x & \text{if } x > y \end{cases}$$

We can, however, avoid extending our formalization by reformulating the problem. One way to define an ordering is in terms of the `Ordering` datatype and the accompanying `compare` function.

```
data Ordering = LT | EQ | GT
compare : ∀ a. Ord a ⇒ a → a → Ordering
```

We can capture the ordering of the inputs using the `compare` function and pass it to the fully polymorphic helper function `max'`. The feasibility of `max` and `max'` are equivalent, as they can be defined in terms of each other.

```
max x y = max' (compare x y) x y
  where max' : ∀ a. Ordering → a → a → a
```

We can now compute the following polymorphic examples, showing that `max'` (and thus `max`) is feasible.

$$\forall x\, y.\, \exists z \in \{x, y\}.\quad \begin{cases} \texttt{max'}\ \texttt{LT}\ x\ y \equiv y \\ \texttt{max'}\ \texttt{EQ}\ x\ y \equiv z \\ \texttt{max'}\ \texttt{GT}\ x\ y \equiv x \end{cases}$$

Better yet, we can use example coverage checking and program extraction to find a total solution for `max`:

```
max x y = case ordering x y of LT → y; EQ → x; GT → x
```

Effectively, the `Ord` constraint is replaced by the `Ordering` argument. Every function with an `Ord` constraint can be automatically rewritten in a similar manner, in terms of a helper function that replaces the constraint with an argument that carries the ordering information. We automate this process for dealing with `Ord` constraints, and do the same for `Eq` constraints.

## 4.4 Sketching

We have seen how to use program extraction (Section 4.3.6) to generate a partial program implementing a feasible specification. Such programs do not, however, generalize beyond the given input-output examples. Instead, we want to introduce common abstractions that generalize beyond the input-output examples in a natural way. To do so, we will use *program sketching*, a technique whereby a program is written incrementally, leaving holes for the parts that are yet to be defined. At each increment, we make sure that the program is still feasible and compute which specification should hold on each hole to guarantee a correct solution.

### 4.4.1 An Example: delete in terms of filter

Consider the following specification for the `delete` function, along with a simple sketch.

```
delete : ∀ a. Eq a ⇒ a → [a] → [a]
delete 0 [] ≡ []
delete 1 [1] ≡ []
delete 2 [2,2] ≡ []
delete 3 [1,3] ≡ [1]

delete a xs = ⓪
```

After confirming that the specification of `delete` is feasible, we may attempt to implement it using `filter`, by introducing the following hole filling.

```
⓪ ⟼ filter (predicate a) xs    where predicate x y = ①
```

Using type checking and equational reasoning, we can infer that `predicate` should satisfy the following specification:

```
predicate : ∀ a. Eq a ⇒ a → a → Bool
predicate 1 1 ≡ False
predicate 2 2 ≡ False
predicate 3 1 ≡ True
```

We can compute the polymorphic examples as described in Section 4.3 to show that `predicate` is feasible.

$$\forall x\, y.\ \begin{cases} \texttt{predicate}\ x\ y \equiv \texttt{False} & \text{if } x \equiv y \\ \texttt{predicate}\ x\ y \equiv \texttt{True} & \text{if } x \not\equiv y \end{cases}$$

The resulting constraint is not only feasible, but also exhaustive and unique. This means that there is a single, total solution for `predicate`, which we can find using program extraction.

```
data Sketch hole = ... | Hole hole
data Signature = Signature { args : Args, out : Type }
type Args = [(String, Type)]
data Example = Example { args : [Value], out : Value }
data Spec = Spec { sig : Signature, exs : [Example] }
```

Figure 13. The datatypes used in sketching.

After inlining `predicate` and normalizing, we end up with the following implementation of `delete`:

```
delete x xs = filter (λy. x ≠ y) xs
```

### 4.4.2 A Sketching Framework

We define Driver, a domain-specific language in Haskell for sketching. The main datatypes are defined in Figure 13. A specification consists of a type signature and a list of input-output examples. A sketch is an expression from the lambda-calculus extended with data (de)constructors and holes. A sketch is parameterized over the type of values stored in the holes. For example, we may store numbers in holes to refer to them. More interestingly, we can store specifications in holes to describe the subgoals.

**Tactics.** Starting from a specification, we may want to introduce a sketch and compute the subgoals. Imagine a function `propagate` that can compute subgoals by propagating a specification through a sketch.

```
propagate : Sketch a → Spec → Maybe (Sketch Spec)
```

Unfortunately, we cannot define this function in general. The process of inferring the input-output examples that should hold on the holes of an expression, known as *example propagation* (Osera and Zdancewic 2015), is not well-defined for all sketches. For example, when implementing `delete`, if we try to introduce the hole filling ⓪ ⟼ `filter` ① ②, there is an infinite number of possible input-output examples we could infer for ②. As a solution, we only allow sketching through tactics, i.e. functions that attempt to introduce a sketch for which example propagation is well-defined. For example, we can describe a tactic that applies `filter` to a variable in scope, introducing the following hole filling:

```
filter (predicate a) xs    where predicate x y = ◯
```

In addition to failing when the sketch does not fit the specification, the tactic should be able to (1) arbitrarily choose a variable `xs` to apply `filter` to (possibly leading to multiple correct sketches), and (2) generate fresh variable names for `predicate`, `x`, and `y`. To this end, we define tactics in terms of a `Synth` monad, which allows failure, branching, and generating fresh variables.

```
type Tactic = Spec → Synth (Sketch Spec)
(||) :: Tactic → Tactic → Tactic
fail :: Tactic
```

We define a tactic `introFilter` that applies `filter` to a specific variable in scope, passes all the other variables in scope along to the predicate, computes the specification that

should hold on the predicate, ensures that this specification is feasible, and applies program extraction if all cases are covered.

```
introFilter :: Var → Tactic
```

To apply this function to any variable in scope, we define the tactic combinator `anywhere`.

```
anywhere :: (Var → Tactic) → Tactic
anywhere t spec = foldr (||) fail ts
  where ts = map t (variables spec)
```

To find an implementation for the function `delete`, we simply apply the tactic `anywhere introFilter` to the specification and enumerate the values in the resulting `Synth` monad. If we find a sketch without holes, it is guaranteed to be a total program that implements the specification.

We define many such tactics. For example, `exact` introduces a sketch without holes; `introCase` introduces a case distinction on a variable in scope (provided it is an algebraic datatype); and `introConstructor` introduces a constructor if all input-output examples agree. Additionally, we define tactics introducing higher-order combinators such as `filter`, `map`, and `foldr`. While `introMap` and `introFilter` only work on lists, `introFold` works on any inductively defined datatype.

```
exact            :: Sketch Void → Tactic
introCase        :: Var → Tactic
introConstructor :: Tactic
introMap         :: Var → Tactic
introFilter      :: Var → Tactic
introFold        :: Var → Tactic
```

**Recursion Schemes.** The tactic `introFold` is particularly important for defining recursive programs. Unfortunately, example propagation on folds results in subgoals in terms of *input-output traces*, rather than input-output examples. Consider the following example for `reverse`:

$$\texttt{reverse [1,2,3]} \equiv \texttt{[3,2,1]}$$

If we try to implement `reverse` as `foldr f []`, this leaves us with the following input-output trace:

$$\texttt{f 1 (f 2 (f 3 []))} \equiv \texttt{[3,2,1]}$$

It is possible to reason about the feasibility of such input-output traces by encoding the constraints in an SMT solver, as we did in Chapter 3. Taxi can, however, only reason about constraints that are just input-output pairs. We can remedy this by requiring that the input-output examples are *trace-complete* (Osera and Zdancewic 2015). A set of input-output examples is trace-complete if any recursive call is also represented in the input-output examples. In the case of `reverse`, we additionally require the following input-output examples:

```
reverse [2,3] ⇒ [3,2]
reverse [3] ⇒ [3]
reverse [] ⇒ []
```

With these examples, the constraint on `f` becomes tractable:

```
f 1 [3,2] ⇒ [3,2,1]
f 2 [3] ⇒ [3,2]
f 3 [] ⇒ [3]
```

The tactic `introFold` only succeeds on specifications that are trace-complete.

### 4.4.3 Composing Tactics

Typically, we want to apply tactics one after the other. Consider the function `intersect : ∀a. [a] → [a] → [a]`, which removes all elements from the first list `xs` that do not occur in the second list `ys`. Applying the tactic `introFilter` to a specification for `intersect` results in a sketch with a single hole containing a specification for (`∈ ys`). Then, we traverse the sketch, applying `introFold` to the hole. This leaves us with a sketch of sketches (of type `Sketch (Sketch Spec)`), which we flatten by *accepting* the hole filling.[20]

```
accept :: Sketch (Sketch hole) → Sketch hole
```

We can define tactic composition in terms of `accept`.

```
compose :: Tactic → Tactic → Tactic
compose t u spec = do
  sketch ← t spec
  nested ← traverse u sketch
  return (accept nested)
```

We can implement `intersect` using the tactic

```
compose (anywhere introFilter) (anywhere introFold)
```

After introducing `filter` and `foldr`, coverage checking kicks in, leading to the following solution:

```
intersect xs ys = filter (λx. foldr (f x) False ys) xs
  where f x y False = x == y
        f x y True  = True
```

### 4.4.4 Program Synthesis

Note that the tactics we defined so far describe only the high-level structure of the functions they implement. The details, such as which variables to apply functions to and how to solve the subgoals, are handled by the `anywhere` combinator and by example coverage checking. This means that these tactics can also be used to implement various other functions that use the same structure. The right tactic can synthesize programs for a variety of different specifications. Often, however, we cannot rely so much on example coverage checking, and require many more tactics to be used in a specific order to find the definition we are looking

[20] Note that `Sketch` is a monad, with `join = accept` and `return = Hole`.

for. We will define a tactic that repeatedly tries many different tactics. First, we define the tactic combinator `repeat`.

```
repeat :: Tactic → Tactic
repeat tactic = compose tactic (repeat tactic)
```

Next, we define a `step` tactic that tries a variety of different tactics in parallel.

```
step :: Tactic
step = anywhere assume || anywhere eliminate
  || introTuple || introConstructor
  || anywhere2 compare || anywhere2 equate
eliminate x = introMap x || introFilter x
  || introFold x || introCase x
```

The tactic `assume` tries to fill a hole with a variable in scope. The tactics `compare` and `equate` introduce a pattern match on the ordering and equality of pairs of inputs, respectively. To apply them, we use a variant of `anywhere`, called `anywhere2`, that applies its argument to each unique pair of variables in scope. We define a synthesis tactic as `synth = repeat step`. This tactic is general enough to synthesize `delete`, `intersect`, `reverse`, as well as many other functions.

**Enumeration Strategy.** Synthesis proceeds by building up a tactic, applying it to a specification, and then enumerating the results. We use Dijkstra's algorithm (Dijkstra 1959) to enumerate the results, allowing the programmer to add weights to tactics. For `synth`, we want to prioritize simple tactics over recursion schemes, and especially discourage pattern matching, because it leads to overfitting. As such, we give simple tactics a weight of 1, recursion schemes a weight of 4, and pattern matching a weight of 7.

### 4.4.5 Biased Choice

The `step` tactic performs a lot of branching to cover a large space of programs. Some of this branching, however, is redundant. For example, if the tactic `assume x` succeeds, there is no need to attempt any other tactic, since a correct solution is found. We introduce a new tactic combinator `◁` for biased choice, where the right-hand tactic is only tried when the left-hand tactic fails. Together with a biased version of the `anywhere` tactic, we redefine the `step` tactic so that no other tactic is tried if introducing any variable is successful.

```
step = anywhereBiased assume ◁ ...
```

It may be tempting to replace any occurrence of `(||)` with `(◁)`, but this will lead to some programs never being reached. For example, if we write `introCase x ◁ introMap x`, the tactic `introMap x` will never be reached, because pattern matching on lists always succeeds.

The biased choice operator only makes sense if we can be sure that if the left-hand tactic succeeds, the resulting subgoals are feasible. This means that proper feasibility reasoning is a prerequisite for using a biased choice operator.[21] We should, however, only use `t ◁ u` instead of `t || u` if we always prefer a solution in terms of `t` over a solution in terms of `u` (when both are applicable). This implies that we can always use `t ◁ u` or `u ◁ t`

[21] Except if the left-hand tactic does not introduce holes.

interchangeably when `t` and `u` are disjoint (i.e. they never both succeed on the same specification). We use this principle to adjust the definition of `eliminate`:

- If both `introMap x` and `introFilter x` apply, the specification describes the identity function on lists, which would have already been caught by `assume x`. Hence, we can consider them disjoint.
- If `introFilter x` applies, `introFold x` will also apply, since `filter` can be defined in terms of `foldr`. We prefer a definition in terms of `filter`, to avoid reimplementing `filter` in terms of `foldr`.
- Since `introCase x` will always apply (provided that `x` is an algebraic datatype), we have to decide whether we prefer defining a function as a fold whenever possible. While there are cases where pattern matching leads to a simpler solution, we accept the risk of finding some convoluted solutions.[22]

This leads to the following redefinition of `eliminate`:

```
eliminate x = introMap x ◁ introFilter x ◁ introFold x ◁ introCase x
```

## 4.5 Evaluation

We have implemented both Taxi and Driver in Haskell. The full implementation is available on Zenodo (Mulleners 2026c) for reproduction and on GitHub[23] for reuse. In this section, we evaluate our implementation on two benchmarks: one for automatically testing whether a function is a fold, and one for program synthesis. Both benchmarks are run on an HP Elitebook 850 G6 with an Intel® Core™ i7-8565U CPU (1.80GHz) and 32 GB of RAM.

### 4.5.1 Benchmark: Fold Detection

In Chapter 3, we evaluated SMT-based feasibility reasoning by automatically checking whether a series of functions from the Haskell prelude can be implemented as a fold. We can similarly check whether a function is a fold by applying the tactic `introFold`. To evaluate Taxi, we test it on the same benchmark of functions. Whereas in Chapter 3 we distinguished between trace-complete and trace-incomplete example sets, Taxi only works for trace-complete example sets. For simplicity and for a more equal comparison, we ran both tools on trace-complete example sets. We made the following minor changes to the benchmark:

- The examples for `append` and `prepend` were not trace-complete, so we added the missing case.
- We replaced integer arguments with natural numbers.
- For `zip` and `unzip`, we use the more generic type as seen in the Haskell prelude, using two separate type variables rather than a single one.

As seen in Table 4, we improve significantly on this benchmark, achieving speedups of more than a hundredfold. This paves the way for applications where feasibility reasoning

[22] In general, it is up to the programmer to decide whether this is a risk worth taking when defining a synthesis tactic. Luckily, it is always possible to try a different tactic when the result is not satisfactory.

[23] https://github.com/NiekM/taxi-driver

Table 4. Benchmark testing fold detection on a series of common functions. Times in milliseconds. Where possible, we compare Taxi to SMT-based feasibility reasoning as defined in Chapter 3. We use ✓ and ✘ respectively to denote whether a function is or is not a fold. Note that `union` is a fold over its second, but not its first argument.

| name | : Type | Fold? | Taxi | SMT | Gain |
|---|---|---|---|---|---|
| append | : ∀a. [a] → [a] → [a] | ✓ | 0.455 | 159 | 349× |
| concat | : ∀a. [[a]] → [a] | ✓ | 0.568 | 126 | 222× |
| drop | : ∀a. Nat → [a] → [a] | ✘ | 0.263 | 45 | 171× |
| head | : ∀a. [a] → Maybe a | ✓ | 0.240 | 33 | 138× |
| index | : ∀a. Nat → [a] → Maybe a | ✘ | 0.235 | 39 | 166× |
| init | : ∀a. [a] → [a] | ✘ | 0.260 | 31 | 119× |
| last | : ∀a. [a] → Maybe a | ✓ | 0.314 | 31 | 99× |
| length | : ∀a. [a] → Nat | ✓ | 0.182 | 23 | 126× |
| null | : ∀a. [a] → Bool | ✓ | 0.203 | 20 | 99× |
| prepend | : ∀a. [a] → [a] → [a] | ✓ | 0.460 | 149 | 324× |
| reverse | : ∀a. [a] → [a] | ✓ | 0.272 | 57 | 210× |
| splitAt | : ∀a. Nat → [a] → ([a], [a]) | ✓ | 0.396 | 384 | 970× |
| tail | : ∀a. [a] → [a] | ✘ | 0.160 | 31 | 194× |
| take | : ∀a. Nat → [a] → [a] | ✓ | 0.423 | 83 | 196× |
| unzip | : ∀a b. [(a, b)] → ([a], [b]) | ✓ | 0.350 | 126 | 360× |
| zip | : ∀a b. [a] → [b] → [(a, b)] | ✓ | 0.324 | 128 | 395× |
| bfe | : ∀a. Tree a → [a] | ✘ | 0.887 | - | - |
| depth | : ∀a. Tree a → Nat | ✓ | 0.536 | - | - |
| inorder | : ∀a. Tree a → [a] | ✓ | 1.21 | - | - |
| levels | : ∀a. Tree a → [[a]] | ✓ | 1.15 | - | - |
| mirror | : ∀a. Tree a → Tree a | ✓ | 0.554 | - | - |
| size | : ∀a. Tree a → Nat | ✓ | 0.624 | - | - |
| compress | : ∀a. Eq a ⇒ [a] → [a] | ✓ | 0.745 | - | - |
| group | : ∀a. Eq a ⇒ [a] → [[a]] | ✓ | 0.673 | - | - |
| elem | : ∀a. Eq a ⇒ a → [a] → Bool | ✓ | 0.723 | - | - |
| elemIndex | : ∀a. Eq a ⇒ a → [a] → Maybe Nat | ✓ | 0.482 | - | - |
| nub | : ∀a. Eq a ⇒ [a] → [a] | ✓ | 0.313 | - | - |
| union | : ∀a. Eq a ⇒ [a] → [a] → [a] | ✘/✓ | 1.07 | - | - |
| insert | : ∀a. Ord a ⇒ a → [a] → [a] | ✓ | 0.569 | - | - |
| maximum | : ∀a. Ord a ⇒ [a] → Maybe a | ✓ | 0.608 | - | - |
| ordNub | : ∀a. Ord a ⇒ [a] → [a] | ✓ | 0.475 | - | - |
| pivot | : ∀a. Ord a ⇒ a → [a] → ([a], [a]) | ✓ | 0.732 | - | - |
| sort | : ∀a. Ord a ⇒ [a] → [a] | ✓ | 0.566 | - | - |
| sorted | : ∀a. Ord a ⇒ [a] → Bool | ✘ | 0.542 | - | - |

Table 5. Benchmark testing synthesis using Driver on a series of common functions. Times in milliseconds. We test various levels of feasibility reasoning: no feasibility reasoning (N); feasibility reasoning (F); with coverage checking (C); with branching control (B); and with both coverage checking and branching control (C+B). Timeouts (taking longer than 1 second) are denoted by ⊥. Incorrect synthesis results are shown in gray. Noticeable improvements (⩽ 50%) are shown in **bold**.

| name | : Type | Synthesis Options | | | | |
|---|---|---|---|---|---|---|
| | | N | F | C | B | C+B |
| append | : ∀a. [a] → [a] → [a] | 1.28 | 1.59 | 1.57 | 1.36 | 1.35 |
| concat | : ∀a. [[a]] → [a] | 3.66 | 3.90 | 2.92 | 2.39 | **1.74** |
| drop | : ∀a. Nat → [a] → [a] | 4.15 | 2.76 | 2.77 | 2.21 | 2.20 |
| head | : ∀a. [a] → Maybe a | 0.305 | 0.362 | 0.288 | 0.321 | 0.295 |
| index | : ∀a. Nat → [a] → Maybe a | 14.0 | 11.6 | 11.7 | **5.45** | 5.42 |
| init | : ∀a. [a] → [a] | 23.4 | **9.40** | 8.30 | **4.51** | 4.21 |
| last | : ∀a. [a] → Maybe a | 1.60 | 1.30 | **0.363** | 0.706 | 0.371 |
| length | : ∀a. [a] → Nat | 0.303 | 0.350 | 0.306 | 0.312 | 0.313 |
| null | : ∀a. [a] → Bool | 0.237 | 0.271 | 0.242 | 0.281 | 0.248 |
| prepend | : ∀a. [a] → [a] → [a] | 1.20 | 1.57 | 1.58 | 1.37 | 1.38 |
| reverse | : ∀a. [a] → [a] | 3.26 | 3.27 | **1.63** | **1.58** | 1.36 |
| splitAt | : ∀a. Nat → [a] → ([a], [a]) | 223 | 221 | 226 | 190 | 192 |
| tail | : ∀a. [a] → [a] | 0.494 | 0.303 | 0.311 | 0.308 | 0.304 |
| take | : ∀a. Nat → [a] → [a] | 122 | 114 | 113 | **29.9** | 30.2 |
| unzip | : ∀a b. [(a, b)] → ([a], [b]) | 3.69 | 3.89 | 3.42 | 1.90 | **1.45** |
| zip | : ∀a b. [a] → [b] → [(a, b)] | 21.6 | 23.0 | 23.2 | **8.54** | 8.79 |
| bfe | : ∀a. Tree a → [a] | ⊥ | ⊥ | ⊥ | ⊥ | ⊥ |
| depth | : ∀a. Tree a → Nat | 4.77 | 4.74 | 4.36 | 2.67 | 2.64 |
| inorder | : ∀a. Tree a → [a] | 8.74 | 8.79 | 8.66 | 5.43 | 5.42 |
| levels | : ∀a. Tree a → [[a]] | ⊥ | ⊥ | ⊥ | ⊥ | ⊥ |
| mirror | : ∀a. Tree a → Tree a | 0.713 | 0.966 | 0.905 | 0.866 | 0.807 |
| size | : ∀a. Tree a → Nat | 3.76 | 3.76 | 3.04 | 2.38 | 2.34 |
| compress | : ∀a. Eq a ⇒ [a] → [a] | 105 | **28.9** | 16.9 | 23.0 | **12.8** |
| group | : ∀a. Eq a ⇒ [a] → [[a]] | ⊥ | ⊥ | ⊥ | ⊥ | ⊥ |
| elem | : ∀a. Eq a ⇒ a → [a] → Bool | 0.836 | 0.979 | 0.852 | 0.961 | 0.845 |
| elemIndex | : ∀a. Eq a ⇒ a → [a] → Maybe Nat | 631 | 603 | **212** | **28.4** | 18.2 |
| nub | : ∀a. Eq a ⇒ [a] → [a] | 6.47 | 6.46 | **1.52** | 3.81 | 1.27 |
| union | : ∀a. Eq a ⇒ [a] → [a] → [a] | ⊥ | ⊥ | ⊥ | ⊥ | ⊥ |
| insert | : ∀a. Ord a ⇒ a → [a] → [a] | 108 | 102 | **37.8** | **45.5** | 21.8 |
| maximum | : ∀a. Ord a ⇒ [a] → Maybe a | 5.73 | 5.29 | **0.710** | **1.84** | **0.697** |
| ordNub | : ∀a. Ord a ⇒ [a] → [a] | 72.5 | 68.9 | **20.7** | **24.1** | **8.63** |
| pivot | : ∀a. Ord a ⇒ a → [a] → ([a], [a]) | 95.4 | 94.6 | **5.62** | **46.5** | 4.02 |
| sort | : ∀a. Ord a ⇒ [a] → [a] | 216 | 201 | **55.1** | **66.7** | **20.4** |
| sorted | : ∀a. Ord a ⇒ [a] → Bool | 20.7 | 20.1 | 20.5 | 19.9 | 20.2 |

is called repeatedly, for example to prune the search space during program synthesis. In addition, we extend the benchmark with various functions on trees, to show how Taxi can reason about folds over inductive datatypes other than lists, as well as ad hoc polymorphic functions using `Eq` and `Ord` constraints.

### 4.5.2 Benchmark: Program Synthesis

We evaluate Driver on the same benchmark of programs as Taxi, by applying the `synth` tactic defined in Section 4.4.4. To see the effect of the techniques described in this chapter, we test synthesis using various different options: we perform synthesis without feasibility reasoning (N); with feasibility reasoning (F); with example coverage checking (C), using program extraction to shortcut the search when a specification is exhaustive; with branching control (B), using the `eliminate` tactic from Section 4.4.5; and with both coverage checking and branching control (C+B).

This benchmark suite, shown in Table 5, shows that we can use `synth` to synthesize a large variety of functions at reasonable speed. While feasibility reasoning only seldom leads to significant speedups, it allows for coverage checking and branching control, leading to additional speedups with minimal overhead.

Coverage checking works particularly well for programs with `Eq` and `Ord` constraints. This can be explained by the additional `equate` and `compare` tactics, introduced by `Eq` and `Ord` constraints respectively, that can be circumvented by coverage checking.

Branching control mostly affects programs working on lists, since `introMap` and `introFilter` are specialized to lists. In a way, branching control using a biased choice operator allows us to add helper functions like `map` and `filter` to the search space without increasing the branching factor.

Several functions still fail to synthesize, either due to a timeout (denoted by $\perp$) or because another function that fits the examples is found. Sometimes this happens because the intended function cannot be implemented in terms of the combinators used by the `synth` tactic, i.e. they require a stronger recursion scheme.

The functions `insert`, `sorted`, `sort`, and `ordNub`, are most naturally defined using paramorphisms (Meertens 1992), rather than a fold. We can define a tactic `introPara`, which generalizes `introFold`. Both `insert` and `sorted` are synthesized correctly by composing `introPara` and `synth`. The functions `sort` and `ordNub` are synthesized by composing `introFold`, `introPara`, and `synth`.

Interestingly, the function `sort` *does* synthesize correctly when branching control (B) is turned off. On closer inspection, we see that this is an implementation of insertion sort, with the following curious, inefficient solution for `insert`:

```
insert x = foldr (λy r. min x y : map (max y) r) [x]
```

Proving the correctness of this implementation is left as an exercise to the reader. It may seem that branching control has made synthesis worse, but this is only accidental: the `synth` tactic finds three solutions at the same depth, of which only one is correct. In the benchmark

we only consider the first of these solutions, and the order of solutions is arbitrary, but influenced by the use of branching control. This example highlights two ways in which the programmer can interact with the synthesizer: (1) the programmer can enumerate the first $n$ results returned by the synthesizer, to see if any of them have the intended behavior; and (2) the programmer can test the returned solution, find a counterexample,[24] and update the specification accordingly. Both of these approaches would help the programmer find a correct (albeit very inefficient) solution for `sort`. Unsurprisingly, we can use the same approach to synthesize `insert` directly.

The functions `drop`, `index`, `splitAt`, `take`, and `zip` are all most naturally defined by recursing over both inputs simultaneously. This can be done as a fold over one of the inputs, by passing the other input as an additional argument to the fold. As explained in Section 3.5.1, however, the `introFold` tactic does not allow passing additional arguments. As such, these functions all fail to synthesize.

The expected definition for `depth` requires computing the maximum of two natural numbers, requiring structural recursion over both numbers at the same time.[25] Similarly, the function `levels` can be implemented as a fold in terms of `longZip`, which traverses two lists at the same time.

Breadth-first enumeration (`bfe`) can be implemented in many ways, but typically using an intermediate result. For example, `bfe` can be implemented by first computing `levels`, and then concatenating the result. The `synth` tactic provides no way to compute or reason about intermediate results, so naturally it fails to implement `bfe`.

## 4.6 Related Work

### 4.6.1 Programming with Holes

When writing a program, there are many points at which the state of the program is not syntactically correct, in particular when parts of the program are still unfinished or missing. This makes it difficult to test the program. A common practice is to have unfinished parts of a program raise an exception (such as `NotImplemented`). This allows the program to still compile and type check, as well as be executed (exceptionally). A more principled approach, however, is to use *holes*. Holes explicitly tell the compiler which parts of the program are yet to be finished, allowing it to give helpful feedback, for example inferring the types of the holes.

There have been many efforts to make programming with holes (also known as *sketching* (Solar-Lezama 2009)) an integral part of the development process. Gissurarson (2018) uses the type of a hole to give suggestions on how to fill it. Omar et al. (2017), Yuan et al. (2023), Zhao et al. (2024) use holes in the structure editor Hazel, in order to make as many editor states as possible meaningful. Omar et al. (2019) define *live evaluation*, which proceeds around the holes rather than aborting when a hole is encountered during evaluation. Lubin

[24] E.g. using property-based testing to find a minimal counterexample.

[25] The `compare` tactic currently only allows comparing polymorphic variables with an `Ord` constraint, not natural numbers.

et al. (2020) compare the result of live evaluation against an expected output to turn top-level assertions into assertions on the holes of a program. Taxi brings types, assertions (in the form of input-output examples), and sketching together to allow reasoning about incomplete programs and possible hole suggestions.

### 4.6.2 Type-and-Example-Directed Program Synthesis

Program synthesis concerns the automatic generation of programs based on a specification. Many synthesizers focus on types and input-output examples as specifications. A common idea for top-down synthesizers is that the space of type-correct programs can be explored by "inverting the typing rules" (Osera 2019). Some synthesizers focus on efficiently exploring this search space using compact representations of type-correct programs (Guo et al. 2019, Koppel et al. 2022, Botelho Guerra et al. 2023). Other synthesizers aim to explore fewer programs, narrowing down the search space by only allowing those refinements that preserve the input-output constraints (Osera and Zdancewic 2015, Feser et al. 2015, Frankle et al. 2016, Lubin et al. 2020). Scrybe (Chapter 2) also falls in this category. Driver takes a similar approach, but focuses on the interactions between input-output constraints and polymorphic types, and explores how synthesizers can benefit from these interactions to prune and shortcut the search.

### 4.6.3 Feasibility of Specifications

When programming from specifications, an important question to ask is whether a program satisfying the specification exists. Morgan (1990) describes specifications for which no implementation exists as *infeasible*. Feasibility reasoning is related to proof search and program synthesis, but focuses more on figuring out that a specification is *not* feasible. How well we can reason about the feasibility of specifications depends on the nature of the specifications, as well as the underlying programming language. Urzyczyn (1997) reasons about the feasibility of types, known as *type inhabitation*. The term *realizability* is often used instead of feasibility, typically to refer to the feasibility of programs in a restricted grammar (Hu et al. 2019, Hu et al. 2020, Kim et al. 2023). In Chapter 3, we reasoned about the feasibility of polymorphic types and input-output examples. Taxi extends upon this work, allowing for faster and more transparent reasoning about a wider range of types.

### 4.6.4 Tactic Programming

Tactic programming is a form of metaprogramming, where tactics are functions that refine a goal (i.e. a specification) (Gordon et al. 1979). Tactics transform a single goal into subgoals (a process referred to as *backward reasoning*), such that a solution to these subgoals leads to a solution of the original goal. Tactic programming is most commonly seen in proof assistants, such as The Rocq Prover (Coquand and Huet 1985),[26] Lean (Moura et al. 2015),[27] and Isabelle/HOL (Nipkow et al. 2002). These proof assistants typically use the Curry-Howard correspondence to prove propositions by encoding them in a dependent type system and then implementing a program of that type to serve as a proof witness. Tactics allow the programmer to focus on solving the goals, typically extracting the proof witness only as

[26] https://rocq-prover.org/docs/metaprogramming-ltac2
[27] https://lean-lang.org/doc/reference/latest/Tactic-Proofs

a byproduct. *Tactic combinators*, or *tacticals*, are used to combine simple tactics into more complex ones, to the point that a single tactic may perform *automated proof search* to solve a large range of goals.

There has been some effort towards introducing tactics outside of proof assistants. In general-purpose languages, the intended behavior of programs is typically not so rigorously defined, and as a result, there are not always clear goals to be discharged by tactics. In addition, we care more about the exact nature of programs, beyond just their existence as proof witnesses. In general-purpose statically typed languages, however, we can still use types as goals, and use tactics to introduce type-correct refinements for a sketch. An example is `refinery`,[28] a framework for tactic programming loosely based on (Sterling and Harper 2017), which used to power the Wingman for Haskell synthesis tool.[29] Wingman allowed the programmer to refine their programs by applying tactics through the language server protocol.

Driver uses a tactic language inspired by `refinery` to provide a structured way of exploring sketches, but extends it in two ways: Driver uses backward reasoning not just with types, but also with input-output examples (i.e. example propagation); and Driver tactics can return multiple possible sketches, such that tactics build up a program space that can be traversed and enumerated.

## 4.7 Conclusion

We have shown how automated reasoning about the feasibility of polymorphic functions with input-output examples can be made efficient enough to serve as a pruning tool during type-and-example-driven program synthesis, ensuring that every synthesis state corresponds to a correct (albeit partial) implementation. These guarantees allow the use of a biased choice operator, which in turn allows the programmer to use more building blocks during synthesis, without the downside of additional branching. Example coverage checking allows the synthesizer to automatically finish any subgoals that correspond to a total implementation, without the need to further explore the search space.

### 4.7.1 Limitations and Future Work

Taxi can only deal with conjunctions of input-output examples, as opposed to more complex constraints, such as input-output traces. As such, Driver may get stuck when trying to apply a recursion scheme to trace incomplete input-output examples. This is particularly difficult to avoid when the recursion occurs deeper in the program, as it is unlikely that propagated input-output examples turn out to be trace-complete, even if the top-level input-output examples *are* trace-complete. An SMT-based approach can represent more complex constraints, and therefore reason about the feasibility of input-output traces, at the cost of efficiency, transparency, and decidability. Future work could explore how such constraints can still be solved efficiently, as well as how example propagation can be adapted to allow propagating such constraints further.

[28] https://github.com/TOTBWF/refinery

[29] https://hackage.haskell.org/package/hls-tactics-plugin

Various functions require more complex recursion patterns than the ones used in this chapter. For example, the typical implementation for `zip` recurses over both input lists in tandem, requiring a specialized tactic. Determining which recursion schemes are worth including and figuring out how their inclusion influences performance requires more experimentation.

# 5

# Finding the Right Level of Abstraction

In Chapter 4, we found that, while feasibility reasoning itself does not have a big effect in pruning the search space, it allows the synthesizer to try out various refinements at different levels of abstraction without additional branching. The exploration of this idea was, however, very minimal: we only considered `map` and `filter` as concrete instantiations of `foldr`. In this chapter, we will explore, at a high level, different situations where feasibility reasoning can be used to find the right level of abstraction. The ideas presented here are at various levels of completion, and are, as such, unpublished.

## 5.1 A Hierarchy of Refinements

Any function that can be implemented in terms of `map` can also be implemented in terms of `foldr`. This means that, for any specification, if the refinement ① ⊑ `map` ② is correct, then so is ① ⊑ `foldr` ② ③. Conversely, if ① ⊑ `foldr` ② ③ is not a correct refinement, then neither is ① ⊑ `map` ②. We can capture this relation in a partial order, stating that `map` ≺ `foldr`. We say that `foldr` "abstracts over" `map`.

Now consider a situation where we have many refinements, all possibly related to one another in terms of their level of abstraction. For example, when trying to implement the function `takeThree` (which returns the first three elements of a list), we may have access to `map`, `filter`, and `foldr`, as well as indexed variants of each combinator (prefixed by `i`), whose arguments have access to the index of the list elements. Figure 14 shows how all these refinements relate to one another in a lattice structure, and whether each of them is a valid refinement for `takeThree`.

«OPEN QUESTION»

*For a set of functions at varying levels of abstraction, can we efficiently find which correspond to valid refinements?*

In Section 4.4.5, we use the biased choice operator ◁ to ensure that `foldr` is only tried if both `map` and `filter` fail. We may want to generalize this to other combinators, such that a function is only used as a refinement if all functions it abstracts over are not valid

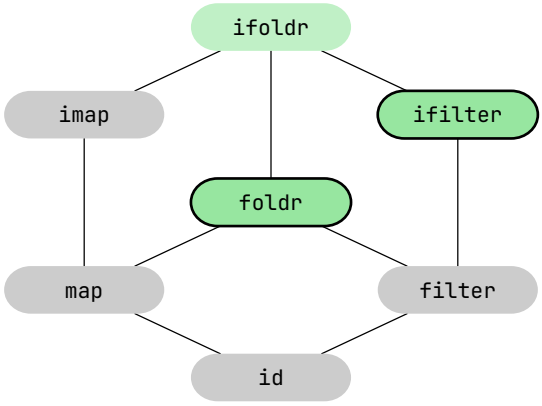



Figure 14. A hierarchy of possible functions to use as refinements for the function `takeThree`. Incorrect refinements are shown in gray, correct ones in green, and of those, the ones on the frontier are outlined.

refinements. For example, the indexed combinators are only tried when their non-indexed counterparts fail. Capturing all this behavior using just a combination of ◁ and || might get tricky and cumbersome, akin to using many nested **`if`**-statements. A better solution would be to compute the *frontier* of the lattice shown in Figure 14, i.e. the set of most concrete functions that are valid refinements.

## 5.2 A Hierarchy of Types

In type-driven synthesis, the goal type has a big influence on the program space. More abstract types lead to smaller program spaces, but a type that is too abstract will no longer include the program we are interested in. Finding the right level of abstraction is thus a crucial part of performing efficient synthesis. This is, however, not always an easy task, in particular for beginner programmers. James et al. (2020) show how to guess likely type signatures of a function based on a set of input-output examples. They present these type signatures to the user and ask them to pick the most appropriate one. For instance, for the input-output example `dedup "abaa" ≡≡ "aba"`, they list the following types, of which the 5th is the desired one:

(1) `∀a. [a] → [a]`
(2) `∀a. a → a`
(3) `[Char] → [Char]`
(4) `∀a. Ord a ⇒ [a] → [a]`
(5) `∀a. Eq a ⇒ [a] → [a]`
(6) `∀a. [a] → [Char]`
(7) `∀a. Ord a ⇒ [a] → [Char]`
(8) `∀a. Eq a ⇒ [a] → [Char]`

By presenting the user with these options and allowing them to pick the appropriate type, the synthesizer can efficiently search for programs at the right level of abstraction. This still relies on the expertise of the user to select the right type among these suggestions.

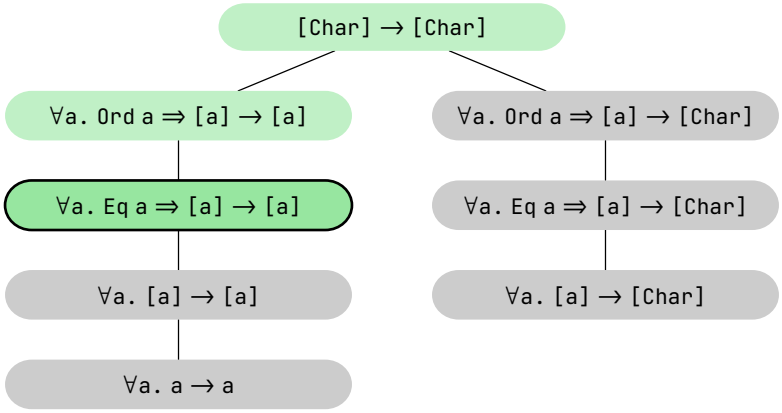



Figure 15. A hierarchy of possible type signatures for the function `dedup`. Infeasible types are shown in gray, feasible ones in green, and of those, the most abstract one is outlined.

«OPEN QUESTION»

*Given a list of likely type signatures, can we use feasibility reasoning to automatically find the right type signature?*

Similar to how we can prune suggested refinements by checking their feasibility, we can also prune suggested type signatures. With just the single input-output example, we can find a contradiction with suggestion (2), as only the identity function has type `∀a. a → a`. To discard more suggestions, we need additional input-output examples:

- The input-output examples should capture that the `Eq` constraint is relevant for `dedup`. In particular, we need two examples whose inputs can only be distinguished by inspecting the list elements. For instance, by adding `dedup "aaaa" ≡ "a"`, suggestions (1) and (6) can be discarded.
- The input-output examples should capture that the input elements are relevant for computing the output. For instance, by adding `dedup "acaa" = "aca"`, suggestions (6), (7), and (8) can be discarded.

By selecting the most abstract of the remaining types, we get the ideal type for `dedup`. Infeasible type signatures will not be shown to the user, and the feasible ones can be shown from most to least abstract. Interaction with the user can even be avoided by simply synthesizing at the most abstract, yet feasible type(s). As we did with refinements, we can represent type signatures in a hierarchy based on their level of abstraction, see Figure 15. Finding the most abstract, yet feasible types, corresponds to computing the frontier of this hierarchy with respect to the input-output examples.

### 5.2.1 Inferring Generalized Examples

Rather than present the types to the user, we may also present them other input-output examples that are implied by specific types. For example, if the type is `∀a. [a] → [Char]`, then the example `dedup "abaa" ≡ "aba"` implies `dedup "aaaa" ≡ "aba"`. By generating a set of examples based on the various possible type signatures and asking the programmer

which ones are correct, we can find the correct type, without requiring the user to understand the types.

«OPEN QUESTION»

*Can we programmatically find distinguishing examples and present them to the user to find the right type signature, without the user needing to understand (ad hoc) polymorphic types?*

### 5.2.2 Type Abstraction Tactics

So far, we considered inferring a type signature from a set of examples *before* performing program synthesis. In some cases, however, we may want to make goal types more abstract *during* synthesis. In Table 5, we saw how the function `maximum : ∀a. Ord a ⇒ [a] → Maybe a` is very easily synthesized. In fact, due to example coverage checking, it only takes a single tactic (`introFold`) to synthesize it. At the type `maximum : [Nat] → Maybe Nat`, this is significantly harder.

Now consider a slightly different function, `maximum0 : [Nat] → Nat`, which returns `0` when the input list is empty and otherwise returns the maximum. It is not possible to make its type more abstract, since the base case `0` has type `Nat`. After using the `introCase` tactic, the base case is filled automatically using example coverage checking, and we are left with the following sketch:

```
maximum0 : [Nat] → Nat
maximum0 xs = case xs of
  Nil → 0
  Cons y ys → ①
```

Interestingly, to fill hole ①, we do not need as concrete a type anymore. Consider the tactic `natToOrd` that replaces any occurrence of `Nat` in the goal type with a fresh type variable constrained by `Ord`, essentially refining the sketch as follows:

```
maximum0 : [Nat] → Nat
maximum0 xs = case xs of
  Nil → 0
  Cons y ys → helper y ys
  where helper : Ord a ⇒ a → [a] → [a]
        helper = ②
```

The feasibility of ② is checked to ensure that this is a valid refinement. As with `maximum`, the `helper` function is synthesized using just the `introFold` tactic. While this example shows that `natToOrd` can be a powerful tactic, it is highly specialized to this particular case. Ideally, we would have a more generic `typeAbstraction` tactic that uses the same procedure as we saw before to find the most abstract type that is still feasible. Still, it may not be trivial to know exactly when to apply this tactic.

«OPEN QUESTION»

*Can we speed up program synthesis by figuring out when the type of a hole can be made more abstract?*

## 5.3 Relevancy Reasoning

During top-down sketching of a program, variables may be introduced, but they never leave the scope. Those variables may be irrelevant, or only be relevant in part of the scope, and there is no way to refine the scope manually.[30] This can lead to the scopes of holes getting cluttered. It would be useful to figure out programmatically whether variables in scope are still relevant.

In program synthesis, removing irrelevant variables from the scope is of particular interest, as they can have a big influence on the search space. Additionally, having more variables in scope makes it harder to find a total implementation through example coverage checking. To see why this is the case, consider the type `∀ a. Nat → a → a`. The first argument cannot be relevant for computing the output, but we also cannot cover all possible inputs of type `Nat`.

A simple example of irrelevancy is when two variables are equal to each other, in which case at most one of them is relevant. Consider, for example, the following sketch:

```
if x == y then ① else ②
```

In the hole ①, the variables `x` and `y` will be equal, and thus equally relevant. It is therefore not necessary to have both of them in scope. As such, we can give preference to either of them and ignore the other variable. It may be the case, however, that neither `x` nor `y` is relevant. One way to check this would be to remove `x` and `y` from the scope of ① and check if it is still feasible.

For a more involved example, consider the following sketch for the `maximum` function, which uses a paramorphism:

```
maximum : ∀ a. Ord a ⇒ [a] → Maybe a
maximum = para (λh t r → ①) Nothing
```

The variable `h` represents the head of a list, the variable `t` its tail, and the variable `r` the recursive result of calling `maximum` on the tail. The variable `h` is relevant, since we should always consider every element when computing the maximum. The variables `r` and `t` are equally relevant: both can be used to complete the sketch (since both contain the relevant information of which element in the tail is largest).

Using this reasoning, when presenting the variables in the scope of ① to the programmer, we can first present `h`, the most relevant variable, and then `r` and `t`, which are equally relevant. Ideally, we would combine this with the fact that `r` contains less information than `t` (it only contains the relevant largest element of the tail, rather than the whole tail), and reason that this makes it easier to extract the relevant information from `r`.[31] As such, we would present `r` before `t`. The same logic can be used in program synthesis, where we would conclude that it is reasonable to ignore the variable `t`, after which example coverage checking kicks in, finding a correct solution:

[30] Unless we explicitly introduce a helper function to create a new scope.

[31] Some other heuristics may also be used in deciding the order of variables.

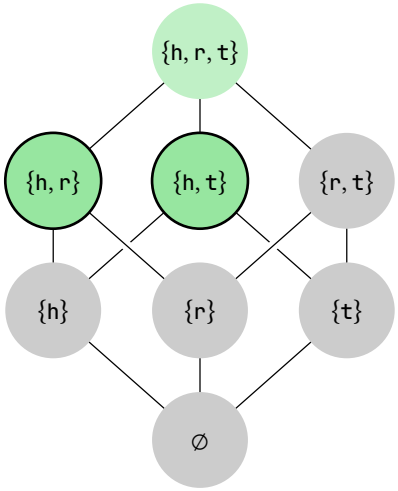



Figure 16. The lattice of subsets of variables and their feasibility w.r.t. the hole ① in the sketch `maximum = para (λh t r → ①) Nothing`. All infeasible subsets of variables are shown in gray, feasible in green, and of those, the ones on the frontier are outlined, which represent subsets of variables of which all variables are relevant.

```
maximum = para (λh _t r → case r of
    Nothing → Just h
    Just m  → Just (max h m)
  ) Nothing
```

We can visualize the relevance of variables in a lattice, see Figure 16. The goal of relevancy reasoning is to efficiently divide this lattice into feasible and infeasible variable sets. Most interesting are the variable sets on the frontier, i.e. the smallest sets of variables that are still feasible, since they only contain variables that are relevant.

«OPEN QUESTION»

*Can we efficiently compute the relevancy of all (sets of) variables in scope and use this information to speed up program synthesis?*

# Part III

# Closing

Contents

# 6

# Discussion

In this thesis, I investigated the use of input-output examples in type-directed program synthesis. I set out to answer the following research question:

**RQ.** *How can input-output examples be used to guide the search in top-down type-directed program synthesis, such that all intermediate steps are valid refinements?*

To answer this question, I looked at how to preserve example information during component-based program synthesis; how to automate reasoning about the feasibility of polymorphic programs constrained by input-output examples; and how to use input-output examples to guide the refinement of programs during synthesis.

**SQ1.** *Can example information be preserved during sketching with arbitrary components?*

The goal of preserving example information is to allow easy reasoning about the input-output behavior of subproblems. Typically, the language required to express example information gets more complex as it is propagated through a sketch: when starting with just input-output examples, many refinements will introduce disjunctions or input-output traces. To make matters worse, propagating this information again may introduce even more complexity.

Ideally, problems and their subproblems are constrained in the same language, i.e. the constraint language should be closed under example propagation. This means that one has to choose between having constraints written in a complex logic that is hard to reason about, or restricting refinements to *well-behaved* components that keep the complexity low. Scrybe takes the former approach, constraining holes with the results of live bidirectional evaluation (Lubin et al. 2020). While this allows synthesis using arbitrary components, Scrybe occasionally runs into exponential or even diverging constraints. Driver takes the latter approach, using only refinements that result in holes constrained by simple input-output examples. This makes Driver less flexible, but allows for better reasoning about hole constraints.

**SQ2.** *Can parametricity be used to guarantee that only feasible sketches are explored?*

Example propagation ensures that the sketches explored during synthesis never contradict the input-output examples. Driver uses this technique to prune the search during type-driven synthesis. When dealing with polymorphic programs, however, there are cases where both the examples and type are individually feasible, but their combination is not. It turns out that polymorphic types and input-output examples interact with each other through parametricity, which can make otherwise feasible specifications infeasible. As a result, some infeasible sketches will still be explored.

In Chapter 3, we show how to automate reasoning about the feasibility of programs constrained by a polymorphic type and input-output examples, as long as we restrict the types to morphisms between container functors. By using an SMT solver, we can reason about complex example constraints, such as the input-output traces encountered by Scrybe. This comes, however, at the cost of efficiency, to the point that solver-based feasibility reasoning is too slow for use during program synthesis, where it would be called many times over. In Chapter 4, we define Taxi, a more efficient implementation of feasibility reasoning, specialized to input-output examples. By restricting the synthesis to well-behaved components, Driver can call Taxi to ensure that any intermediate synthesis state is feasible, i.e. every intermediate sketch can be refined to a correct (albeit possibly partial) solution.

**SQ3.** *Can examples be used to decide which refinements to introduce?*

The combination of example propagation and feasibility reasoning guarantees that only valid refinements are introduced during type-driven enumeration. In addition, we show how to use example information to guide the introduction of refinements, helping decide which refinements should be explored and in which order. Scrybe gives preference to refinements that lead to simpler constraints, and recognizes which holes should be refined next so the sketch can be evaluated further. Driver uses example coverage checking to recognize when a set of input-output examples corresponds to a total implementation, and refines the corresponding hole using program extraction, thereby shortcutting the search. In addition, Driver allows expressing dependencies between tactics in terms of their feasibility, such as "only use `foldr` if `map` and `filter` fail." This allows Driver to benefit from more specialized tactics like `map` and `filter`, without introducing additional branching.

## 6.1 Conclusion & Future Work

We have shown that it is possible to ensure that every intermediate program explored during top-down type-directed program synthesis corresponds to a correct (albeit partial) solution. We start by checking the feasibility of the input specification, taking parametricity into account to ensure that the examples and polymorphic type agree with each other. Program extraction guarantees a partial solution, but this solution is undefined on any inputs not covered by the specification. To alleviate this, the partial solution can be refined using common recursion schemes, generalizing beyond the input-output examples in a natural way. By restricting refinements to those that propagate *both* type and example information, we build up a program space of increasingly more refined programs that are all correct by construction. The choices made during exploration of this program space determine how the solution generalizes the input specification. As soon as the programmer has sufficiently

expressed their intent, that is, when the combination of a type, examples, and a sketch corresponds to a total solution, we can easily recognize this and extract that solution.

Throughout this thesis, we developed several tools for program analysis and synthesis. While these tools are useful in their own right, it would be valuable to apply them in a real-world setting. We envision several areas where the ideas developed in this thesis could be put into practice:

**API Search** Finding the right function in an API can be difficult. Rather than guessing the name of a hypothetical function, API search engines such as Hoogle allow searching based on the type of a function.[32] Hoogle+ (James et al. 2020) and Hoogle⋆ (Botelho Guerra et al. 2023) additionally allow the user to provide input-output examples, and they return not just a single function, but perform component-based synthesis to find a program with the expected behavior. If it is not possible to find a complete function according to the user's intent, we expect it might be possible to use top-down synthesis with feasibility reasoning to find a sketch composed of functions from the API, with clear subgoals for the user to solve. Hoogle+ also infers likely type signatures from input-output examples, but leaves the exact choice of signature to the user. Feasibility reasoning might be used to help the user find the right type signature simply by specifying the expected behavior in terms of input-output examples.

**Programming Tutors** Programming tutors provide an environment for students to learn how to write code, typically by working on a programming exercise and receiving feedback along the way. An example of such a programming tutor is Ask-Elle (Gerdes et al. 2017), which compares student solutions to model solutions to let them know whether they are on the right track, as well as giving next-step hints on how to proceed. These techniques rely on the ability to match student solutions to one of the model solutions. When this is not possible, feasibility reasoning might be used to see if the student is on the right track, and top-down synthesis to give next-step hints.

**Compiler Feedback** Programmers of statically typed languages rely on the compiler not just to generate executable code, but also to perform static analysis of the program and give feedback, even if they know their program will not compile (Lubin and Chasins 2021). Programming with holes allows the programmer to be explicit about the parts of the program that are unfinished, and in turn the compiler can let the programmer know the inferred type of the hole, or even give suggestions of valid refinements (Gissurarson 2018). Typically, however, input-output examples are not taken into account when giving suggestions for hole refinements. Feasibility reasoning may be used to ensure that hole suggestions are valid with respect to the expected input-output behavior.

Of course, our work is not without its limitations, which should be considered when applying the ideas presented in a different setting. In particular, we believe the following topics deserve further exploration:

[32] https://hoogle.haskell.org/

**Strict Positivity** Feasibility reasoning as described in this thesis only works on container morphisms, that is, functions with strictly positive inputs and outputs, with the exception of `Eq` and `Ord` constraints. Can we extend this further, reasoning about, for example, `Monoid` constraints and higher-order functions? Bernardy et al. (2010) use a representation similar to, but distinct from, container morphisms to reason about parametricity in the context of property-based testing. It would be interesting to see if this representation also works well for feasibility reasoning.

**Decidability** Example propagation may lead to very large or complex hole constraints. To what extent is reasoning about such constraints still decidable? The complexity of hole constraints depends heavily on which refinements are used. Can we categorize refinements based on the kind of constraints they introduce, and come up with guidelines to keep the complexity of hole constraints low?

**Explainability** The programs we reason about in this thesis are typically ones that a beginner programmer would write. Ideally, we would give them feedback about the feasibility of their program as they write it, but the underlying techniques (in particular parametricity) require a more advanced understanding of type theory. How do we explain the (in)feasibility of a problem to the user in a way that is understandable without necessarily having to grasp parametricity?

**Domain-Specific Languages** Feasibility states that a specification is not contradictory, and that there exists a function implementing that specification. We do not, however, consider the language in which this function is written. This is not a problem in general-purpose languages, as explored in this thesis, but in domain-specific languages, feasibility of a type and input-output examples does not necessarily imply a solution exist. *Unrealizability logic* (Kim et al. 2023) considers the language as part of the specification when reasoning about the feasibility of programs. Can we use these techniques to extend our work on feasibility reasoning to the setting of domain-specific languages?

Part IV

# Appendices

Contents

# Bibliography


[1] M. Felleisen, R. B. Findler, M. Flatt, and S. Krishnamurthi, *How to Design Programs: An Introduction to Programming and Computing*. The MIT Press, 2018. [Online]. Available: https://htdp.org/

[2] P.-M. Osera and S. Zdancewic, "Type-and-Example-Directed Program Synthesis," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation*, in PLDI '15. Portland, OR, USA: Association for Computing Machinery, 2015, pp. 619–630. doi: 10.1145/2737924.2738007.

[3] J. K. Feser, S. Chaudhuri, and I. Dillig, "Synthesizing data structure transformations from input-output examples," *ACM SIGPLAN Notices*, vol. 50, no. 6, pp. 229–239, Aug. 2015, doi: 10.1145/2813885.2737977.

[4] J. C. Reynolds, "Types, Abstraction and Parametric Polymorphism," in *IFIP Congress*, 1983.

[5] N. Mulleners, J. Jeuring, and B. Heeren, "Program Synthesis Using Example Propagation," in *Practical Aspects of Declarative Languages: 25th International Symposium, PADL 2023, Boston, MA, USA, January 16–17, 2023, Proceedings*, Boston, MA, USA: Springer-Verlag, 2023a, pp. 20–36. doi: 10.1007/978-3-031-24841-2_2.

[6] N. Mulleners, J. Jeuring, and B. Heeren, "Example-Based Reasoning about the Realizability of Polymorphic Programs," *Proc. ACM Program. Lang.*, vol. 8, no. ICFP, Aug. 2024b, doi: 10.1145/3674636.

[7] N. Mulleners, J. Jeuring, and W. Swierstra, "Hole Refinements for Polymorphic Type-and-Example Driven Synthesis," in *Proceedings of the 2026 ACM SIGPLAN International Workshop on Partial Evaluation and Program Manipulation*, in PEPM '26. Rennes, France: Association for Computing Machinery, 2026c, pp. 17–30. doi: 10.1145/3779209.3779535.

[8] Z. Guo *et al.*, "Program Synthesis by Type-Guided Abstraction Refinement," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371080.

[9] J. Koppel, Z. Guo, E. de Vries, A. Solar-Lezama, and N. Polikarpova, "Searching Entangled Program Spaces," *Proc. ACM Program. Lang.*, vol. 6, no. ICFP, Aug. 2022, doi: 10.1145/3547622.

[10] C. Smith and A. Albarghouthi, "Program Synthesis with Equivalence Reduction," in *Verification, Model Checking, and Abstract Interpretation*, C. Enea and R. Piskac, Eds., Cham: Springer International Publishing, 2019, pp. 24–47. doi: 10.1007/978-3-030-11245-5_2.

[11] S. Katayama, "Efficient Exhaustive Generation of Functional Programs Using Monte-Carlo Search with Iterative Deepening," in *PRICAI 2008: Trends in Artificial Intelligence*, T.-B. Ho and Z.-H. Zhou, Eds., Springer Berlin Heidelberg, 2008, pp. 199–210. doi: 10.1007/978-3-540-89197-0_21.

[12] H. Peleg, R. Gabay, S. Itzhaky, and E. Yahav, "Programming with a Read-Eval-Synth Loop," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020, doi: 10.1145/3428227.

[13] J. Frankle, P.-M. Osera, D. Walker, and S. Zdancewic, "Example-Directed Synthesis: A Type-Theoretic Interpretation," *SIGPLAN Not.*, vol. 51, no. 1, pp. 802–815, Jan. 2016, doi: 10.1145/2914770.2837629.

[14] J. Lubin, N. Collins, C. Omar, and R. Chugh, "Program Sketching with Live Bidirectional Evaluation," *Proc. ACM Program. Lang.*, vol. 4, no. ICFP, Aug. 2020, doi: 10.1145/3408991.

[15] M. Hofmann and E. Kitzelmann, "I/O Guided Detection of List Catamorphisms: Towards Problem Specific Use of Program Templates in IP," in *Proceedings of the 2010 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*, in PEPM '10. Madrid, Spain: Association for Computing Machinery, 2010, pp. 93–100. doi: 10.1145/1706356.1706375.

[16] A. Solar-Lezama, "Program Synthesis by Sketching," Doctoral dissertation, 2008. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[17] A. Solar-Lezama, "The Sketching Approach to Program Synthesis," in *Programming Languages and Systems*, Z. Hu, Ed., Springer Berlin Heidelberg, 2009, pp. 4–13. doi: 10.1007/978-3-642-10672-9_3.

[18] O. Danvy and M. Spivey, "On Barron and Strachey's Cartesian Product Function," in *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming*, in ICFP '07. Freiburg, Germany: Association for Computing Machinery, 2007, pp. 41–46. doi: 10.1145/1291151.1291161.

[19] C. Omar, I. Voysey, R. Chugh, and M. A. Hammer, "Live Functional Programming with Typed Holes," *Proc. ACM Program. Lang.*, vol. 3, no. POPL, Jan. 2019, doi: 10.1145/3290327.

[20] K. Claessen, N. Smallbone, and J. Hughes, "QuickSpec: Guessing Formal Specifications Using Testing," in *Tests and Proofs*, G. Fraser and A. Gargantini, Eds., Springer Berlin Heidelberg, 2010, pp. 6–21. doi: 10.1007/978-3-642-13977-2_3.

[21] N. Mulleners, "Scrybe - Component-Based Synthesis Using Example Propagation." Zenodo, Jul. 2026a. doi: 10.5281/zenodo.21415976.

[22] J. Kim, L. D'Antoni, and T. Reps, "Unrealizability Logic," *Proc. ACM Program. Lang.*, vol. 7, no. POPL, Jan. 2023, doi: 10.1145/3571216.

[23] P. Urzyczyn, "Inhabitation in typed lambda-calculi (a syntactic approach)," in *Typed Lambda Calculi and Applications*, P. de Groote and J. Roger Hindley, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 373–389. doi: 10.1007/3-540-62688-3_47.

[24] P. Wadler, "Theorems for Free!," in *Proceedings of the Fourth International Conference on Functional Programming Languages and Computer Architecture*, in FPCA '89. Imperial College, London, United Kingdom: Association for Computing Machinery, 1989, pp. 347–359. doi: 10.1145/99370.99404.

[26] GHC Team, "GHC 9.8.1 User's Guide." Accessed: Nov. 16, 2023. [Online]. Available: https://downloads.haskell.org/~ghc/9.8.1/docs/users_guide/exts/typed_holes.html

[25] M. P. Gissurarson, "Suggesting valid hole fits for typed-holes (experience report)," *SIGPLAN Not.*, vol. 53, no. 7, pp. 179–185, Sep. 2018, doi: 10.1145/3299711.3242760.

[27] U. Norell, "Dependently Typed Programming in Agda," in *Advanced Functional Programming: 6th International School, AFP 2008, Heijen, The Netherlands, May 2008, Revised Lectures*, P. Koopman, R. Plasmeijer, and D. Swierstra, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 230–266. doi: 10.1007/978-3-642-04652-0_5.

[28] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. A. Hammer, "Hazelnut: a bidirectionally typed structure editor calculus," in *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, in POPL '17. Paris, France: Association for Computing Machinery, 2017, pp. 86–99. doi: 10.1145/3009837.3009900.

[29] M. Abbott, T. Altenkirch, and N. Ghani, "Containers: Constructing Strictly Positive Types," *Theor. Comput. Sci.*, vol. 342, no. 1, pp. 3–27, Sep. 2005, doi: 10.1016/j.tcs.2005.06.002.

[30] M. Abbott, T. Altenkirch, and N. Ghani, "Categories of Containers," in *Proceedings of the 6th International Conference on Foundations of Software Science and Computation Structures and Joint European Conference on Theory and Practice of Software*, in FOSSACS'03/ETAPS'03. Warsaw, Poland, 2003, pp. 23–38. doi: 10.1007/3-540-36576-1_2.

[31] R. Prince, N. Ghani, and C. McBride, "Proving Properties about Lists Using Containers," in *Proceedings of the 9th International Conference on Functional and Logic Programming*, in FLOPS'08. Ise, Japan: Springer-Verlag, 2008, pp. 97–112. doi: 10.1007/978-3-540-78969-7_9.

[32] J. Gibbons, G. Hutton, and T. Altenkirch, "When is a function a fold or an unfold?," *Electronic Notes in Theoretical Computer Science*, vol. 44, no. 1, pp. 146–160, 2001, doi: 10.1016/S1571-0661(04)80906-X.

[33] M. Hofmann, "Data-Driven Detection of Catamorphisms - Towards Problem Specific Use of Program Schemes for Inductive Program Synthesis," in *Preproceedings of the 22nd Symposium on Implementation and Application of Functional Languages (IFL 2010) ; Alphen aan den Rijn, 1. - 3. Sept. 2010*, J. Hage, Ed., Utrecht, 2010, pp. 25–39.

[34] N. Mulleners, "Parametrickery - Feasibility of Polymorphic Programs." Zenodo, Jul. 2026b. doi: 10.5281/zenodo.21415997.

[35] L. de Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Tools and Algorithms for the Construction and Analysis of Systems*, C. R. Ramakrishnan and J. Rehof, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 337–340. doi: 10.1007/978-3-540-78800-3_24.

[38] P.-M. Osera, "Constraint-Based Type-Directed Program Synthesis," in *Proceedings of the 4th ACM SIGPLAN International Workshop on Type-Driven Development*, in TyDe 2019. Berlin, Germany: Association for Computing Machinery, 2019, pp. 64–76. doi: 10.1145/3331554.3342608.

[37] J. P. Inala, N. Polikarpova, X. Qiu, B. S. Lerner, and A. Solar-Lezama, "Synthesis of Recursive ADT Transformations from Reusable Templates," in *Tools and Algorithms for the Construction and Analysis of Systems*, A. Legay and T. Margaria, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2017, pp. 247–263. doi: 10.1007/978-3-662-54577-5_14.

[36] N. Polikarpova, I. Kuraj, and A. Solar-Lezama, "Program Synthesis from Polymorphic Refinement Types," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, in PLDI '16. Santa Barbara, CA, USA: Association for Computing Machinery, 2016, pp. 522–538. doi: 10.1145/2908080.2908093.

[39] M. P. Gissurarson, D. Roque, and J. Koppel, "Spectacular: Finding Laws from 25 Trillion Terms," in *IEEE Conference on Software Testing, Verification and Validation, ICST 2023, Dublin, Ireland, April 16-20, 2023*, IEEE, 2023, pp. 293–304. doi: 10.1109/ICST57152.2023.00035.

[40] W. Lee and H. Cho, "Inductive Synthesis of Structurally Recursive Functional Programs from Non-Recursive Expressions," *Proc. ACM Program. Lang.*, vol. 7, no. POPL, Jan. 2023, doi: 10.1145/3571263.

[41] D. Seidel and J. Voigtländer, "Proving Properties about Functions on Lists Involving Element Tests," in *Proceedings of the 20th International Conference on Recent Trends in Algebraic Development Techniques*, in WADT'10. Etelsen, Germany: Springer-Verlag, 2010, pp. 270–286. doi: 10.1007/978-3-642-28412-0_17.

[42] J.-P. Bernardy, P. Jansson, and K. Claessen, "Testing Polymorphic Properties," in *Programming Languages and Systems*, A. D. Gordon, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 125–144. doi: 10.1007/978-3-642-11957-6_8.

[43] F. Teegen, K.-O. Prott, and N. Bunkenburg, "Haskell$^{-1}$: Automatic Function Inversion in Haskell", in *Proceedings of the 14th ACM SIGPLAN International Symposium on Haskell*, in Haskell 2021. Virtual, Republic of Korea: Association for Computing Machinery, 2021, pp. 41–55. doi: 10.1145/3471874.3472982.

[44] Q. Hu and L. D'Antoni, "Syntax-Guided Synthesis with Quantitative Syntactic Objectives," in *Computer Aided Verification*, H. Chockler and G. Weissenbacher, Eds., Cham: Springer International Publishing, 2018, pp. 386–403. doi: https://doi.org/10.1007/978-3-319-96145-3_21.

[45] Q. Hu, J. Breck, J. Cyphert, L. D'Antoni, and T. Reps, "Proving Unrealizability for Syntax-Guided Synthesis," in *Computer Aided Verification*, I. Dillig and S. Tasiran, Eds., Cham: Springer International Publishing, 2019, pp. 335–352. doi: 10.1007/978-3-030-25540-4_18.

[46] Q. Hu, J. Cyphert, L. D'Antoni, and T. Reps, "Exact and approximate methods for proving unrealizability of syntax-guided synthesis problems," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, in PLDI 2020. London, UK: Association for Computing Machinery, 2020, pp. 1128–1142. doi: 10.1145/3385412.3385979.

[47] J. Kim, Q. Hu, L. D'Antoni, and T. Reps, "Semantics-guided synthesis," *Proc. ACM Program. Lang.*, vol. 5, no. POPL, Jan. 2021, doi: 10.1145/3434311.

[48] A. Farzan, D. Lette, and V. Nicolet, "Recursion synthesis with unrealizability witnesses," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, in PLDI 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 244–259. doi: 10.1145/3519939.3523726.

[49] J. Lubin and S. E. Chasins, "How statically-typed functional programmers write code," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Oct. 2021, doi: 10.1145/3485532.

[50] M. Jaskelioff and R. O'Connor, "A representation theorem for second-order functionals," *Journal of Functional Programming*, vol. 25, p. e13, 2015, doi: 10.1017/S0956796815000088.

[51] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische mathematik*, vol. 1, no. 1, pp. 269–271, 1959, doi: 10.1007/BF01386390.

[52] N. Mulleners, "Taxi-Driver - Type-and-Example-Driven Refinements." Zenodo, Jul. 2026c. doi: 10.5281/zenodo.21414599.

[53] L. Meertens, "Paramorphisms," *Form. Asp. Comput.*, vol. 4, no. 5, pp. 413–424, Sep. 1992, doi: 10.1007/BF01211391.

[54] Y. Yuan, S. Guest, E. Griffis, H. Potter, D. Moon, and C. Omar, "Live Pattern Matching with Typed Holes," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, Apr. 2023, doi: 10.1145/3586048.

[55] E. Zhao, R. Maroof, A. Dukkipati, A. Blinn, Z. Pan, and C. Omar, "Total Type Error Localization and Recovery with Holes," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, Jan. 2024, doi: 10.1145/3632910.

[56] H. Botelho Guerra, J. F. Ferreira, and J. Costa Seco, "Hoogle★: Constants and λ-abstractions in Petri-net-based Synthesis using Symbolic Execution," in *37th European Conference on Object-Oriented Programming (ECOOP 2023)*, K. Ali and G. Salvaneschi, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 263. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, p. 4:1–4:28. doi: 10.4230/LIPIcs.ECOOP.2023.4.

[57] C. Morgan, "Programming from specifications," in *Prentice Hall International Series in computer science*, 1990. [Online]. Available: https://api.semanticscholar.org/CorpusID:3449831

[58] M. Gordon, R. Milner, and C. P. Wadsworth, *Edinburgh LCF: A Mechanized Logic of Computation*, no. 78. in Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1979. doi: 10.1007/3-540-09724-4.

[59] T. Coquand and G. Huet, "Constructions: A higher order proof system for mechanizing mathematics," in *EUROCAL '85*, B. Buchberger, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 1985, pp. 151–184. doi: 10.1007/3-540-15983-5_13.

[60] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean Theorem Prover (System Description)," in *Automated Deduction - CADE-25*, A. P. Felty and A. Middeldorp, Eds., Cham: Springer International Publishing, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.

[61] T. Nipkow, M. Wenzel, and L. C. Paulson, *Isabelle/HOL: a proof assistant for higher-order logic*. Berlin, Heidelberg: Springer-Verlag, 2002. doi: 10.1007/3-540-45949-9.

[62] J. Sterling and R. Harper, "Algebraic Foundations of Proof Refinement," *ArXiv*, vol. abs/1703.05215, 2017, [Online]. Available: https://api.semanticscholar.org/CorpusID:15770001

[63] M. B. James *et al.*, "Digging for fold: synthesis-aided API discovery for Haskell," *Proc. ACM Program. Lang.*, vol. 4, no. OOPSLA, Nov. 2020, doi: 10.1145/3428273.

[64] A. Gerdes, B. Heeren, J. Jeuring, and L. T. van Binsbergen, "Ask-Elle: an Adaptable Programming Tutor for Haskell Giving Automated Feedback," *International Journal of Artificial Intelligence in Education*, vol. 27, no. 1, pp. 65–100, Mar. 2017, doi: 10.1007/s40593-015-0080-x.

Part V

# Back matter

Contents

# English Summary

Computer programs describe instructions for the computer to follow, and the computer will happily follow those instructions without asking questions. Humans, however, tend to make mistakes, leading the computer to do the wrong thing. To try and avoid these situations, programmers often write specifications. These describe some expectation of the programmer about the program in question. This thesis focuses on two common types of specifications: *tests* and *types*. Tests describe which output the programmer expects for specific inputs. Types express the programmer's expectation about the structure of the data handled by the program. The main reason for writing a specification is that the computer can check automatically whether the program matches the programmer's expectation. It is often encouraged to write specifications *before* writing the program: it forces the programmer to think about their expectations, and it allows the computer to take those expectations into account when helping the programmer along. In practice, however, specifications are often not used to their full potential. For example, the computer may fail to recognize that (1) the programmer has introduced a mistake while the program cannot yet be executed, (2) the specification contains a contradiction, or (3) there is a way to fill in gaps in the program so that it meets the specification.

In this thesis, I develop techniques to get more out of specifications. These techniques are developed in the context of interactive program synthesis, a form of programming from specifications. In Chapter 2, I explore how bidirectional evaluation can be used to find mistakes in incomplete programs that are generated during program synthesis. In Chapter 3, I develop techniques for finding contradictions between tests and polymorphic types, and for proving the absence of such contradictions. In Chapter 4, I expand on these techniques and explore how to fill in gaps in a program so that it matches the specification. In Chapter 5, I look at different scenarios in which the techniques developed in this thesis can be applied.

# Nederlandse Samenvatting

Computerprogramma's beschrijven instructies die de computer hoort te volgen. De computer voert die maar al te graag uit zonder daar vragen bij te stellen. Mensen maken echter vaak fouten, waardoor de computer de verkeerde dingen uitvoert. Om dit soort situaties te voorkomen, schrijven programmeurs vaak specificaties. Deze beschrijven een verwachting van de programmeur. Dit proefschrift focust op twee veelgebruikte specificaties: *tests* en *types*. Tests beschrijven welke output de programmeur verwacht bij specifieke inputs. Types drukken de structuur van de data uit die de programmeur in diens programma verwacht. De voornaamste reden voor het schrijven van specificaties is dat de computer automatisch kan controleren of het programma aan de verwachting van de programmeur voldoet. Het wordt vaak aangeraden om eerst specificaties te schrijven en daarna pas het programma zelf. Dit dwingt de programmeur om na te denken over diens verwachtingen én het geeft de computer de mogelijkheid om met die verwachtingen rekening te houden om de programmeur op weg te helpen. In de praktijk worden specificaties echter meestal niet ten volle benut. De computer herkent bijvoorbeeld vaak niet dat (1) de programmeur een fout heeft geïntroduceerd nog voordat het programma uitgevoerd kan worden, (2) de specificatie een contradictie bevat, of (3) er een manier is om de gaten in het programma zó te vullen dat het aan de specificatie voldoet.

In dit proefschrift ontwikkel ik technieken om meer te halen uit specificaties. Deze technieken zijn ontwikkeld in de context van programmasynthese, een vorm van programmeren aan de hand van specificaties. In Hoofdstuk 2 onderzoek ik hoe bidirectionele evaluatie toegepast kan worden om fouten te vinden in incomplete programma's die tijdens programmasynthese gegenereerd zijn. In Hoofdstuk 3 ontwikkel ik technieken voor het vinden van contradicties tussen tests en polymorfe types, en voor het bewijzen dat zulke contradicties niet voorkomen. In Hoofdstuk 4 breid ik die technieken uit en onderzoek ik hoe gaten in een programma zó gevuld kunnen worden dat het aan de specificatie voldoet. In Hoodstuk 5 kijk ik naar verschillende scenario's waarin de technieken die ik in dit proefschrift ontwikkeld heb toegepast kunnen worden.

# List of Publications

## Research Articles

1. **Niek Mulleners**, Johan Jeuring, and Bastiaan Heeren (2023). Program Synthesis Using Example Propagation. In *Proceedings of the 25th International Symposium on Practical Aspects of Declarative Languages*.
2. **Niek Mulleners**, Johan Jeuring, and Bastiaan Heeren (2024). Example-Based Reasoning about the Realizability of Polymorphic Programs. In *Proceedings of the 29th ACM SIGPLAN International Conference on Functional Programming*.
3. **Niek Mulleners**, Johan Jeuring, and Wouter Swierstra (2026). Hole Refinements for Polymorphic Type-and-Example Driven Synthesis. In *Proceedings of the 2026 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation*.

# Curriculum Vitæ

Throughout his life, Niek has always been drawn to different ways to express himself and create things. As a kid, he spent much of his time drawing, worldbuilding, designing writing systems, and reenvisioning (in his mind) the video games he liked to play. His journey into computer science was ignited when he discovered that it was, in fact, possible to create your own games.

**2009 – 2015** *Sint-Maartenscollege, Maastricht*

Niek went to high school in Maastricht, where he developed an interest in technical sciences through the Nature & Technology track. In his free time, he learned to develop simple games in Game Maker and explored many creative hobbies, such as drawing, acting, songwriting, and playing in a band. He finished high school cum laude.

**2015 – 2018** *Bachelor Informatica at Universiteit Utrecht*

After considering both technical and creative studies, Niek decided to combine the two by choosing a computer science bachelor with a track in game development. Quickly, however, he became captivated with the more theoretical side of programming. In his second year, Niek fell in love with Haskell and learned to see programming and programming language design as a form of creativity and expression. He continued playing in bands and acting in plays. After three years, Niek finished his bachelor cum laude.

**2018 – 2020** *Master Computing Science at Universiteit Utrecht*

For his master's, Niek decided to stay at Utrecht University to enroll in the programming technologies track of the Computing Science master's program. He developed an interest in formal verification and program synthesis, and planned to do a thesis project in the latter. Program synthesis, however, was not taught at Utrecht University, so Niek ended up taking on a project with Wouter Swierstra on formal verification of effectful programs in terms of predicate transformer semantics. Enjoying the research process, he got interested in doing a PhD abroad. Unfortunately, at this time, the pandemic hit, and Niek decided it was not the right time to move abroad. Coincidentally, Johan Jeuring was looking for a PhD candidate to improve the automated feedback in the Ask-Elle tutor. Niek applied for this position, proposing the use of program synthesis to provide automated feedback. His proposal was accepted, and Niek started his PhD shortly after handing in his master's thesis. Despite a bit of delay, he finished his master's cum laude.

### 2020 – 2026 *PhD at Universiteit Utrecht*

Niek started his PhD in September 2020 with Johan Jeuring and Bastiaan Heeren. Despite the lockdown, he started with a sprint, handing in a first abstract describing his vision of program synthesis as a feedback tool at the brand new HATRA workshop after only two weeks. Niek started researching and developing program synthesis techniques, but at the end of the first year, he got stuck, having not made much progress. He ended up changing topics, only to change back half a year later. Around this time, the lockdown ended, and Niek decided that it was time for a change. He went on a research visit to Chalmers University of Technology in Göteborg, Sweden. There, he finished his first paper, and in collaboration with Koen Claessen came up with a new research direction, exploring the effects of parametricity on pruning during synthesis. This turned out to be a challenging paper to write, getting rejected several times. Eventually, the hard work paid off when the paper got accepted at ICFP ‘24, receiving a distinguished paper award. Niek continued this line of research, eventually handing in his thesis at the end of January 2026. Throughout his PhD, Niek explored many new hobbies, including bouldering, Lindy hop, improvisation theatre, volunteering, and studying French, Swedish, German, and Arabic.

### 2026 – present

With his partner Lis, Niek took some time off after handing in his thesis to travel and study Arabic abroad, before moving to Austria.

# Acknowledgements

Throughout the course of my PhD, there were times when I had no idea what I was doing or whether I was in the right place. There were even several times when I was about to give up. And I would have given up if it were not for my wonderful friends, family, and colleagues. I am very happy with where I am today, and I would not have managed without you. So thank you, for being there, for listening, for laughing and complaining, and for believing in me, especially when I did not.

---

Firstly, thank you to all my colleagues at Utrecht University. You made coming to the office far more enjoyable than working from home. I will really miss the atmosphere and having lunch together.

**Johan**, thank you for giving me the chance to make this PhD my own; for always encouraging me to explore new ideas and directions, while making sure that I never went too far off track.

**Bastiaan**, thank you for your feedback and guidance, your kindness and your insights; for always remaining interested and supportive, even when you could not actively contribute anymore. I am very grateful to have known you.

**Wouter**, thank you for getting me into research, for always taking the time to have a chat and for stepping in as my second supervisor when Bastiaan passed away.

**Jacco & Eduardo**, thank you for being the best office mates I could hope for; for all the table tennis lessons, the trips together, and for all the after-work dinners, when we could not be bothered to cook something after a long day. Thank you, **Jacco**, for all the sessions brainstorming our research and exchanging Vim bindings. Thank you, **Eduardo**, for all your encouragement, and for making sure there was always a new puzzle or drawing on the whiteboard.

Many thanks to everyone in the **ST4LT** group for being welcoming and interested, and for listening to my rambling about functional programming. It was a pleasure to see the group outgrow Johan's office. Thank you as well to the **ST** group for letting me be an honorary member.

I really enjoyed spending time with colleagues outside off office hours, so thank you all for all the fun events and meet ups.

**David, Enrico, Upal, and Calò**, thank you for being my bouldering buddies.

**Lawrence**, thank you for always being in for a game of chess.

**Enas**, thank you for being my language buddy. وعشان علمتيني الفرق بين مختلف ومتخلف.

My visit to Chalmers was very important to me, at a time when I really needed a fresh perspective. During my time in Göteborg, I was able to finish and publish my first paper,

but I also managed to set a new course for the rest of my PhD. Thank you to all the people who made me feel welcome and introduced me to the joy of *fika. Tack så mycket.*

**Alex**, thank you for arranging everything to make my visit possible.

**Robert & Abhiroop**, thank you for being amazing office mates.

**Koen**, thank you for all the brainstorming sessions on your whiteboard and for helping me establish a new research direction.

---

I am very lucky to have had and met such great friends throughout my PhD. I loved going on vacation with you, to parties and festivals, but also just having dinners, playing board games, going bouldering, and simply enjoying each other's company.

**Anna**, while we met through the university, it always felt more like a happy coincidence to have such a great friend working in the same department. You know how to have a good time, and I loved singing, dancing, bouldering, and acting together. I also really appreciate your openness in discussing more serious matters, like mental health and the struggles of doing a PhD. Last but not least, thank you for introducing me to the wonderful people at the Twijnstraat.

**Barbara**, thank you for all the conversations, for getting me back into theater, and for being my *official friend!*

**Mirte**, thank you for our spontaneous bouldering sessions and for your help in designing the thesis cover; I am very happy with how it turned out!

Many thanks as well to the friends I already had, but who kept coming back:

**Mischa**, we have been friends for as long as I have memories, but at some point lost touch for a while. Thank you for rekindling our friendship.

**Paul & Camilo**, my favorite *krullenbollen*, thank you for still sharing so many great moments, from *vasteloavend* to our trips across the channel.

**Sophie, Laurens, Maruša, Camilo, and Lis**, thank you for being the highlight of my summer vacations; I hope we will have many more adventures together.

**Jenthe & Truusje**, thank you for being my bachelor besties.

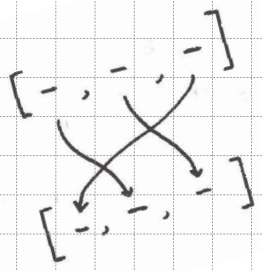
**Jonathan, Florian, and Jelle**, thank you for saving the world with me three times over.

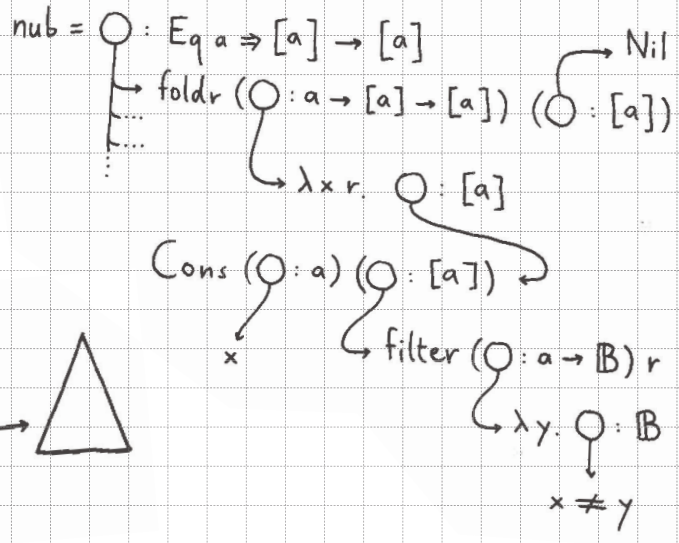
---

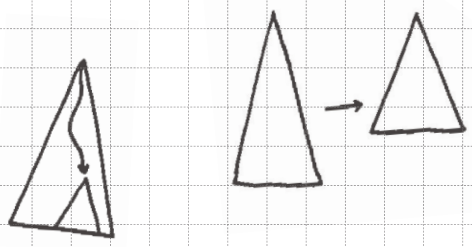
**Papa, Mama, Cas, en Stan**, ik kan jullie niet genoeg bedanken voor alle steun en liefde. Niet alleen voelt het altijd echt als thuiskomen wanneer ik weer in Maastricht ben, maar mede dankzij jullie had ik ook een heel fijn thuis hier in Utrecht.

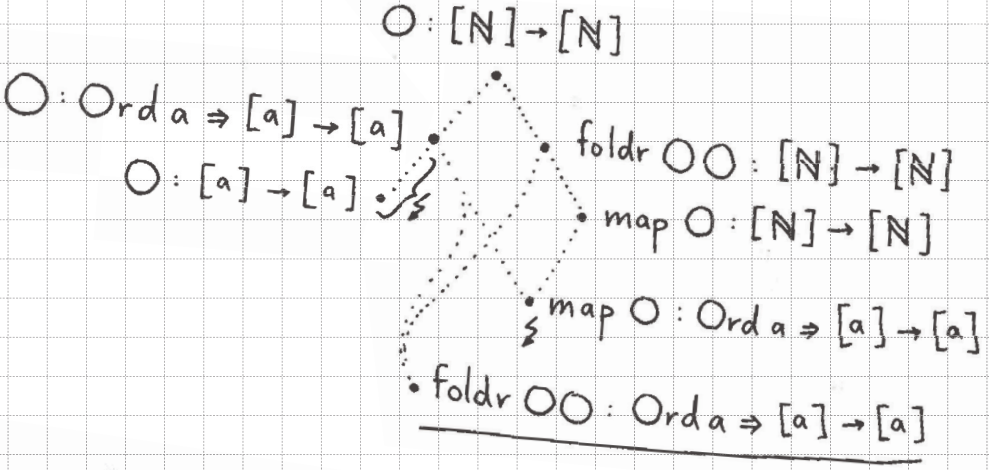
---

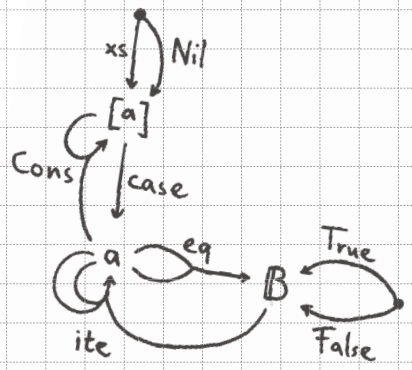
**Lis**, thank you for always supporting me, both when I was at the point of quitting, and when I eventually decided to keep going; for choosing to go on this adventure with me; for having listened to me explaining my research countless different ways; for making sure I will never call myself a doctor on an airplane.




nub = ○ : Eq a ⇒ [a] → [a]
foldr (○ : a → [a] → [a]) (○ : [a])
Nil
λx r.
○ : [a]
Cons (○ : a) (○ : [a])
x
filter (○ : a → 𝔹) r
λy. ○ : 𝔹
x ≠ y




○ : [ℕ] → [ℕ]
○ : Ord a ⇒ [a] → [a]
○ : [a] → [a]
foldr ○○ : [ℕ] → [ℕ]
map ○ : [ℕ] → [ℕ]
map ○ : Ord a ⇒ [a] → [a]
foldr ○○ : Ord a ⇒ [a] → [a]


xs
Nil
[a]
Cons
case
a
eq
𝔹
True
False
ite